F25-286-D-Splash

Project Team

Muhammad Tabish	22I-1540
Emama Noor	22I-2538
Haroon Waheed	22I-2565

Session 2022-2026

Supervised by

Mr.Junaid Ali Khan

Department of Computer Science

National University of Computer and Emerging Sciences
Islamabad, Pakistan

June, 2022

Contents

1 Introduction **1**

1.1 Problem Statement 1

1.2 Scope 2

1.3 Project Modules 3

1.3.1 Data Gathering and Influencer On-boarding 3

1.3.2 Recommendation System 4

1.3.3 AI Generation (Creatives) 4

1.3.4 Campaign Management and Chat System 4

1.3.5 Analytics Dashboard 5

1.3.6 Payment Gateway 5

1.3.7 Conversation Intelligence Assistant 6

1.3.8 Notification and Smart Alerts 6

1.3.9 Admin, User Profile, and Personalization 7

1.4 User Classes and Characteristics 7

2 Project Requirements **9**

2.1 Use-Case Diagram 9

2.2 Use Cases 10

2.2.1 Data Gathering and Influencer On-boarding 10

2.2.2 Recommendation System 15

2.2.3 AI Generation (Creatives) 20

2.2.4 Campaign Management and Chat System 24

2.2.5 Analytics Dashboard 29

2.2.6 Payment Gateway 34

2.2.7 Conversation Intelligence Assistant 39

2.2.8 Notification and Smart Alerts 43

2.2.9 Admin, User Profile, and Personalization 46

2.3 Functional Requirements 50

2.3.1 Data Gathering and Influencer On-boarding 50

2.3.2 Recommendation System 50

2.3.3 AI Generation (Creatives) 50

2.3.4 Campaign Management and Chat System 51

2.3.5 Analytics Dashboard 51
2.3.6 Payment Gateway 52
2.3.7 Conversation Intelligence Assistant 52
2.3.8 Notifications and Smart Alerts 53
2.3.9 Search and Discovery 53
2.4 Non-Functional Requirements 54
2.4.1 Performance Requirements 54
2.4.2 Availability Requirements 54
2.4.3 Security Requirements 54
2.4.4 Maintainability Requirements 55
2.4.5 Usability Requirements 55
2.5 Domain Model 55

3 System Overview **57**
3.1 Architectural Design 57
3.1.1 High-Level Architecture Layers 57
3.1.2 Modules Collaboration 58
3.1.3 System Collaboration Overview 59
3.1.4 Summary 59
3.1.5 Layered Architecture 59
3.1.6 Box and Line Diagram 60
3.1.7 Modules Collaboration Diagram 60
3.2 Design Models 61
3.2.1 Activity Diagrams 61
3.2.1.1 Data Gathering and Influencer On-boarding 61
3.2.1.2 Recommendation System 64
3.2.1.3 Ad Generation 66
3.2.1.4 Campaign Management and Chat System 68
3.2.1.5 Analytics Dashboard 72
3.2.1.6 Payment Gateway 75
3.2.1.7 Chatbot 79
3.2.1.8 Notification and Smart Alerts 82
3.2.1.9 Admin, User Profile and Personalization 85
3.2.2 Data Flow Diagram 89
3.2.2.1 Level 0 89
3.2.2.2 Level 1 90
3.2.2.3 Level 2 90
3.2.3 System Level Sequence Diagram 95
3.2.3.1 Data Gathering and Influencer On-boarding 95
3.2.3.2 Recommendation System 98

3.2.3.3 Ad Generation 100
3.2.3.4 Campaign Management and Chat 102
3.2.3.5 Analytics Dashboard 105
3.2.3.6 Payment Gateway 106
3.2.3.7 Conversation Intelligence Assistant 108
3.2.3.8 Notification and Smart Alerts 110
3.2.3.9 Admin, User Profile and Personalization 112
3.2.4 State Transition Diagram 115
3.2.4.1 Data Gathering and Influencer On-boarding 115
3.2.4.2 Recommendation System 118
3.2.4.3 Ad Generation 120
3.2.4.4 Campaign Management and Chat 122
3.2.4.5 Analytics Dashboard 126
3.2.4.6 Payment Gateway 129
3.2.4.7 Conversation Intelligence Assistant 132
3.2.4.8 Notification and Smart Alerts 136
3.2.4.9 Admin, User Profile and Personalization 139
3.3 Data Design 142
3.3.1 Schema and Entity Design 142
3.3.2 Data Storage and Indexing Strategy 143
3.3.3 Backup and Recovery Plan 144
3.3.4 Access Control and Security Model 144
3.3.5 Scalability and Migration Plan 144
3.4 Deployment Diagram 145

4 Implementation and Testing **147**
4.1 Overview 147
4.2 Algorithm Design 147
4.2.1 End-to-End Encryption Algorithm 147
4.2.2 Conversation Key Derivation Algorithm 149
4.2.3 Search and Matching Algorithm 149
4.2.4 CSV Parsing Algorithm 150
4.2.5 File Upload and Encryption Algorithm 150
4.2.6 User Registration Algorithm 151
4.2.7 User Login Algorithm 151
4.2.8 Conversation Creation Algorithm 151
4.2.9 Message Sending Algorithm 152
4.2.9.1 Message Decryption and Display Algorithm 152
4.2.9.2 Unread Message Count Algorithm 153
4.2.9.3 Follower Count Parsing Algorithm 154

4.2.9.4 Recommendation Engine Algorithm 155
4.2.9.5 Profile Claim Submission Algorithm 157
4.2.9.6 Profile Claim Review Algorithm (Admin) 158
4.2.9.7 Profile Completeness Calculation Algorithm 159
4.2.9.8 Dashboard Statistics Calculation Algorithm 160
4.2.9.9 User Approval Algorithm (Admin) 161
4.2.9.10 CSV Matching Algorithm for Registration 161
4.2.9.11 Engagement Rate Calculation Algorithm 162
4.2.9.12 Unread Message Count Algorithm (Per Conversation) . 163
4.3 External APIs and SDKs 164
4.4 Unit Testing 165
4.4.1 TC_ENC_001: Text Encryption and Decryption 166
4.4.2 TC_ENC_002: Binary File Encryption and Decryption 167
4.4.3 TC_ENC_003: Voice Note Encryption 168
4.4.4 TC_ENC_004: Key Derivation Consistency 169
4.4.5 TC_ENC_005: Backward Compatibility - Unencrypted Messages 170
4.4.6 TC_AUTH_001: User Registration - Valid Data 171
4.4.7 TC_AUTH_002: User Login - Valid Credentials 172
4.4.8 TC_AUTH_003: User Login - Invalid Password 173
4.4.9 TC_AUTH_004: User Registration - CSV Matching 174
4.4.10 TC_SEARCH_001: Search - Exact Name Match 175
4.4.11 TC_SEARCH_002: Search - Substring Match 176
4.4.12 TC_SEARCH_003: Search - Multi-word Query 177
4.4.13 TC_SEARCH_004: Search - Minimum Length Validation 178
4.4.14 TC_REC_001: Recommendation Engine - Category Match 179
4.4.15 TC_REC_002: Recommendation Engine - Engagement Quality Scoring 180
4.4.16 TC_REC_003: Recommendation Engine - Gender-specific Matching 181
4.4.17 TC_CLAIM_001: Profile Claim Submission 182
4.4.18 TC_CLAIM_002: Profile Claim Approval (Admin) 183
4.4.19 TC_CLAIM_003: Profile Claim Rejection (Admin) 184
4.4.20 TC_ADMIN_001: User Approval (Admin) 185
4.4.21 TC_ADMIN_002: User Suspension (Admin) 186
4.4.22 TC_ADMIN_003: Dashboard Statistics - Admin 187
4.4.23 TC_PROFILE_001: Profile Completeness Calculation 188
4.4.24 TC_MSG_001: Send Encrypted Message 189
4.4.25 TC_MSG_002: Receive and Decrypt Message 190
4.4.26 TC_MSG_003: Unread Message Count 191
4.4.27 TC_CSV_001: CSV Parsing with Parentheses 192

4.4.28 TC_FILE_001: File Upload - Image 193
4.4.29 TC_FILE_002: File Upload - Size Validation 194
4.4.30 TC_CONV_001: Conversation Creation 195
4.4.31 TC_CONV_002: Duplicate Conversation Prevention 196
4.4.32 Testing Summary 198

References **201**

List of Figures

2.1 Usecase Diagram 9
2.2 Domain Model 56

3.1 Architecture Diagram 59
3.2 Box and Line Diagram 60
3.3 Modules Collaboration Diagram 60
3.4 Activity Diagram (Import Dataset) 61
3.5 Activity Diagram (Claim Existing Profile) 62
3.6 Activity Diagram (Verify User) 63
3.7 Activity Diagram (Generate Recommendations) 64
3.8 Activity Diagram (Shortlist Influencers) 65
3.9 Activity Diagram (Generate Creatives) 66
3.10 Activity Diagram (Review and Edit Creatives) 67
3.11 Activity Diagram (Open Campaign Workspace) 68
3.12 Activity Diagram (Chat and File Exchange) 69
3.13 Activity Diagram (Submit and Approve Deliverables) 70
3.14 Activity Diagram (Milestone Tracking and Reminders) 71
3.15 Activity Diagram (View Live Campaign Performance) 72
3.16 Activity Diagram (Compare Creators and Campaigns) 73
3.17 Activity Diagram (Generate Insight Prompts) 74
3.18 Activity Diagram (Create Escrow for Campaign) 75
3.19 Activity Diagram (Release Payment On Approval) 76
3.20 Activity Diagram (Manage Subscriptions) 77
3.21 Activity Diagram (Summarize Chats and Briefs) 79
3.22 Activity Diagram (Suggest Replies and Brief Improvements) 80
3.23 Activity Diagram (Follow Up Nudges) 81
3.24 Activity Diagram (Real Time Campaign Alerts) 82
3.25 Activity Diagram (Priority Sorting) 83
3.26 Activity Diagram (Preference Management) 84
3.27 Activity Diagram (Moderate Users and Disputes) 85
3.28 Activity Diagram (Manage Plans and Billing) 86
3.29 Activity Diagram (Configure Roles and Access) 87
3.30 Activity Diagram (Generate Personalized Dashboards) 88

3.31 Data Flow Diagram- level 0 89
3.32 Data Flow Diagram level 1 90
3.33 Data Flow Diagram level 2 90
3.34 Data Flow Diagram level 2 91
3.35 Data Flow Diagram level 2 91
3.36 Data Flow Diagram level 2 91
3.37 Data Flow Diagram level 2 92
3.38 Data Flow Diagram level 2 93
3.39 Data Flow Diagram level 2 93
3.40 Data Flow Diagram level 2 94
3.41 Data Flow Diagram level 2 94
3.42 Import Dataset 95
3.43 Claim Existing Profile 96
3.44 Verify User 97
3.45 Generate Recommendations 98
3.46 Shortlist Influencers 99
3.47 Generate Creatives 100
3.48 Review and Edit Creatives 101
3.49 Open Campaign Workspace 102
3.50 Chat and File Exchange 102
3.51 Submit and Approve Deliverables 103
3.52 Milestone Tracking and Reminders 104
3.53 View Live Campaign Performance 105
3.54 Compare Creators and Campaigns 105
3.55 Generate Insight Prompts 106
3.56 Create Escrow for Campaign 106
3.57 Release Payment on Approval 107
3.58 Manage Subscriptions 108
3.59 Summarize Chats and Briefs 108
3.60 Suggest Replies and Brief Improvements 109
3.61 Follow Up Nudges 109
3.62 Real Time Campaign Alerts 110
3.63 Priority Sorting 110
3.64 Preference Management 111
3.65 Moderate Users and Disputes 112
3.66 Manage Plans and Billing 113
3.67 Configure Roles and Access 114
3.68 Generate Personalized Dashboards 114
3.69 Import Dataset 115
3.70 Claim Existing Profile 116

3.71 Verify User 117
3.72 Generate Recommendations 118
3.73 Shortlist Influencers 119
3.74 Generate Creatives 120
3.75 Review and Edit Creatives 121
3.76 Open Campaign Workspace 122
3.77 Chat and File Exchange 123
3.78 Submit and Approve Deliverables 124
3.79 Milestone Tracking and Reminders 125
3.80 View Live Campaign Performance 126
3.81 Compare Creators and Campaigns 127
3.82 Generate Insight Prompts 128
3.83 Create Escrow for Campaign 129
3.84 Release Payment on Approval 130
3.85 Manage Subscriptions 131
3.86 Summarize Chats and Briefs 132
3.87 Suggest Replies and Brief Improvements 133
3.88 Follow Up Nudges 134
3.89 Real Time Campaign Alerts 136
3.90 Priority Sorting 137
3.91 Preference Management 138
3.92 Moderate Users and Disputes 139
3.93 Manage Plans and Billing 140
3.94 Configure Roles and Access 141
3.95 Generate Personalized Dashboards 142
3.96 Deployment Diagram 145
3.97 Deployment Diagram 146

List of Tables

1.1 User Classes and Characteristics for Splash 8

2.1 UC-DG-01: Sign Up and Account Match 10
2.2 UC-DG-02: Claim Existing Profile 11
2.3 UC-DG-03: Create New Profile 12
2.4 UC-DG-04: Update Profile and Refresh Browse 13
2.5 UC-RS-01: Create Campaign Brief for Matching 15
2.6 UC-RS-02: Generate Recommended Influencers 16
2.7 UC-RS-03: Filter and Shortlist Influencers 17
2.8 UC-RS-04: Learn from Historical Outcomes 18
2.9 UC-AI-01: Draft Campaign Creatives 20
2.10 UC-AI-02: Personalize Creative to Influencer Tone 21
2.11 UC-AI-03: Review and Edit Creative 22
2.12 UC-CM-01: Open Campaign Workspace 24
2.13 UC-CM-02: Structured Chat and File Exchange 25
2.14 UC-CM-03: Submit and Approve Deliverables 26
2.15 UC-CM-04: Milestone Tracking and Reminders 27
2.16 UC-AD-01: View Live Campaign Performance 29
2.17 UC-AD-02: Compare Creators and Campaigns 30
2.18 UC-AD-03: Export and Share Reports 31
2.19 UC-AD-04: Generate Insight Prompts 32
2.20 UC-PG-01: Create Escrow for Campaign 34
2.21 UC-PG-02: Release Payment on Approval 35
2.22 UC-PG-03: Manage Subscriptions 36
2.23 UC-PG-04: Enforce Plan Entitlements 37
2.24 UC-CI-01: Summarize Chats and Briefs 39
2.25 UC-CI-02: Suggest Replies and Brief Improvements 40
2.26 UC-CI-03: Follow-Up Nudges 41
2.27 UC-NA-01: Real-Time Campaign Alerts 43
2.28 UC-NA-02: Priority Sorting 44
2.29 UC-NA-03: Preference Management 45
2.30 UC-ADP-01: Moderate Users and Disputes 46
2.31 UC-ADP-02: Manage Plans and Billing 47

2.32 UC-ADP-03: Configure Roles and Access 48
2.33 UC-ADP-04: Generate Personalized Dashboards 49

4.1 APIs and SDKs Used in the Splash Platform 164
4.2 Test Case TC_ENC_001: Text Encryption and Decryption 166
4.3 Test Case TC_ENC_002: Binary File Encryption and Decryption 167
4.4 Test Case TC_ENC_003: Voice Note Encryption 168
4.5 Test Case TC_ENC_004: Key Derivation Consistency 169
4.6 Test Case TC_ENC_005: Backward Compatibility for Unencrypted Messages 170
4.7 Test Case TC_AUTH_001: User Registration with Valid Data 171
4.8 Test Case TC_AUTH_002: User Login with Valid Credentials 172
4.9 Test Case TC_AUTH_003: User Login with Invalid Password 173
4.10 Test Case TC_AUTH_004: User Registration with CSV Matching 174
4.11 Test Case TC_SEARCH_001: Search with Exact Name Match 175
4.12 Test Case TC_SEARCH_002: Search with Substring Match 176
4.13 Test Case TC_SEARCH_003: Search with Multi-word Query 177
4.14 Test Case TC_SEARCH_004: Search Minimum Length Validation 178
4.15 Test Case TC_REC_001: Recommendation Engine Category Match . . . 179
4.16 Test Case TC_REC_002: Recommendation Engine Engagement Quality Scoring 180
4.17 Test Case TC_REC_003: Recommendation Engine Gender-specific Matching 181
4.18 Test Case TC_CLAIM_001: Profile Claim Submission 182
4.19 Test Case TC_CLAIM_002: Profile Claim Approval (Admin) 183
4.20 Test Case TC_CLAIM_003: Profile Claim Rejection (Admin) 184
4.21 Test Case TC_ADMIN_001: User Approval (Admin) 185
4.22 Test Case TC_ADMIN_002: User Suspension (Admin) 186
4.23 Test Case TC_ADMIN_003: Dashboard Statistics for Admin 187
4.24 Test Case TC_PROFILE_001: Profile Completeness Calculation 188
4.25 Test Case TC_MSG_001: Send Encrypted Message 189
4.26 Test Case TC_MSG_002: Receive and Decrypt Message 190
4.27 Test Case TC_MSG_003: Unread Message Count 191
4.28 Test Case TC_CSV_001: CSV Parsing with Parentheses 192
4.29 Test Case TC_FILE_001: File Upload of Image 193
4.30 Test Case TC_FILE_002: File Upload Size Validation 194
4.31 Test Case TC_CONV_001: Conversation Creation 195
4.32 Test Case TC_CONV_002: Duplicate Conversation Prevention 196
4.33 Testing Summary 198

Chapter 1

Introduction

Splash is an AI-powered brand–influencer collaboration platform that bridges the gap between brands seeking partnerships and creators looking for professional, transparent, and authentic opportunities. The platform streamlines the entire influencer marketing lifecycle through discovery, campaign management, collaboration, and analytics integrated into a single digital workspace.

Influencer marketing is one of the leading channels for brand promotion, but existing solutions often rely on fragmented tools with manual coordination. Brands must juggle between multiple applications for creator discovery, contract handling, communication, and performance tracking. Similarly, influencers too face difficulties in being transparent and managing collaborations with different brands. This scattered approach leads to inefficiencies and ineffectiveness, lack of accountability, and limited results for marketing campaigns. [1] [5] Splash addresses this challenge by introducing automation, verified data analytics, and AI assistance at every stage of the process. It will allows brands to pin point relevant creators, manage deliverables within a shared campaign space, and monitor real-time performance through quantified analytics. The platform also empowers influencers to build trust by presenting verified and authentic metrics and managing their collaborations in one place as well.

The vision behind Splash is to make influencer marketing transparent, data-driven, and accessible for businesses/brands of all sizes. By integrating recommendation systems, creative support, and real-time campaign insights all under one roof, Splash simplifies complex workflows and ensures measurable results for both brands and creators.

1.1 Problem Statement

The influencer marketing industry has grown rapidly, becoming an essential part of modern brand promotion strategies. Despite this growth, both brands and influencers face

challenges that hinder effective collaboration. The problem lies in the fragmented nature of current influencer brand marketing workflows. Brands often rely on different tools for influencer discovery, communication, contract management, campaign tracking, and payments. This disjointed approach creates inefficiencies, delays, and data inconsistencies throughout the campaign lifecycle.

From the brand's point of view, identifying the correct influencers remains a major challenge. Many existing platforms focus on follower counts rather than engagement authenticity or audience relevance. Eventually leading to partnerships that may not align with brand goals, resulting in ineffective campaign and waste of resources, or at worse negative impact on the brand itself. In addition, brands lack real-time and quantifiable insights into influencer reliability, audience demographics, and return on investment.

On the other hand, Influencers struggle to present themselves credibly to brands. They also lack tools that help them manage multiple collaborations efficiently and in one place. Negotiating deliverables, tracking payments, and maintaining transparency with brands often require manual effort and informal communication channels. These issues not only affect efficiency, but also erode trust between them both.

The lack of a unified, data-driven solution for influencer marketing results in confusion, miscommunication, and unreliable performance measures. This shows a clear need for a platform that brings together verified influencer data, AI-powered matchmaking, campaign management, and analytics within one system. Splash solves these challenges by offering an integrated ecosystem that improves transparency, simplifies collaboration, and ensures measurable results for both parties.

1.2 Scope

The scope of Splash encompasses the complete influencer marketing lifecycle, from influencer on-boarding and campaign creation to performance tracking and post-campaign analytics. The platform serves both the brands and the influencers by providing an integrated and shared environment in which discovery, collaboration, and management occur seamlessly. It intends to enhance transparency, efficiency, and effectiveness throughout the process.

Splash enables brands to search for relevant influencers based on metrics such as engagement rate, content category, and audience demographics. It's recommendation engine ensures that brand–influencer matches are aligned with campaign objectives first and foremost. Once a collaboration is established, Splash provides a shared campaign workspace where both teams can manage campaign briefs, share different assets, track deliverables, and communicate in real time.

For influencers, the platform offers a professional dashboard that allows them to manage brand deals, showcase verified insights of their audience, and receive their payments securely. The objective is to simplify and unify communication and campaign tracking while ensuring that influencers are rewarded promptly.

Additionally, Splash incorporates AI assistance in content creation, and campaign optimization. Its analytics dashboard provides comprehensive performance metrics, allowing brands to evaluate campaign success and predict future outcomes. The system has multiple roles, which also includes administrators who oversee platform operations and handle dispute resolution.

Splash encompasses influencer on-boarding, campaign management, AI-driven recommendations, creative assistance, communication, analytics, and secure transactions. The platform is designed to be scalable, user-friendly, and adaptable for marketing needs across various industries.

1.3 Project Modules

Our system is divided into the following modules:

1.3.1 Data Gathering and Influencer On-boarding

This module builds the foundation of the platform by integrating influencers into Splash's verified creator database. When a new influencer signs up, the system first sees for an existing record in the dataset. If a match is found, the influencer can submit a request to claim and verify said profile; if not, a new one will be created. After being verified, influencers gain access to edit their details such as categories, and portfolio samples. These updates are then reflected across the platform so that discovery and recommendation engines operate on up-to-date data.

Key Features:

- Detection of existing influencer profiles during sign up.
- Claim and verification process for ownership of existing accounts.
- Creation of new verified profiles when no match is found.
- Continuous refresh of the dataset with up-to-date data.

1.3.2 Recommendation System

This module powers the intelligent pairing between brands and influencers using verified metrics and campaign context. When a brand creates a campaign brief, the system checks information like objectives, category, target audience, and budget, then connects them with the most relevant influencer profiles. It considers engagement quality, niche alignment, and past collaboration outcomes to generate a ranked suggestion of multiple potential influencers.

Key Features:

- Matching of influencer profiles to campaign briefs using verified engagement data.
- Compatibility evaluation based on audience demographics, category, and reliability.
- Learning from past collaborations to refine future recommendations.
- Ranked influencer lists with transparent match scores.
- Advanced filtering to help brands select the best fit efficiently.

1.3.3 AI Generation (Creatives)

This module assists brands and influencers in creating content, generate ideas, captions, and visual concepts. It examines campaign briefs, target audience details, and influencer tone to produce draft creatives that align with brand guidelines. The objective is not to replace human creativity but to add to it by providing suggestions and samples that save time. Both brands and creators can edit and personalize these AI-generated drafts before publishing.

Key Features:

- Generation of AI-based caption and content suggestions aligned with campaign objectives.
- Maintenance of brand consistency through tone and theme matching.
- Manual review and customization of AI-generated drafts.

1.3.4 Campaign Management and Chat System

It manages all stages of a campaign's lifecycle, on a shared workspace where brands and influencers both can collaborate. After a partnership is formed, the campaign workspace

tracks briefs, content submissions, approvals, and milestones. The integrated chat supports structured communication, file sharing, and automated reminders, reducing the need for external tools and manual work. This unified single approach ensures transparency, accountability, and coordination between the brands and influencers.

Key Features:

- Centralized campaign workspace for briefs, tasks, and deliverables.
- Real-time integrated chat for communication.
- File sharing and media upload options for creative submissions.
- Automated reminders for pending approvals and upcoming deadlines.
- Transparent activity logs for accountability and progress tracking.

1.3.5 Analytics Dashboard

This system provides brands with a complete analytical view of their campaigns, allowing them to monitor performance and measure return on investment in real time. It collects metrics such as reach, and engagement rate to evaluate the effectiveness of each campaign. The dashboard also visualizes historical trends and predictive insights, helping brands to make informed decisions for future collaborations. The data is presented clearly through interactive charts with filters.

Key Features:

- Tracking of key campaign metrics such as engagement, impressions, and reach.
- Real-time performance dashboards with visual and interactive analytics.
- Calculation of estimated ROI and comparison of performance across campaigns.
- Historical trend analysis and predictive insights for decision making.
- Allow to export charts and data with custom range and filters.

1.3.6 Payment Gateway

This module handles all financial related transactions on the site, covering both campaign-related payments and subscription-based billing. For campaign payments, Splash ensures secure and transparent transactions using an escrow-style flow where funds are held until deliverables are approved.

Splash's platform services follows a subscription model that offers multiple plans suited to various business needs. The **Basic Plan** is free and allows users to explore influencers and generate up to five ad creatives with a watermark but without access to chat or analytics. The **Tier 1** plan removes watermarks, offers ten ad generations, and grants access to analytics and brand-influencer chat. The **Premium Plan** provides unlimited creative generation, unrestricted analytics, direct messaging, and early access to beta features. **Team** plans allow organizations to manage multiple users under a single workspace with shared billing. **Key Features:**

- Escrow-based payment system for secure brand–influencer transactions.
- Tiered subscription model with increasing access and privileges.
- Basic, Plan 1, and Premium options tailored for different marketing needs.
- Support for team and enterprise billing under one workspace.

1.3.7 Conversation Intelligence Assistant

This module is an AI-powered assistant that assists both brands and influencers during their interactions on the platform. It analyzes campaign discussions, briefs, and messages to glean relevant information, summarize key points, and suggest appropriate responses. The assistant also helps brands generate better campaign briefs and assists influencers in structuring professional replies. By giving real-time recommendations and reminders, it enhances communication quality and makes sure that no critical details are missed during collaborations.

Key Features:

- AI-powered conversation summaries and key point extraction.
- Smart response suggestions based on the message context.
- Campaign brief drafting assistance for brands.
- Real-time reminders and follow-ups.

1.3.8 Notification and Smart Alerts

This module keeps users stay informed and updated about all critical activities on the platform. It sends real-time notifications for campaign updates, message alerts, payment statuses, and deadline reminders. Smart alert logic prioritizes notifications based on significance, ensuring that users focus on keys actions first. The system offers both in-app

and email alerts, allowing brands and influencers to remain connected and synchronized even when offline.

Key Features:

- Real-time notifications for campaign milestones and updates.
- Alerts for new messages, payments, and pending approvals.
- Smart prioritization system for tasks and milestones.
- In-app and email alerts for continuous connectivity.

1.3.9 Admin, User Profile, and Personalization

This module handles platform administration, user management, and experience personalization. The admin panel allows system administrators to monitor activity, resolve disputes, manage subscription plans, and maintain data integrity. Each user has a personalized profile that adapts to their activity, preferences, and role. Brands see campaign analytics and influencer recommendations tailored to their niche, while influencers view insights relevant to their performance. This personalization ensures that every user experiences a dashboard suited to their objectives and workflow.

Key Features:

- Centralized admin panel for monitoring users, payments, and campaigns.
- Tools for managing subscriptions, reports, and dispute resolution.
- Personalized dashboards based on user type and activity.
- Role-based access control for security and data segregation.
- Adaptive recommendations that improve with user behavior.

1.4 User Classes and Characteristics

This section lists the major user classes of the Splash platform and outlines their roles, responsibilities, and characteristics. Each user group interacts with the system in various capacities to ensure seamless operation of campaigns, collaborations, and administration.

User Class	Description
Brand / Marketing Team	Represents companies or brands that want to develop marketing campaigns, search for relevant influencers, and manage collaborations. They will define campaign objectives, budgets, and timelines, and monitor analytics through the performance dashboard. They can contact influencers directly and make secure payments within the platform.
Influencer / Creator	Verified creators who join the platform to collaborate with brands. They can manage their profiles, submit deliverables, and review campaign briefs. Influencers can generate creative content using AI tools, and track payments and campaign performance through their dashboards.
Platform Administrator	Responsible for managing overall platform operations. Administrators monitor user activity, handle dispute resolution, manage subscription plans, and ensure compliance with platform policies. They also have authority to approve or deactivate users and maintain system-wide data integrity.
Team Member (Agency/Brand Sub-user)	Works for a brand or agency workspace with delegated permissions. They help in campaign creation, influencer communication, and data analysis. They have restricted access based on assigned roles by the brand/agency but can contribute to collaborative marketing tasks.
System AI Assistant	Assists users by generating campaign creatives, analyzing conversations, recommending influencers, and summarizing analytics or conversations. It enhances user experience and efficiency by automating repetitive and manual tasks, and offers intelligent insights across modules.

Table 1.1: User Classes and Characteristics for Splash

Chapter 2

Project Requirements

2.1 Use-Case Diagram

This section contains the use-case diagrams for the Splash platform, which shows the interactions between actors and system functionalities. Each figure provide a high-level view of module-specific operations and relationships.

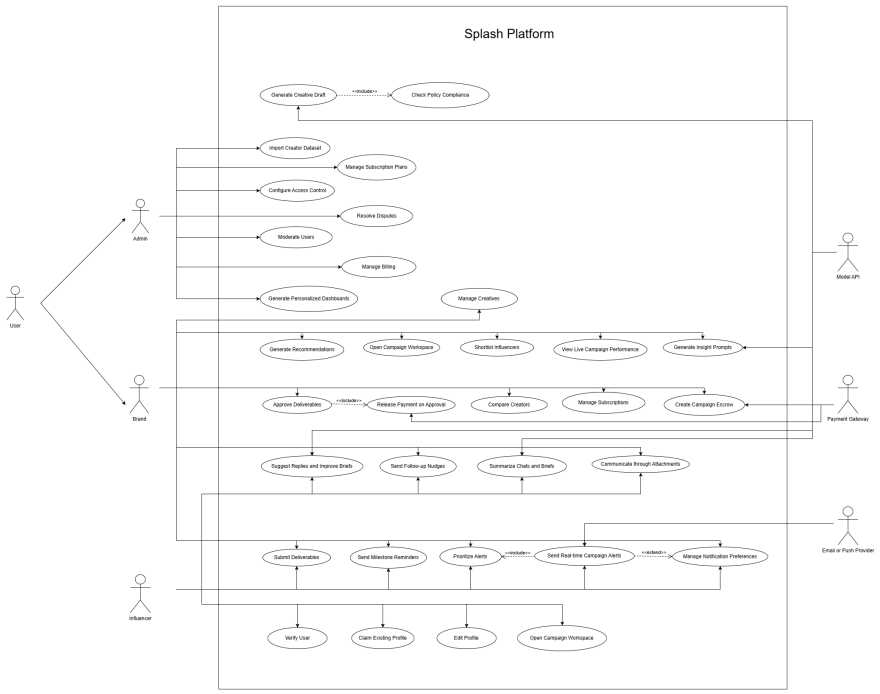


Figure 2.1: Usecase Diagram

2.2 Use Cases

2.2.1 Data Gathering and Influencer On-boarding

Fields	**Description**
Use Case ID	UC-DG-01
Use Case Name	Sign Up and Account Match
Scope	Splash AI-Powered Brand–Influencer Collaboration Platform
Level	Primary
Primary Actor	Influencer / Creator
Stakeholders and Interests	**Influencer:** Wants to sign up smoothly and avoid duplicate profiles. **System:** Ensures consistency by checking the existing creator dataset before creating new records.
Preconditions	Influencer provides a valid email and at least one public handle.
Postconditions	Influencer either claims an existing verified profile or a new profile is created and linked to the dataset.
Main Success Scenario	1. Influencer opens the sign up form. 2. System requests email and public handle. 3. System searches the existing creator dataset. 4. If a match exists, influencer proceeds to claim; otherwise a new profile is created. 5. Influencer accesses their verified profile.
Extensions	2a. Invalid or missing data — system prompts correction. 3a. Multiple matches — system requests additional identifiers.
Special Requirements	Validation and dataset check should complete within 3 seconds. Duplicate profile creation must be prevented.
Technology and Data Variation List	CSV based creator dataset ingestion and cleaning. SQLite for user accounts and profile ownership. Variations: handle formatting, missing fields, duplicate rows.
Frequency of Occurrence	One time per influencer during on-boarding.
Open Issues	Handling partial matches and verification turnaround time.

Table 2.1: UC-DG-01: Sign Up and Account Match

Fields	Description
Use Case ID	UC-DG-02
Use Case Name	Claim Existing Profile
Scope	Splash AI-Powered Brand–Influencer Collaboration Platform
Level	Primary
Primary Actor	Influencer / Creator
Stakeholders and Interests	**Influencer:** Wants to gain control over an existing dataset profile that represents them. **System:** Ensures ownership claims are verified and linked to the correct user account. **Administrator:** Reviews claims and maintains data authenticity.
Preconditions	The influencer's profile already exists in the dataset; the user has completed the sign-up and match process.
Postconditions	The influencer's account is linked to the verified dataset entry and marked as an authenticated profile.
Main Success Scenario	1. Influencer submits a claim request for an existing dataset profile. 2. System logs the claim and notifies the administrator. 3. Administrator reviews the submitted verification evidence. 4. Administrator approves ownership. 5. Influencer gains edit access to the verified profile.
Extensions	3a. Insufficient evidence — administrator requests additional proof. 4a. Claim rejected — influencer notified with reasons and can re-submit.
Special Requirements	Claims must be reviewed within 48 hours. Ownership verification must rely on verifiable identifiers such as email or linked handles.
Technology and Data Variation List	CSV dataset entries are cross-referenced with SQLite user data. Variations: duplicate claims, incomplete verification evidence.
Frequency of Occurrence	Occasional, when influencers discover pre-existing profiles.
Open Issues	Handling multiple claims for the same dataset entry and dispute resolution workflow.

Table 2.2: UC-DG-02: Claim Existing Profile

Fields	Description
Use Case ID	UC-DG-03
Use Case Name	Create New Profile
Scope	Splash AI-Powered Brand–Influencer Collaboration Platform
Level	Primary
Primary Actor	Influencer / Creator
Stakeholders and Interests	**Influencer:** Wants to create a verified and professional profile to showcase skills and audience data. **System:** Needs to register the new profile without duplicating existing records and maintain data accuracy.
Preconditions	No matching profile found during the dataset search. Influencer must provide required details and consent for verification.
Postconditions	A new verified profile is successfully created and stored in the system database, ready for discovery.
Main Success Scenario	1. Influencer selects the option to create a new profile. 2. System prompts for personal, category, and portfolio details. 3. Influencer fills out all required information and submits. 4. System validates entries and creates a new record. 5. The profile is added to the dataset and flagged for verification.
Extensions	2a. Incomplete data submission — system prompts influencer to fill missing fields. 4a. Validation error — user notified and asked to correct entries.
Special Requirements	Data validation must ensure unique email and handle. Verification must occur before profile becomes publicly searchable.
Technology and Data Variation List	SQLite used for profile creation and verification tracking. CSV dataset updated periodically with new verified entries.
Frequency of Occurrence	Common, especially during initial on-boarding and new influencer registrations.
Open Issues	Handling incomplete verification submissions and syncing delays between SQLite and CSV datasets.

Table 2.3: UC-DG-03: Create New Profile

Fields	Description
Use Case ID	UC-DG-04
Use Case Name	Update Profile and Refresh Browse
Scope	Splash AI-Powered Brand–Influencer Collaboration Platform
Level	Primary
Primary Actor	Influencer / Creator
Stakeholders and Interests	**Influencer:** Wants to keep their information current to maintain discoverability and brand trust. **System:** Must ensure updated data is validated and reflected across browse and recommendation modules.
Preconditions	Influencer has an approved and verified profile. The system is connected to the dataset and active.
Postconditions	Updated information is stored in SQLite and synchronized with the CSV dataset; browse and recommendation indexes are refreshed.
Main Success Scenario	1. Influencer accesses their profile dashboard. 2. Influencer edits details such as category, portfolio items, or audience insights. 3. System validates all updated entries. 4. System saves the changes to the SQLite database. 5. The CSV dataset and browse index are updated accordingly.
Extensions	3a. Invalid or incomplete data — system displays an error message. 4a. Update fails due to sync issue — system retries or alerts admin.
Special Requirements	Updates must propagate to browse and recommendation modules in under one minute. Version control of profile edits should be maintained.
Technology and Data Variation List	SQLite for live profile updates and CSV dataset for browse index synchronization. Variations: missing sync cycles, partial updates, or concurrent edits.
Frequency of Occurrence	Frequent; influencers may update details multiple times a month.
Open Issues	Handling synchronization conflicts and ensuring real-time updates to dependent modules.

Table 2.4: UC-DG-04: Update Profile and Refresh Browse

2.2.2 Recommendation System

Fields	Description
Use Case ID	UC-RS-01
Use Case Name	Create Campaign Brief for Matching
Scope	Splash AI-Powered Brand–Influencer Collaboration Platform
Level	Primary
Primary Actor	Brand / Marketing Team
Stakeholders and Interests	**Brand:** Wants to create a clear campaign brief to find suitable influencers efficiently. **System:** Uses structured inputs to enable accurate AI-driven matching. **Team Members:** Support in adding campaign details and reviewing requirements.
Preconditions	The brand or team member must be logged into a verified account with an active subscription.
Postconditions	A campaign brief is successfully created, saved in the database, and ready for influencer matching.
Main Success Scenario	1. Brand selects "Create Campaign Brief." 2. System displays a form requesting campaign details such as category, objectives, target audience, and budget range. 3. Brand fills out all required information. 4. System validates and stores the data in SQLite. 5. The brief becomes available for the recommendation engine to process.
Extensions	2a. Missing mandatory fields — system prompts the user to complete them. 4a. Validation fails — system displays an error message and saves a draft copy.
Special Requirements	The system must support multiple campaign drafts per brand. Auto-save functionality should prevent data loss.
Technology and Data Variation List	SQLite for campaign storage. Variations: incomplete fields, inconsistent category entries, duplicate brief names.
Frequency of Occurrence	Frequent; occurs each time a new campaign is launched.
Open Issues	Need for campaign templates to simplify repeated campaigns.

Table 2.5: UC-RS-01: Create Campaign Brief for Matching

Fields	Description
Use Case ID	UC-RS-02
Use Case Name	Generate Recommended Influencers
Scope	Splash AI-Powered Brand–Influencer Collaboration Platform
Level	Primary
Primary Actor	System AI Assistant
Stakeholders and Interests	**Brand:** Wants accurate and relevant influencer recommendations for active campaigns. **System:** Aims to generate data-driven results by comparing campaign briefs with verified influencer profiles.
Preconditions	A valid campaign brief has been created and stored in the database. Influencer dataset must be up-to-date.
Postconditions	A ranked list of recommended influencers is displayed to the brand, ready for shortlisting and review.
Main Success Scenario	1. Brand selects "Generate Recommendations." 2. System analyzes the campaign brief data and retrieves relevant influencer metrics. 3. The AI model calculates compatibility scores based on engagement rate, audience match, and content relevance. 4. System generates a ranked list of influencers with match scores. 5. Results are displayed to the brand in the campaign dashboard.
Extensions	2a. Dataset connection failure — system shows an error and retries after delay. 4a. No influencers meet criteria — system expands search or suggests similar categories.
Special Requirements	The recommendation process must complete in under 10 seconds for optimal user experience. AI scoring weights should be configurable by admins.
Technology and Data Variation List	CSV dataset of influencer profiles. SQLite for campaign brief retrieval and result caching. Variations: missing influencer metrics or incomplete data rows.
Frequency of Occurrence	Frequent; typically triggered for each new campaign brief.
Open Issues	Fine-tuning of AI scoring parameters for diverse industries.

Table 2.6: UC-RS-02: Generate Recommended Influencers

Fields	Description
Use Case ID	UC-RS-03
Use Case Name	Filter and Shortlist Influencers
Scope	Splash AI-Powered Brand–Influencer Collaboration Platform
Level	Primary
Primary Actor	Brand / Marketing Team
Stakeholders and Interests	**Brand:** Needs to refine influencer results to find the best fit for campaign goals. **System:** Must allow multi-criteria filtering without compromising data accuracy or performance.
Preconditions	Influencer recommendations have been successfully generated and displayed to the brand.
Postconditions	A refined and saved shortlist of influencers is created for the campaign.
Main Success Scenario	1. Brand opens the list of recommended influencers. 2. Brand applies filters such as location, niche, audience demographics, or engagement rate. 3. System dynamically updates the list based on the applied filters. 4. Brand selects preferred influencers and adds them to a shortlist. 5. The shortlist is saved for further communication and campaign planning.
Extensions	2a. Filters return no results — system suggests expanding search criteria. 4a. Shortlist already exists — system asks if brand wants to overwrite or merge results.
Special Requirements	Filtering and shortlisting must update results instantly and support multiple filter combinations.
Technology and Data Variation List	Influencer data retrieved from the CSV dataset. SQLite used for storing shortlist metadata and user preferences.
Frequency of Occurrence	Frequent; used for almost every campaign setup.
Open Issues	Integration of AI-assisted suggestions for dynamic filter recommendations.

Table 2.7: UC-RS-03: Filter and Shortlist Influencers

Fields	Description
Use Case ID	UC-RS-04
Use Case Name	Learn from Historical Outcomes
Scope	Splash AI-Powered Brand–Influencer Collaboration Platform
Level	Secondary
Primary Actor	System AI Assistant
Stakeholders and Interests	**Brand:** Expects continuously improving recommendations based on previous campaign outcomes. **System:** Uses campaign feedback and engagement data to enhance its predictive accuracy over time.
Preconditions	Completed campaign data, influencer performance metrics, and engagement statistics are available in the database.
Postconditions	AI model updates its recommendation weights and improves future matching accuracy.
Main Success Scenario	1. Campaign concludes and results are logged in the system. 2. AI Assistant analyzes campaign success metrics such as engagement, conversions, and ROI. 3. AI compares predicted vs actual performance for each influencer. 4. System adjusts weight parameters for similar future campaigns. 5. Enhanced model becomes active for upcoming recommendations.
Extensions	2a. Incomplete or corrupted campaign data — system flags it for admin review. 4a. Low data volume — AI uses averaged parameters instead of retraining.
Special Requirements	Model retraining should occur asynchronously to avoid slowing down live recommendations.
Technology and Data Variation List	Campaign performance data stored in SQLite. AI model trained periodically using Python-based pipeline and CSV exports.
Frequency of Occurrence	Periodic; triggered after each campaign completion or at scheduled intervals.
Open Issues	Establishing data thresholds for retraining and model drift detection.

Table 2.8: UC-RS-04: Learn from Historical Outcomes

2.2.3 AI Generation (Creatives)

Fields	Description
Use Case ID	UC-AI-01
Use Case Name	Draft Campaign Creatives
Scope	Splash AI-Powered Brand–Influencer Collaboration Platform
Level	Primary
Primary Actor	System AI Assistant
Stakeholders and Interests	**Brand:** Wants quick generation of high-quality ad copy and creative ideas for campaigns. **System:** Uses AI models to generate captions, slogans, and content outlines aligned with campaign goals.
Preconditions	A campaign brief must be created and approved. The brand must have an active subscription that supports AI creative generation.
Postconditions	AI-generated creative drafts are displayed in the campaign workspace for review and editing.
Main Success Scenario	1. Brand opens the campaign workspace and selects "Generate Creative." 2. System AI analyzes campaign objectives, audience, and influencer data. 3. AI generates caption ideas, visuals, and short content drafts. 4. Brand previews the generated content and saves preferred versions. 5. Drafts are stored for further personalization and publishing.
Extensions	2a. Missing campaign data — system prompts user to update brief. 3a. AI output irrelevant — system allows regeneration with adjusted parameters.
Special Requirements	Generated creatives should comply with brand tone and guidelines. AI must provide multiple output options per request.
Technology and Data Variation List	AI model uses NLP and text-to-image generation APIs. SQLite stores campaign references and creative metadata.
Frequency of Occurrence	Frequent; used during each campaign setup.
Open Issues	Integration of image generation models with text-based suggestions.

Table 2.9: UC-AI-01: Draft Campaign Creatives

Fields	Description
Use Case ID	UC-AI-02
Use Case Name	Personalize Creative to Influencer Tone
Scope	Splash AI-Powered Brand–Influencer Collaboration Platform
Level	Secondary
Primary Actor	System AI Assistant
Stakeholders and Interests	**Influencer:** Wants content drafts to match their unique style and audience tone. **Brand:** Seeks consistency in messaging while preserving creator authenticity. **System:** Learns influencer tone from past content and engagement metrics.
Preconditions	Draft creative exists and the influencer's verified profile includes prior content or campaign data.
Postconditions	A personalized creative version aligned with the influencer's tone and engagement style is generated.
Main Success Scenario	1. Influencer accesses campaign workspace and views draft creative. 2. System AI analyzes influencer's content tone, writing style, and visual patterns. 3. System adjusts language, caption tone, and visual suggestions to fit influencer's audience. 4. Personalized content draft is shown to the influencer and brand for review. 5. Both parties approve or request further fine-tuning.
Extensions	3a. No previous data available — AI defaults to campaign's tone profile. 4a. Tone mismatch detected — user requests regeneration.
Special Requirements	AI must adapt writing tone without altering key campaign messaging.
Technology and Data Variation List	AI model references influencer text embeddings and sentiment scores. SQLite stores influencer tone metadata and campaign links.
Frequency of Occurrence	Common; occurs for each influencer collaboration under a campaign.
Open Issues	Incorporating visual tone detection for cross-platform consistency.

Table 2.10: UC-AI-02: Personalize Creative to Influencer Tone

Fields	Description
Use Case ID	UC-AI-03
Use Case Name	Review and Edit Creative
Scope	Splash AI-Powered Brand–Influencer Collaboration Platform
Level	Primary
Primary Actor	Brand / Influencer
Stakeholders and Interests	**Brand:** Wants creative drafts aligned with brand identity and message precision. **Influencer:** Ensures the creative feels natural and fits their audience voice. **System:** Saves approved versions and maintains revision history.
Preconditions	AI-generated creative drafts must exist. Both brand and influencer have access to the campaign workspace.
Postconditions	Finalized creative is approved, versioned, and saved for campaign publishing.
Main Success Scenario	1. Brand or influencer opens AI-generated creative in workspace. 2. User edits captions, visuals, or formats directly within the platform. 3. System tracks changes and saves version history. 4. Final creative is approved and marked ready for publishing. 5. Approved creative is stored in campaign assets.
Extensions	2a. Editing timeout — system autosaves draft to prevent data loss. 3a. Conflicting edits by multiple users — system notifies and merges versions.
Special Requirements	Platform must maintain full version control with rollback capability.
Technology and Data Variation List	SQLite stores version history and approval metadata. Front-end supports rich text and media editing with autosave.
Frequency of Occurrence	Frequent; for each campaign creative approval cycle.
Open Issues	Integration of real-time collaborative editing and conflict resolution.

Table 2.11: UC-AI-03: Review and Edit Creative

2.2.4 Campaign Management and Chat System

Fields	Description
Use Case ID	UC-CM-01
Use Case Name	Open Campaign Workspace
Scope	Splash AI-Powered Brand–Influencer Collaboration Platform
Level	Primary
Primary Actor	Brand / Marketing Team
Stakeholders and Interests	**Brand:** Needs a centralized workspace to manage campaign activities and team collaboration. **Influencer:** Accesses the same workspace to view briefs, tasks, and deliverables. **System:** Creates and manages synchronized workspaces for both brand and influencer participants.
Preconditions	A verified campaign must exist. Brand and influencer should have accepted collaboration terms.
Postconditions	A campaign workspace is created, visible to both parties, and linked to all related modules (chat, deliverables, payments).
Main Success Scenario	1. Brand selects "Create Workspace" for an approved campaign. 2. System initializes a campaign workspace linked to the campaign record. 3. Workspace includes tabs for briefs, deliverables, chat, and analytics. 4. Influencer gains access upon confirmation of participation. 5. Both parties can view and manage campaign progress from the same dashboard.
Extensions	3a. Missing campaign data — system requests brand to complete the brief. 4a. Influencer not yet verified — system locks workspace until verification completes.
Special Requirements	Each workspace must be uniquely tied to one campaign. Must support multiple concurrent sessions without performance lag.
Technology and Data Variation List	SQLite stores workspace metadata and participant details. Frontend uses React-based views with real-time updates.
Frequency of Occurrence	Frequent; each new campaign triggers workspace creation.
Open Issues	Handling archived or completed workspaces efficiently.

Table 2.12: UC-CM-01: Open Campaign Workspace

Fields	Description
Use Case ID	UC-CM-02
Use Case Name	Structured Chat and File Exchange
Scope	Splash AI-Powered Brand–Influencer Collaboration Platform
Level	Primary
Primary Actor	Brand / Influencer
Stakeholders and Interests	**Brand:** Needs a secure and centralized communication system to manage campaign discussions and content approvals. **Influencer:** Wants a single place to receive instructions, share updates, and upload deliverables. **System:** Ensures all chat logs and shared files are securely stored and linked to their respective campaign workspace.
Preconditions	The campaign workspace must be active, and both parties must have verified access.
Postconditions	All chat messages and shared files are saved in the workspace, accessible to both users with appropriate permissions.
Main Success Scenario	1. Brand or influencer opens the chat section within the campaign workspace. 2. System displays structured threads for topics such as deliverables, revisions, and payments. 3. Users exchange text messages and upload related files (images, videos, briefs). 4. System logs every message with timestamps and sender identity. 5. Shared files are stored and linked to their campaign ID for easy retrieval.
Extensions	3a. File upload fails — system retries and prompts user to re-upload. 4a. Internet connection lost — system queues message for later delivery.
Special Requirements	Chat must support file formats up to 50MB and include message search functionality. Data encryption required for secure communication.
Technology and Data Variation List	SQLite stores message metadata and file paths. Files are stored on secure cloud storage linked through unique workspace identifiers.
Frequency of Occurrence	Frequent; used throughout the campaign lifecycle.
Open Issues	Integration of message notifications and typing indicators in real-time chat.

Table 2.13: UC-CM-02: Structured Chat and File Exchange

Fields	Description
Use Case ID	UC-CM-03
Use Case Name	Submit and Approve Deliverables
Scope	Splash AI-Powered Brand–Influencer Collaboration Platform
Level	Primary
Primary Actor	Influencer / Brand
Stakeholders and Interests	**Influencer:** Wants a simple and transparent way to upload campaign deliverables and receive timely feedback. **Brand:** Needs a clear approval process for submitted work to maintain campaign quality and deadlines. **System:** Tracks submission timestamps, version control, and approval status for each deliverable.
Preconditions	Campaign workspace and chat must be active. Influencer must have assigned deliverables.
Postconditions	Deliverables are either approved, marked for revision, or rejected, with status updates reflected in the campaign dashboard.
Main Success Scenario	1. Influencer navigates to the campaign workspace and selects "Submit Deliverable." 2. Influencer uploads the content file and adds optional notes. 3. System logs the submission with timestamp and version number. 4. Brand reviews the submission and selects "Approve" or "Request Revision." 5. System updates the deliverable status and notifies both parties.
Extensions	2a. File format invalid — system notifies influencer and rejects upload. 4a. Brand requests revision — influencer receives detailed comments and resubmits.
Special Requirements	Must support multiple file formats (images, videos, documents). Version tracking and audit history required for all changes.
Technology and Data Variation List	SQLite stores submission metadata, approval logs, and timestamps. Cloud storage integrated for file uploads with secure access tokens.
Frequency of Occurrence	Regular; occurs once or multiple times per deliverable depending on revision cycles.
Open Issues	Integration of automated quality checks (AI-based content validation).

Table 2.14: UC-CM-03: Submit and Approve Deliverables

Fields	Description
Use Case ID	UC-CM-04
Use Case Name	Milestone Tracking and Reminders
Scope	Splash AI-Powered Brand–Influencer Collaboration Platform
Level	Secondary
Primary Actor	System AI Assistant
Stakeholders and Interests	**Brand:** Wants automated tracking of campaign progress to ensure tasks and deliverables meet deadlines. **Influencer:** Receives timely reminders to stay on schedule and avoid penalties for delays. **System:** Monitors due dates and automatically triggers notifications or reminders based on milestone progress.
Preconditions	Active campaign workspace with defined milestones and deliverable deadlines.
Postconditions	Notifications and reminders are sent automatically; milestone status updates are reflected in the campaign dashboard.
Main Success Scenario	1. Brand defines campaign milestones during workspace setup. 2. System logs all deadlines and associates them with deliverables. 3. AI Assistant monitors progress and compares it with submission data. 4. System sends automated reminders to influencers or brands as deadlines approach. 5. Dashboard reflects updated milestone completion percentages.
Extensions	2a. Brand updates milestone dates and system recalculates reminder schedule. 4a. Deliverable submitted early and system marks milestone as completed.
Special Requirements	Reminders must be sent via both in-app notifications and email. System must allow customizable reminder intervals.
Technology and Data Variation List	SQLite stores milestone and reminder metadata. Scheduler service triggers automated reminders through the notification module.
Frequency of Occurrence	Continuous during active campaigns; reminders triggered daily or as configured.
Open Issues	Addition of dynamic rescheduling when multiple milestones overlap?

Table 2.15: UC-CM-04: Milestone Tracking and Reminders

2.2.5 Analytics Dashboard

Fields	Description
Use Case ID	UC-AD-01
Use Case Name	View Live Campaign Performance
Scope	Splash AI-Powered Brand–Influencer Collaboration Platform
Level	Primary
Primary Actor	Brand / Marketing Team
Stakeholders and Interests	**Brand:** Requires real-time insights into engagement, reach, and conversions to evaluate campaign effectiveness. **System:** Must display accurate and up-to-date metrics by fetching verified data from influencer submissions and connected APIs.
Preconditions	Active campaigns must exist with performance metrics logged or synced from influencer data.
Postconditions	Campaign performance metrics are displayed and stored for reporting and analysis.
Main Success Scenario	1. Brand accesses the Analytics Dashboard from the campaign workspace. 2. System retrieves live data from stored metrics and API sources. 3. Dashboard visualizes KPIs such as reach, engagement rate, impressions, and conversions. 4. Brand reviews charts and tables summarizing performance per influencer and campaign. 5. Data auto-refreshes periodically for real-time monitoring.
Extensions	2a. Data source temporarily unavailable and system displays last synced values. 4a. API rate limit reached and system delays refresh and notifies user.
Special Requirements	Dashboard should auto-refresh every 5 minutes and support drill-down views per influencer. Visualizations must be responsive and interactive.
Technology and Data Variation List	SQLite stores campaign KPIs and refresh logs. Frontend uses charting libraries for dynamic data visualization.
Frequency of Occurrence	Continuous during active campaigns; brands may access dashboard multiple times per day.
Open Issues	Integration of third-party social media APIs for real-time metric syncing.

Table 2.16: UC-AD-01: View Live Campaign Performance

Fields	Description
Use Case ID	UC-AD-02
Use Case Name	Compare Creators and Campaigns
Scope	Splash AI-Powered Brand–Influencer Collaboration Platform
Level	Secondary
Primary Actor	Brand / Marketing Team
Stakeholders and Interests	**Brand:** Wants to evaluate influencer performance comparatively to allocate future budgets efficiently. **System:** Provides accurate comparative analytics for multiple campaigns and creators.
Preconditions	At least two completed campaigns or multiple influencer collaborations must exist with stored performance data.
Postconditions	A comparative analysis report is displayed, highlighting performance differences and patterns.
Main Success Scenario	1. Brand selects "Compare Performance" from the analytics section. 2. System loads campaign and influencer data with engagement metrics. 3. System displays side-by-side charts comparing impressions, reach, and ROI. 4. Brand analyzes data and exports the comparison if needed.
Extensions	2a. Insufficient campaign data — system suggests minimum required metrics. 4a. Data visualization error — system reloads using cached values.
Special Requirements	Must support both graphical and tabular comparison modes. Allow filters for category, date range, and influencer tier.
Technology and Data Variation List	SQLite stores campaign summaries and influencer metrics. Visualization tools display bar and trend comparisons.
Frequency of Occurrence	Periodic; used mainly for performance review meetings or strategy adjustments.
Open Issues	Integration of AI insights to recommend top-performing influencers automatically.

Table 2.17: UC-AD-02: Compare Creators and Campaigns

Fields	Description
Use Case ID	UC-AD-03
Use Case Name	Export and Share Reports
Scope	Splash AI-Powered Brand–Influencer Collaboration Platform
Level	Secondary
Primary Actor	Brand / Team Member
Stakeholders and Interests	**Brand:** Needs downloadable reports to share performance outcomes with management or clients. **System:** Must generate reports in multiple formats with proper data visualization and security.
Preconditions	Campaign performance data must be available in the analytics dashboard.
Postconditions	A formatted report file is exported or shared via email/download link.
Main Success Scenario	1. Brand selects "Export Report" from the analytics section. 2. System displays export options (PDF, CSV, or Excel). 3. User selects the preferred format. 4. System compiles the selected data and generates the report. 5. File is downloaded or shared with authorized team members.
Extensions	2a. Export interrupted — system saves current progress and retries. 4a. File too large — system compresses or splits into multiple exports.
Special Requirements	Reports must include branding, charts, and campaign summary pages. Exports should complete within 30 seconds.
Technology and Data Variation List	SQLite stores analytics data. Reporting module uses LaTeX and data visualization libraries for PDF generation.
Frequency of Occurrence	Common after campaign completion or periodic reviews.
Open Issues	Automating report delivery via scheduled exports.

Table 2.18: UC-AD-03: Export and Share Reports

Fields	Description
Use Case ID	UC-AD-04
Use Case Name	Generate Insight Prompts
Scope	Splash AI-Powered Brand–Influencer Collaboration Platform
Level	Secondary
Primary Actor	System AI Assistant
Stakeholders and Interests	**Brand:** Wants intelligent insights that highlight campaign strengths, weaknesses, and recommendations. **System:** Uses stored analytics and AI models to generate automated performance observations.
Preconditions	Completed campaign data with recorded engagement, ROI, and audience metrics.
Postconditions	Insight prompts are displayed on the dashboard and logged for reference.
Main Success Scenario	1. AI Assistant reviews stored campaign data. 2. System identifies trends and anomalies (e.g., sudden engagement drops). 3. AI generates text-based insight prompts and visual flags. 4. Prompts are displayed under the analytics overview for the brand. 5. Brand can acknowledge, save, or dismiss insights.
Extensions	2a. Incomplete data — system waits for next refresh to generate insights. 4a. Duplicate prompts detected — system merges and prioritizes unique recommendations.
Special Requirements	Insights should be context-aware and generated in under 5 seconds.
Technology and Data Variation List	SQLite for analytics data; AI engine trained on prior campaign results. Visualizations created dynamically on the frontend.
Frequency of Occurrence	Continuous; generated automatically after every campaign update.
Open Issues	Integration of natural language explanations alongside visual cues.

Table 2.19: UC-AD-04: Generate Insight Prompts

2.2.6 Payment Gateway

Fields	Description
Use Case ID	UC-PG-01
Use Case Name	Create Escrow for Campaign
Scope	Splash AI-Powered Brand–Influencer Collaboration Platform
Level	Primary
Primary Actor	Brand / Marketing Team
Stakeholders and Interests	**Brand:** Wants secure payment handling that protects funds until deliverables are approved. **Influencer:** Needs assurance that funds are pre-authorized before work begins. **System:** Holds payment securely until completion of predefined milestones.
Preconditions	Campaign workspace must exist, and brand's payment method must be verified.
Postconditions	Escrow account is created, and funds are locked until approval. Transaction details are logged.
Main Success Scenario	1. Brand selects "Create Escrow" in the payment section of the campaign workspace. 2. System verifies brand's payment details and campaign milestones. 3. System creates a temporary escrow account tied to the campaign. 4. Funds are held securely until deliverables are marked as approved. 5. Confirmation message is sent to both brand and influencer.
Extensions	2a. Insufficient funds — system notifies brand to update payment method. 4a. Escrow creation fails — system retries and logs error for admin review.
Special Requirements	Escrow must comply with secure transaction protocols and maintain complete audit trails.
Technology and Data Variation List	SQLite stores transaction references; third-party payment gateway API handles fund locking.
Frequency of Occurrence	Once per campaign before deliverables begin.
Open Issues	Integration testing of multi-currency escrow handling.

Table 2.20: UC-PG-01: Create Escrow for Campaign

Fields	Description
Use Case ID	UC-PG-02
Use Case Name	Release Payment on Approval
Scope	Splash AI-Powered Brand–Influencer Collaboration Platform
Level	Primary
Primary Actor	Brand / System AI Assistant
Stakeholders and Interests	**Brand:** Wants funds released only after deliverables are reviewed and approved. **Influencer:** Expects prompt payment upon completion of campaign tasks. **System:** Validates approvals and executes fund transfer securely.
Preconditions	Deliverables have been submitted and approved by the brand; escrow account exists for the campaign.
Postconditions	Funds are released from escrow to the influencer, and payment logs are updated in the database.
Main Success Scenario	1. Brand approves submitted deliverables in the workspace. 2. System AI verifies milestone completion and approval status. 3. System triggers the payment release request to the payment API. 4. Influencer receives confirmation of successful payment transfer. 5. Payment record and receipts are updated for both users.
Extensions	2a. Approval pending — system waits for confirmation before executing transfer. 3a. Payment API failure — system retries and alerts the administrator.
Special Requirements	Payment release should occur within 24 hours of approval; system must ensure transaction security and accuracy.
Technology and Data Variation List	SQLite tracks payment logs; payment gateway API manages transfers and receipts.
Frequency of Occurrence	Once per milestone or upon campaign completion.
Open Issues	Handling refunds or dispute scenarios after payment release.

Table 2.21: UC-PG-02: Release Payment on Approval

Fields	Description
Use Case ID	UC-PG-03
Use Case Name	Manage Subscriptions
Scope	Splash AI-Powered Brand–Influencer Collaboration Platform
Level	Primary
Primary Actor	Brand / Influencer / Platform Administrator
Stakeholders and Interests	**Brand / Influencer:** Wants to easily subscribe, upgrade, downgrade, or cancel plans. **Administrator:** Oversees subscription billing, renewals, and access management. **System:** Maintains accurate billing cycles, renewals, and feature entitlements.
Preconditions	User must have a verified account and valid payment information.
Postconditions	Subscription plan is activated, modified, or canceled, and database records are updated.
Main Success Scenario	1. User navigates to the subscription management section. 2. System displays current plan details and available upgrades. 3. User selects a new plan or modifies existing one. 4. System updates payment cycle and feature entitlements. 5. Confirmation and invoice are generated automatically.
Extensions	3a. Payment failure — system prompts user to retry with a different method. 4a. Plan downgrade — system warns about feature loss before confirming.
Special Requirements	Must support auto-renewal and pro-rated billing for mid-cycle changes.
Technology and Data Variation List	SQLite maintains subscription history; integrated payment API handles recurring billing.
Frequency of Occurrence	Regular; triggered during plan upgrades, renewals, or cancellations.
Open Issues	Introducing promotional codes and temporary trials.

Table 2.22: UC-PG-03: Manage Subscriptions

Fields	Description
Use Case ID	UC-PG-04
Use Case Name	Enforce Plan Entitlements
Scope	Splash AI-Powered Brand–Influencer Collaboration Platform
Level	Secondary
Primary Actor	System AI Assistant
Stakeholders and Interests	**Brand:** Expects fair access to features based on chosen subscription plan. **System:** Must dynamically control feature availability according to plan type. **Administrator:** Needs visibility into user access levels for auditing.
Preconditions	Active subscription plan exists with defined entitlements.
Postconditions	Access permissions are updated automatically, limiting or unlocking features as per the plan.
Main Success Scenario	1. User logs into the system. 2. System checks the user's active subscription plan. 3. Features and modules are enabled or restricted accordingly. 4. System updates UI and displays upgrade prompts for restricted modules. 5. Access logs are recorded for monitoring.
Extensions	2a. Subscription expired — system redirects user to renewal page. 3a. Temporary access anomaly — system requests admin verification.
Special Requirements	Access control enforcement must happen at runtime. Role-based feature validation required.
Technology and Data Variation List	SQLite stores role and plan mappings. Frontend dynamically loads modules based on access tokens.
Frequency of Occurrence	Continuous; validated each time a user accesses restricted functionality.
Open Issues	Optimization of plan-based caching to improve performance.

Table 2.23: UC-PG-04: Enforce Plan Entitlements

2.2.7 Conversation Intelligence Assistant

Fields	Description
Use Case ID	UC-CI-01
Use Case Name	Summarize Chats and Briefs
Scope	Splash AI-Powered Brand–Influencer Collaboration Platform
Level	Secondary
Primary Actor	System AI Assistant
Stakeholders and Interests	**Brand:** Wants concise summaries of long chat discussions and campaign briefs. **Influencer:** Needs quick recaps of campaign requirements without rereading entire chats. **System:** Processes messages and briefs to generate accurate and context-aware summaries.
Preconditions	Active campaign workspace with chat history and brief content.
Postconditions	Summary text is displayed in the campaign dashboard and stored for future reference.
Main Success Scenario	1. AI Assistant scans chat messages and campaign brief content. 2. System extracts key points, deadlines, and deliverable highlights. 3. AI generates a concise, human-readable summary. 4. Summary is displayed in the workspace for both brand and influencer. 5. Users can download or copy the summary for record keeping.
Extensions	2a. Low message count — system skips summarization and shows direct highlights. 3a. Inconsistent message format — system filters non-relevant text.
Special Requirements	AI must summarize data in under 5 seconds and maintain original context accuracy above 90%.
Technology and Data Variation List	Chat messages stored in SQLite; summarization handled through NLP model integrated via API.
Frequency of Occurrence	Frequent; triggered automatically or manually during campaign discussions.
Open Issues	Integration of multilingual summarization for non-English chats.

Table 2.24: UC-CI-01: Summarize Chats and Briefs

Fields	Description
Use Case ID	UC-CI-02
Use Case Name	Suggest Replies and Brief Improvements
Scope	Splash AI-Powered Brand–Influencer Collaboration Platform
Level	Secondary
Primary Actor	System AI Assistant
Stakeholders and Interests	**Brand:** Seeks assistance in drafting professional, consistent communication with influencers. **Influencer:** Wants to respond quickly with well-structured and polite replies. **System:** Provides AI-generated responses and wording suggestions to maintain tone and relevance.
Preconditions	Active campaign chat or draft brief must exist.
Postconditions	Suggested replies or improved brief versions are presented for user selection or editing.
Main Success Scenario	1. AI Assistant analyzes the context of an ongoing chat or brief draft. 2. System generates relevant message suggestions or rephrased brief text. 3. User reviews AI suggestions and selects the preferred response. 4. Selected message is sent or brief updated accordingly.
Extensions	2a. AI confidence low — system highlights uncertain suggestions for user review. 3a. User rejects all suggestions — manual editing resumes.
Special Requirements	Replies must align with professional tone and maintain confidentiality of campaign details.
Technology and Data Variation List	NLP engine processes text from SQLite message logs. Language model API handles semantic rephrasing.
Frequency of Occurrence	Common; occurs multiple times per chat interaction.
Open Issues	Custom tone training for specific brand communication styles.

Table 2.25: UC-CI-02: Suggest Replies and Brief Improvements

Fields	Description
Use Case ID	UC-CI-03
Use Case Name	Follow-Up Nudges
Scope	Splash AI-Powered Brand–Influencer Collaboration Platform
Level	Secondary
Primary Actor	System AI Assistant
Stakeholders and Interests	**Brand:** Wants automatic reminders for influencers who have not responded or submitted deliverables. **Influencer:** Receives polite AI-generated reminders to take pending actions. **System:** Monitors message activity and automates follow-ups to improve response rates.
Preconditions	Active campaign with pending responses or incomplete deliverables.
Postconditions	Automated follow-up messages are generated and sent to respective users.
Main Success Scenario	1. AI Assistant monitors chat activity and campaign milestones. 2. System detects inactivity or overdue submissions. 3. AI generates a polite follow-up message and sends it automatically. 4. Influencer or brand receives the reminder and takes the required action. 5. Follow-up log is recorded in the campaign workspace.
Extensions	2a. False trigger — system cancels follow-up if new message detected. 3a. User disables auto-follow-ups for specific campaigns.
Special Requirements	Must prevent repetitive reminders; each follow-up should be spaced by at least 24 hours.
Technology and Data Variation List	SQLite logs inactivity timestamps; AI model schedules and sends automated reminders.
Frequency of Occurrence	Frequent; triggered for each overdue task or missed reply.
Open Issues	Adding context-aware tone adjustments based on user responsiveness.

Table 2.26: UC-CI-03: Follow-Up Nudges

2.2.8 Notification and Smart Alerts

Fields	Description
Use Case ID	UC-NA-01
Use Case Name	Real-Time Campaign Alerts
Scope	Splash AI-Powered Brand–Influencer Collaboration Platform
Level	Primary
Primary Actor	System AI Assistant
Stakeholders and Interests	**Brand:** Wants instant alerts for campaign updates, submissions, and approvals. **Influencer:** Expects timely notifications for new tasks, revisions, and messages. **System:** Monitors all modules and dispatches notifications based on events.
Preconditions	Active campaign workspace; notification module enabled for both parties.
Postconditions	Alerts are sent in real-time through in-app and email notifications.
Main Success Scenario	1. System tracks real-time events from active campaigns. 2. Triggered events include deliverable submissions, approvals, or payment updates. 3. AI Assistant formats the message content and dispatches alerts. 4. Users receive notifications in their dashboard and via email. 5. Alerts are logged for record-keeping.
Extensions	2a. Network delay — system retries sending after short intervals. 4a. User offline — notification queued for next login.
Special Requirements	Notifications must be delivered within 10 seconds of event occurrence. Email templates must be branded and consistent.
Technology and Data Variation List	SQLite stores notification logs; background worker handles dispatch scheduling.
Frequency of Occurrence	Continuous during campaign activity.
Open Issues	Support for push notifications in mobile version.

Table 2.27: UC-NA-01: Real-Time Campaign Alerts

Fields	Description
Use Case ID	UC-NA-02
Use Case Name	Priority Sorting
Scope	Splash AI-Powered Brand–Influencer Collaboration Platform
Level	Secondary
Primary Actor	System AI Assistant
Stakeholders and Interests	**Brand:** Prefers high-priority alerts such as pending approvals to appear first. **Influencer:** Needs urgent notifications like approaching deadlines highlighted automatically. **System:** Assigns priority levels to notifications for better readability and response time.
Preconditions	Notification module must be active and user preferences initialized.
Postconditions	Notifications are sorted and displayed based on importance and relevance.
Main Success Scenario	1. System categorizes new notifications based on priority (High, Medium, Low). 2. Critical events like payment releases and due deadlines are flagged as High Priority. 3. Dashboard interface displays priority-based sorting for both user roles. 4. AI learns from user interactions and adjusts priority rankings over time.
Extensions	2a. Conflicting priorities — system uses timestamp and event weight to decide order. 4a. Manual sorting preference overrides AI recommendations.
Special Requirements	Sorting algorithm must execute in under 2 seconds for seamless UI response.
Technology and Data Variation List	SQLite maintains notification records with priority flags. Frontend dynamically updates notification order using real-time listeners.
Frequency of Occurrence	Continuous; every new notification triggers sorting logic.
Open Issues	Adding custom rules for user-defined priority categories.

Table 2.28: UC-NA-02: Priority Sorting

Fields	Description
Use Case ID	UC-NA-03
Use Case Name	Preference Management
Scope	Splash AI-Powered Brand–Influencer Collaboration Platform
Level	Secondary
Primary Actor	Brand / Influencer
Stakeholders and Interests	**User:** Wants control over which notifications to receive and how (email, in-app, or both). **System:** Must respect user preferences while ensuring critical alerts are always delivered. **Administrator:** Oversees notification delivery compliance and audits preferences periodically.
Preconditions	User account must exist, and preferences should be initialized in settings.
Postconditions	User-defined preferences are updated and applied to subsequent notifications.
Main Success Scenario	1. User opens Notification Settings in their dashboard. 2. System displays available channels (in-app, email, push notifications). 3. User toggles preferences or adjusts frequency for each category. 4. System saves settings and applies changes instantly.
Extensions	2a. Channel temporarily unavailable — system disables option and alerts user. 4a. Sync delay — system queues updates and confirms once applied.
Special Requirements	Critical notifications (e.g., payments, security alerts) must override user restrictions.
Technology and Data Variation List	SQLite stores user preferences and logs last update timestamps. Notification service references these preferences before sending alerts.
Frequency of Occurrence	Occasional; when users modify settings or during initial setup.
Open Issues	Adding granular controls for notification categories.

Table 2.29: UC-NA-03: Preference Management

2.2.9 Admin, User Profile, and Personalization

Fields	Description
Use Case ID	UC-ADP-01
Use Case Name	Moderate Users and Disputes
Scope	Splash AI-Powered Brand–Influencer Collaboration Platform
Level	Primary
Primary Actor	Platform Administrator
Stakeholders and Interests	**Administrator:** Needs control over user activities to handle complaints and violations. **Brand / Influencer:** Expects fair dispute resolution and transparent moderation. **System:** Logs all actions to ensure compliance and accountability.
Preconditions	Valid user accounts exist with logged interactions. Dispute must be formally raised.
Postconditions	Dispute resolved; user actions moderated or sanctioned; audit record saved.
Main Success Scenario	1. Admin views reported disputes in the admin panel. 2. System retrieves relevant chat logs, submissions, and activity records. 3. Admin investigates and takes appropriate action (warning, suspension, or resolution). 4. System logs the decision and notifies involved parties.
Extensions	2a. Missing evidence — system requests additional data from users. 3a. Appeal raised — admin reopens case and updates resolution status.
Special Requirements	Moderation actions must be timestamped and auditable for transparency.
Technology and Data Variation List	SQLite stores dispute logs and admin actions. All moderation records backed up in CSV audit logs.
Frequency of Occurrence	Occasional; triggered by disputes or reported violations.
Open Issues	Developing automated flagging for repeated policy violations.

Table 2.30: UC-ADP-01: Moderate Users and Disputes

Fields	Description
Use Case ID	UC-ADP-02
Use Case Name	Manage Plans and Billing
Scope	Splash AI-Powered Brand–Influencer Collaboration Platform
Level	Secondary
Primary Actor	Platform Administrator
Stakeholders and Interests	**Administrator:** Needs to configure subscription plans and monitor payments. **System:** Ensures billing accuracy and plan availability for new users.
Preconditions	Active user base with linked payment gateways.
Postconditions	Updated plans or pricing reflected across all modules; billing records synchronized.
Main Success Scenario	1. Admin accesses "Billing and Plans" from the admin dashboard. 2. System displays current plans, pricing tiers, and user counts. 3. Admin updates pricing, adds new tiers, or deactivates existing ones. 4. System applies updates across all subscription-related modules. 5. Confirmation message logged for audit purposes.
Extensions	3a. Invalid pricing input — system rejects and prompts correction. 4a. Active billing cycle conflict — changes deferred until renewal.
Special Requirements	All plan changes must take effect without interrupting active users.
Technology and Data Variation List	SQLite stores pricing and plan metadata; payment gateway syncs billing details.
Frequency of Occurrence	Rare; only during business or pricing policy updates.
Open Issues	Adding region-based pricing and automated tax calculation.

Table 2.31: UC-ADP-02: Manage Plans and Billing

Fields	Description
Use Case ID	UC-ADP-03
Use Case Name	Configure Roles and Access
Scope	Splash AI-Powered Brand–Influencer Collaboration Platform
Level	Primary
Primary Actor	Platform Administrator
Stakeholders and Interests	**Administrator:** Needs control over role-based access permissions for users. **System:** Must enforce access policies dynamically to prevent unauthorized actions.
Preconditions	Active users exist with assigned roles. Role and access data tables initialized.
Postconditions	Updated roles and permissions saved and applied across the system.
Main Success Scenario	1. Admin selects "Manage Roles" in the admin dashboard. 2. System displays existing roles and associated permissions. 3. Admin modifies access levels or creates new roles. 4. System validates changes and updates permissions in real time. 5. Access logs are updated and synced to the database.
Extensions	2a. Role deletion restricted — system prevents removal of critical admin roles. 3a. Invalid permission combination — system displays error message.
Special Requirements	Role-based access control (RBAC) must be applied dynamically during user sessions.
Technology and Data Variation List	SQLite manages user-role relationships; session tokens updated post changes.
Frequency of Occurrence	Occasional; triggered when new user categories or access levels are introduced.
Open Issues	Adding support for temporary role escalation during issue resolution.

Table 2.32: UC-ADP-03: Configure Roles and Access

Fields	Description
Use Case ID	UC-ADP-04
Use Case Name	Generate Personalized Dashboards
Scope	Splash AI-Powered Brand–Influencer Collaboration Platform
Level	Secondary
Primary Actor	System AI Assistant
Stakeholders and Interests	**Brand:** Expects tailored insights, campaign analytics, and quick access to active collaborations. **Influencer:** Wants personalized metrics on engagement, growth, and potential collaborations. **System:** Curates data widgets dynamically for each user type based on activity and preferences.
Preconditions	User must have an active profile and completed at least one campaign or activity.
Postconditions	Dashboard is rendered dynamically, showing data relevant to user role and behavior.
Main Success Scenario	1. User logs into Splash. 2. AI Assistant identifies user role and retrieves relevant data. 3. System generates a customized dashboard layout using key metrics. 4. User interacts with widgets to view analytics or actions. 5. System stores layout preferences for future sessions.
Extensions	2a. No activity history — system shows on-boarding suggestions. 4a. Widget error — system reloads defaults automatically.
Special Requirements	Dashboards must render under 5 seconds and update based on user interactions in real time.
Technology and Data Variation List	SQLite stores user metrics and preferences; AI Assistant filters data per user role.
Frequency of Occurrence	Continuous; generated each time a user logs in.
Open Issues	Addition of drag-and-drop customization for advanced users.

Table 2.33: UC-ADP-04: Generate Personalized Dashboards

2.3 Functional Requirements

2.3.1 Data Gathering and Influencer On-boarding

1. FR-DG-1. The on-boarding module shall automatically detect existing influencer profiles during sign-up.
2. FR-DG-2. The system shall prevent duplicate profiles.
3. FR-DG-3. The system shall allow claiming existing influencer profiles or the creation of new ones.
4. FR-DG-4. The system shall verify the profiles before public discovery.
5. FR-DG-5. The system shall maintain version control and sync updates to the database.
6. FR-DG-6. The system shall be integrated with social media APIs to extract metrics (followers, demographics engagement rate).
7. FR-DG-7. The system shall perform fraud detection on said metrics and assign a trust score(0-100) to each influencer.
8. FR-DG-8. The system shall refresh social media metrics at least weekly for all active influencers.

2.3.2 Recommendation System

1. FR-RS-1. The system shall provide a campaign brief creation form with auto-save functionality.
2. FR-RS-2. The recommendation engine shall calculate compatibility scores for influencers based on engagement rate, category alignment, audience demographics.
3. FR-RS-3. The system shall generate ranked influencer recommendations.
4. FR-RS-4. The system shall support configurable weights for different requirements through the admin panel.

2.3.3 AI Generation (Creatives)

1. FR-AG-1. The system shall generate AI-based advertising creatives (visuals, captions, slogans).

2. FR-AG-2. The system shall analyze campaign brief, influencer profile, and target audience for personalization.

3. FR-AG-3. The system shall provide three output options per request.

4. FR-AG-4. The system shall maintain compliance with brand guidelines and image.

2.3.4 Campaign Management and Chat System

1. FR-CM-1. The system shall create a dedicated campaign workspace with tabs for brief, deliverables, chat, analytics, payment status.

2. FR-CM-2. The system shall track deliverable submission with status (pending, submitted, revision requested, approved, rejected).

3. FR-CM-3. The system shall send milestone reminders at 7, 3, and 1 day intervals before deadline.

4. FR-CM-4. The system shall support message search by filters like keyword, date, or sender.

5. FR-CM-5. The system shall support file uploads up to 50MB.

6. FR-CM-6. The system shall queues offline messages and deliver upon reconnection.

2.3.5 Analytics Dashboard

1. FR-AD-1. The system shall retrieve and display real-time campaign metrics (reach, impressions, engagement, revenue).

2. FR-AD-2. The system shall automatically fetch metrics from social media APIs.

3. FR-AD-3. The system shall provide drill-down views filtered by influencer, date range, and/or content type.

4. FR-AD-4. The system shall support campaign comparison in both graphical and tabular formats side by side.

5. FR-AD-6. The system shall generate downloadable reports (pdf, csv, Excel).

6. FR-AD-7. The system shall allow email sharing of reports via secure generated access links.

7. FR-AD-8. The system shall automatically analyze data for both anomalies and trends for insights and recommendations.

8. FR-AD-9. The system shall provide performance benchmarking against industry standards and influencer averages.

2.3.6 Payment Gateway

1. FR-PG-1. The payment gateway shall create escrow accounts for brand-influencer payments.

2. FR-PG-2. The payment processor shall automatically release payments from escrow within 1-2 working days of approval.

3. FR-PG-3. The system shall process transfers via Stripe, PayPal, JazzCash, or Easypaisa.

4. FR-PG-4. The system shall maintain audit trails for all transactions.

5. FR-PG-5. The system shall be able to track payment status (pending, completed, failed, refunded) through real-time updates.

2.3.7 Conversation Intelligence Assistant

1. FR-CI-1. The system shall automatically summarize campaign chat histories and briefs.

2. FR-CI-2. The system shall extract key deadlines, deliverables, and action items.

3. FR-CI-3. The system shall allow download/copy of summaries in text or formatted documents.

4. FR-CI-4. The system shall generate intelligent message reply suggestions.

5. FR-CI-5. The system shall provide campaign brief improvement suggestions.

6. FR-CI-6. The system shall monitor inactivity and send follow-up reminders.

7. FR-CI-7. The system shall maintain conversation context across multiple sessions.

2.3.8 Notifications and Smart Alerts

1. FR-NA-1: The system shall monitor campaign activities (submissions, approvals, messages, payments).
2. FR-NA-2: The notification dispatcher shall send alerts through in-app dashboard and email within 10 seconds.
3. FR-NA-3: The priority engine shall assign priority levels (High, Medium, Low) based on it's urgency and business impact.
4. FR-NA-4: The system SHALL provide summaries for non-urgent notifications (daily/weekly).
5. FR-NA-5: The system SHALL provide notification history viewable by users upto 30 days.

2.3.9 Search and Discovery

1. FR-SD-1. The system shall support multi-criteria filtering (follower count, location, niche, engagement threshold, age, etc.).
2. FR-SD-2. The browse module shall apply filters dynamically without page reload.
3. FR-SD-3. The system shall implement sorting options to work together with filters such as: relevance, follower count, newest, most popular.
4. FR-SD-4. The system shall allow shortlist saving, and side-by-side comparison.
5. FR-SD-5. The system shall implement auto-complete suggestions as user types to improve search efficiency.
6. FR-SD-6. The search engine shall allow advanced search with boolean operators (AND, OR, NOT) to allow complex queries.

2.4 Non-Functional Requirements

2.4.1 Performance Requirements

1. NF-PR-1. The system shall respond to user requests within 3 seconds under normal load.
2. NF-PR-2. The system shall process a minimum of 500 requests per second during peak platform usage.
3. NF-PR-3. The system shall render web pages within 3 seconds.
4. NF-PR-4. The system shall execute database queries within 500ms for normal operations.
5. NF-PR-5. The system shall support file uploads up to 50MB.

2.4.2 Availability Requirements

1. NF-AR-1. The system shall maintain 95% uptime manually.
2. NF-AR-2. The system shall maintain zero single points of failure for critical services.
3. NF-AR-3. The system shall have Recovery Time Objective of maximum 30 minutes.
4. NF-AR-4. The system shall have Recovery Point Objective of maximum 60 minutes.

2.4.3 Security Requirements

1. NF-SR-1. The system shall implement Multi-factor Authentication as an optional feature.
2. NF-SR-2. The system shall enforce Role-Based Access Control.
3. NF-SR-3. The system shall never store plain-text payment card data (tokenization mandatory).
4. NF-SR-4. The system shall maintain audit logs of all user actions with timestamp and user ID.

2.4.4 Maintainability Requirements

1. NF-MR-1. The system shall achieve minimum 80% code coverage for critical modules via automated tests.
2. NF-MR-2. The system shall centralize logs from all services.
3. NF-MR-3. The system shall alert on threshold breaches.

2.4.5 Usability Requirements

1. NF-UR-1. The system shall not exceed a 30-minute learning curve for new users to basic functionality.
2. NF-UR-2. The system shall support undo/redo functionality for content creation.
3. NF-UR-3. The system shall adapt date, time, and currency formats to user locale.
4. NF-UR-4. The system shall provide system feedback (loading indicators, success messages).
5. NF-UR-5. The system shall support keyboard navigation for all interactive elements.
6. NF-UR-6. The system shall provide screen reader support with alt text for all images.

2.5 Domain Model

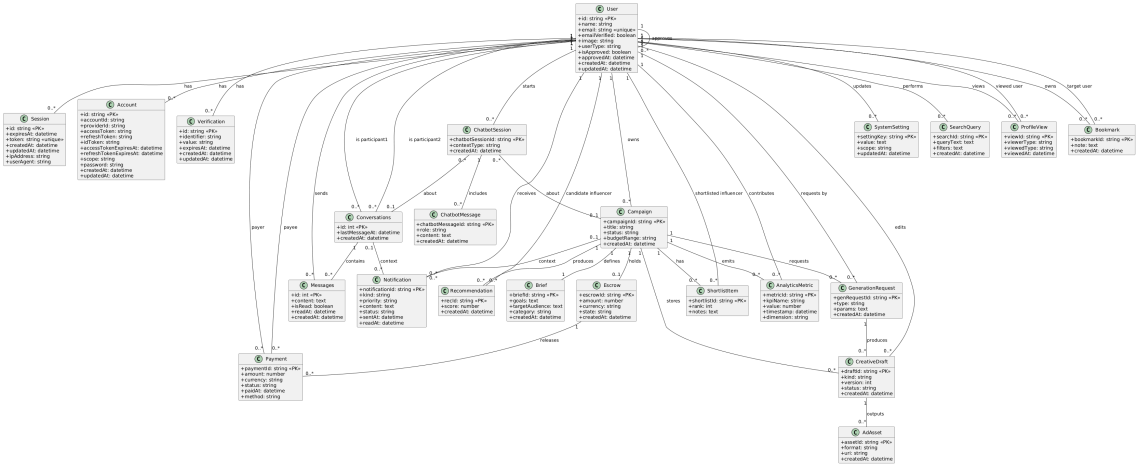


Figure 2.2: Domain Model

Chapter 3

System Overview

3.1 Architectural Design

The Splash platform is a modular web system. The overall architecture follows a three-tier client–server model that separates presentation, application logic, and data management. This flexible separation lets independent development, testing, and scaling of each component while also ensuring smooth collaboration between the three of them.

3.1.1 High-Level Architecture Layers

The system is structured into distinct layers that interact with each other in a linear and well-defined manner. Each layer has specific responsibilities which collectively enable the complete functionality of the system.

- **Client Layer:** This layer is implemented using React and Next.js with TypeScript. It is used for rendering the user interface, handling user inputs, and managing client-side state. The client layer provides access to modules like browsing influencers, creating campaigns, managing chats, viewing analytics, and accessing the AI assistant.

- **Application (API) Layer:** This layer is implemented through Next.js API routes that serve as the bridge between the frontend and the data Layer. It manages authentication, business logic, campaign creation, chat system, AI generation requests, payments, analytics aggregation, and notification dispatching.

- **Data Layer:** This layer uses SQLite with Drizzle ORM to store system data in a relational structure. It manages tables such as users, sessions, campaigns, recommendations, payments, and messages. Additionally, the creators dataset in csv

format is ingested and indexed to help the recommendation engine with it's functionality. Object storage is used to store creative assets and submitted deliverables.

- **External Integrations:** The system integrates with external services for AI model generation (text and image), payment gateway processing, Social Media metrics, and email notifications.

3.1.2 Modules Collaboration

The platform's functionality is divided into many modules that communicate through the API layer and share a unified data model. The collaboration among modules ensures that user actions in one component can be seen across others in real time.

- **On-boarding and Verification:** Manages user registration, profile verification, and links creators to their dataset records.
- **Recommendation Engine:** Reads campaign briefs, processes creator profiles from the dataset, and suggests ranked influencers based on relevance, engagement metrics, and previous collaboration success.
- **AI Generation (Creatives):** Produces creative drafts and advertising assets using AI models, helping brands to generate personalized promotional content automatically.
- **Campaign and Chat:** Offers a shared workspace for brands and influencers to exchange deliverables, communicate, and manage the campaign.
- **Analytics Dashboard:** Aggregates key performance indicators (KPIs) for campaigns and influencers, including impressions, engagement rates, and conversion performance.
- **Payments and Subscriptions:** Handles escrow creation, payment releases, and subscription plan enforcement.
- **Notifications and Alerts:** Delivers in-app and email notifications for critical events such as task completion, payment updates, and new collaboration requests.
- **Admin and Settings:** Allows administrators to moderate users, resolve disputes, manage subscription plans, and configure global system settings.

3.1.3 System Collaboration Overview

The collaboration sequence begins with the brand creating a campaign and defining its brief. The recommendation engine processes the brief and retrieves suitable influencer profiles from the dataset. The shortlisted influencer is then notified and can join the campaign workspace, where both parties can communicate through the chat module and work together on the campaign. The AI ad generation module assists in producing content drafts which are reviewed and approved. Once an influencer submits final deliverables, the payment service verifies the approval status and releases escrowed funds. Analytics are updated throughout the campaign to measure ROI and engagement. Notifications keep both the brand and influencer informed about every update.

3.1.4 Summary

The Splash architecture maintains a clear separation between presentation, logic, and data layers to ensure reliability and maintainability. Each module is responsible for it's specific set of functions, enabling flexible future expansion without affecting other parts of the system. The modular design also allows the integration of external AI and payment services without altering the internal data structure. This layered and event-driven approach ensures scalability, security, no single point of failure, and smooth user experience for all users.

3.1.5 Layered Architecture

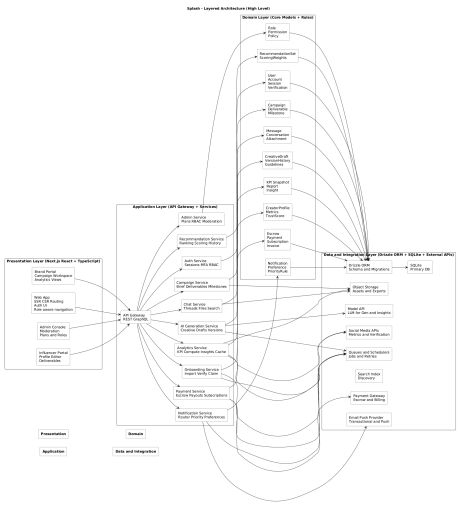


Figure 3.1: Architecture Diagram

3.1.6 Box and Line Diagram

Figure 3.2 illustrates the high-level system architecture, showing the interaction between the client, API, data layer, and external integrations.

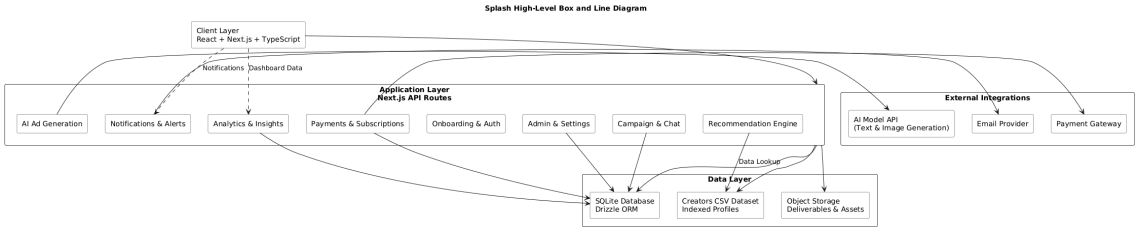


Figure 3.2: Box and Line Diagram

3.1.7 Modules Collaboration Diagram

Figure 3.3 presents the sequence of interaction between modules across a typical campaign workflow.

Figure 3.3: Modules Collaboration Diagram

3.2 Design Models

3.2.1 Activity Diagrams

3.2.1.1 Data Gathering and Influencer On-boarding

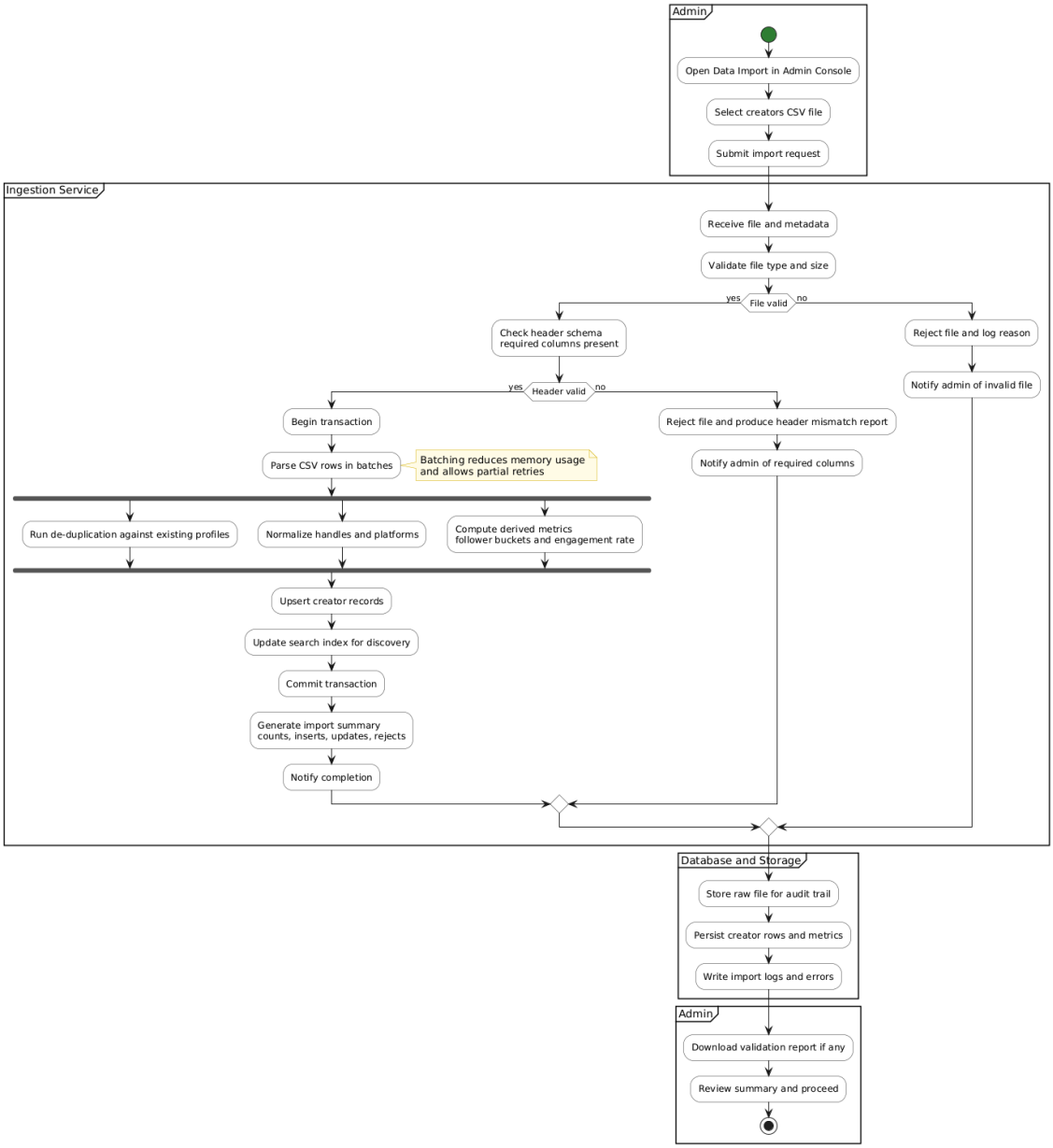


Figure 3.4: Activity Diagram (Import Dataset)

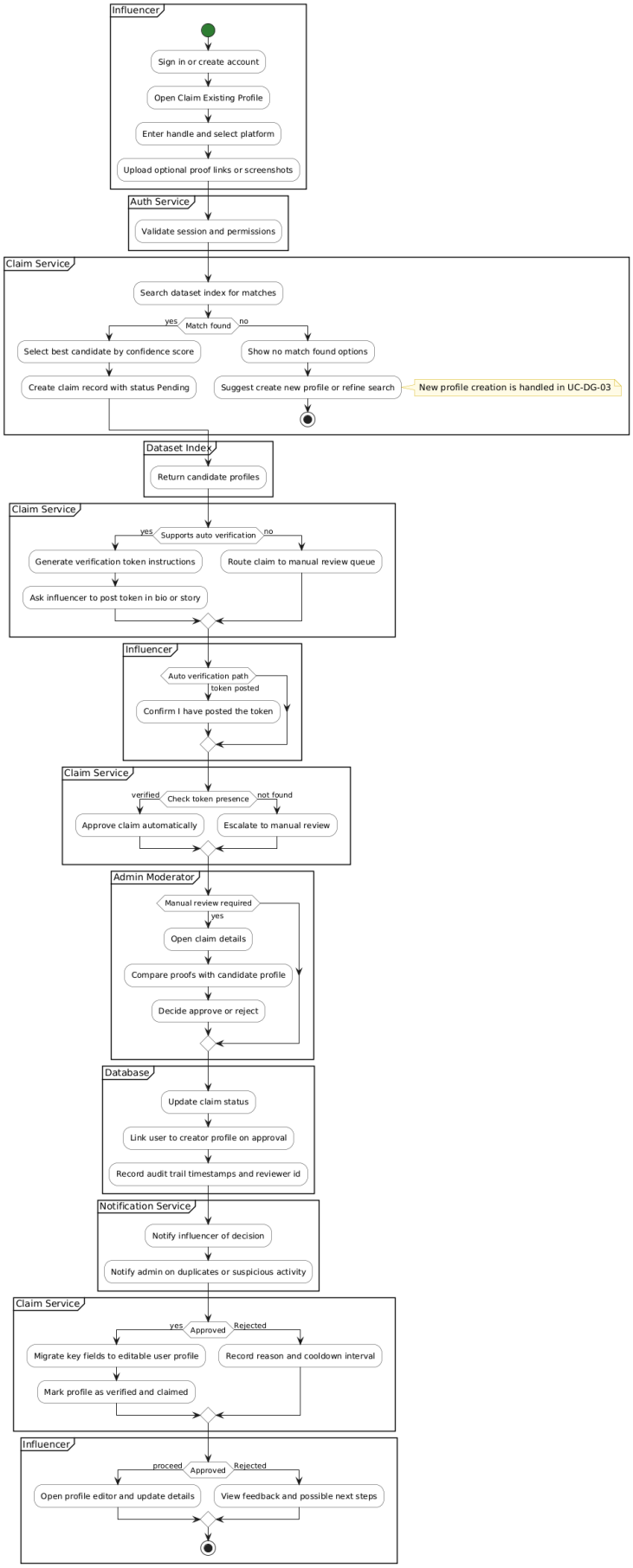


Figure 3.5: Activity Diagram (Claim Existing Profile)

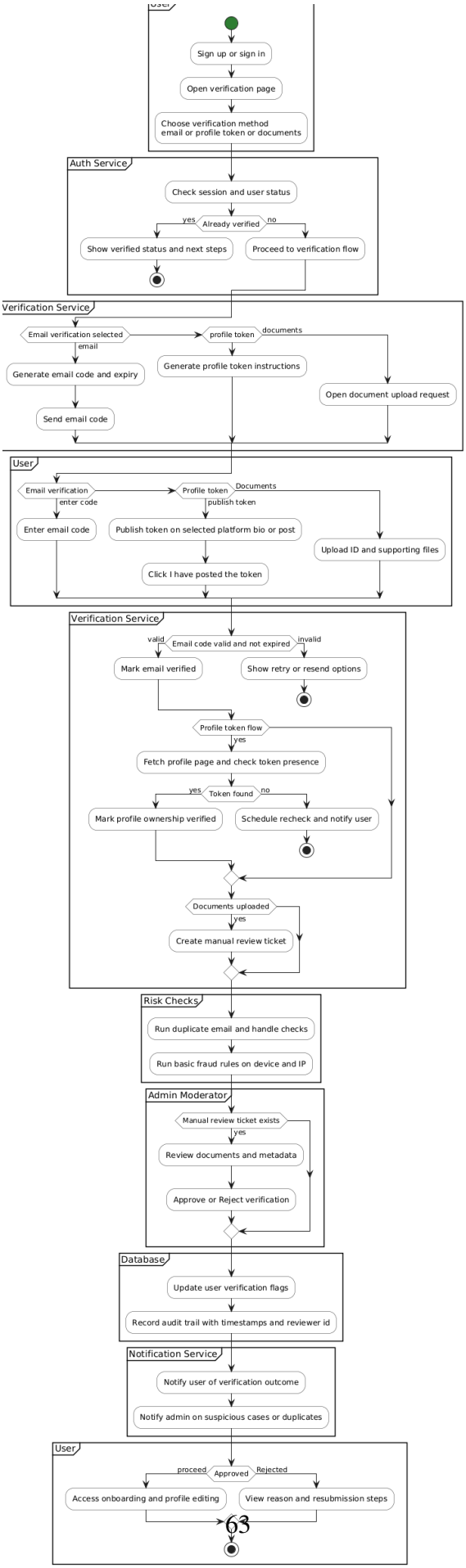


Figure 3.6: Activity Diagram (Verify User)

3.2.1.2 Recommendation System

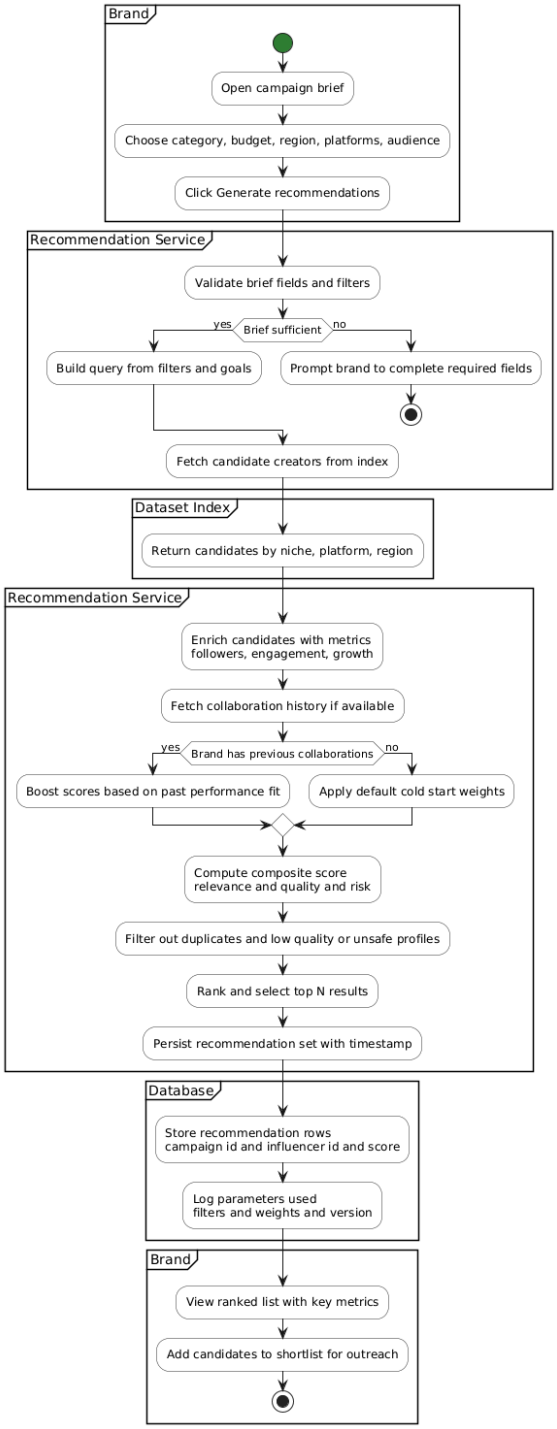


Figure 3.7: Activity Diagram (Generate Recommendations)

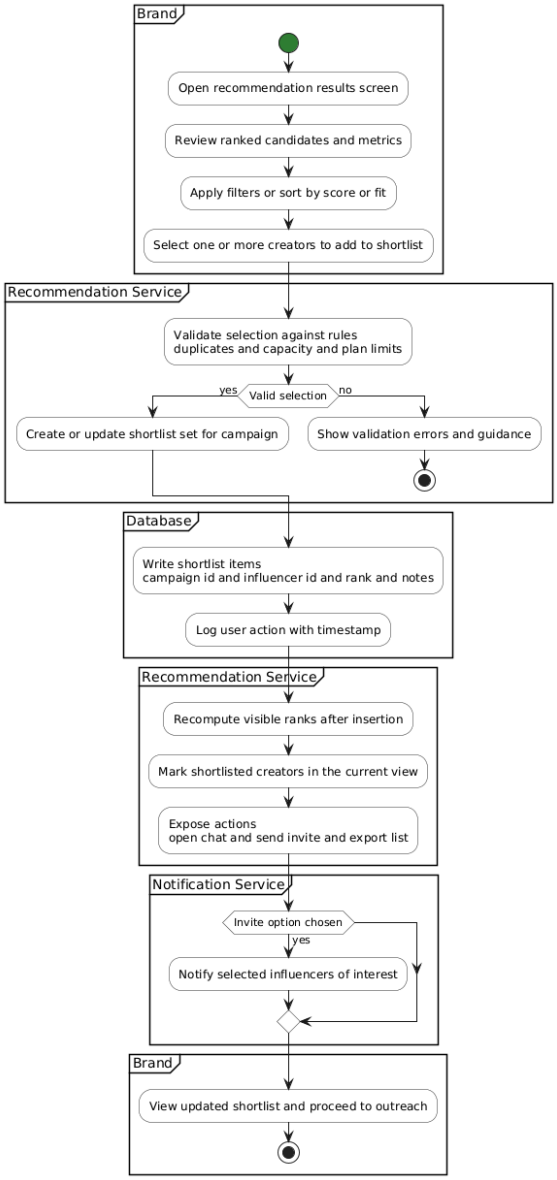


Figure 3.8: Activity Diagram (Shortlist Influencers)

3.2.1.3 Ad Generation

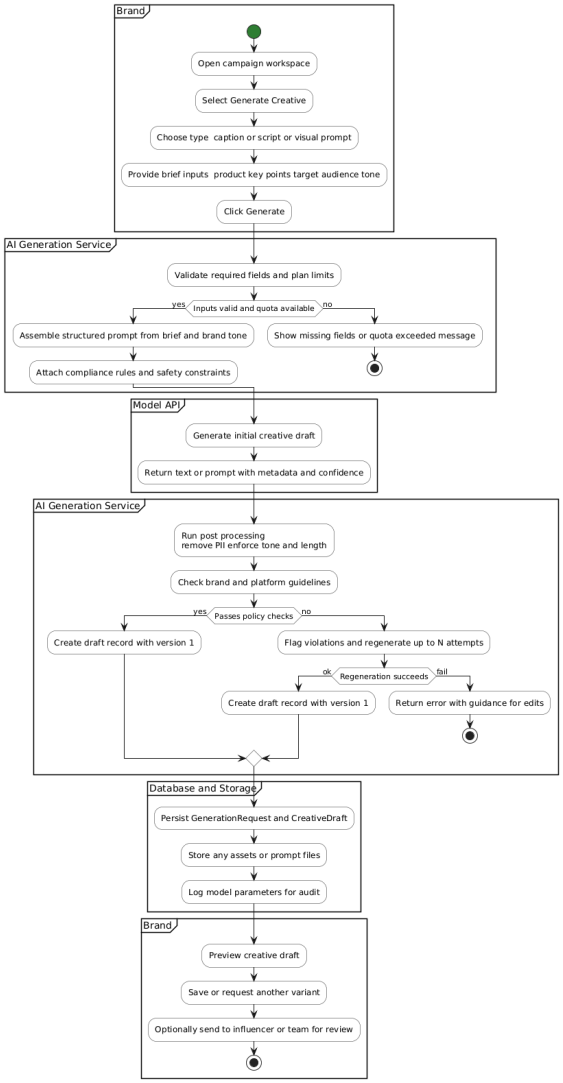


Figure 3.9: Activity Diagram (Generate Creatives)

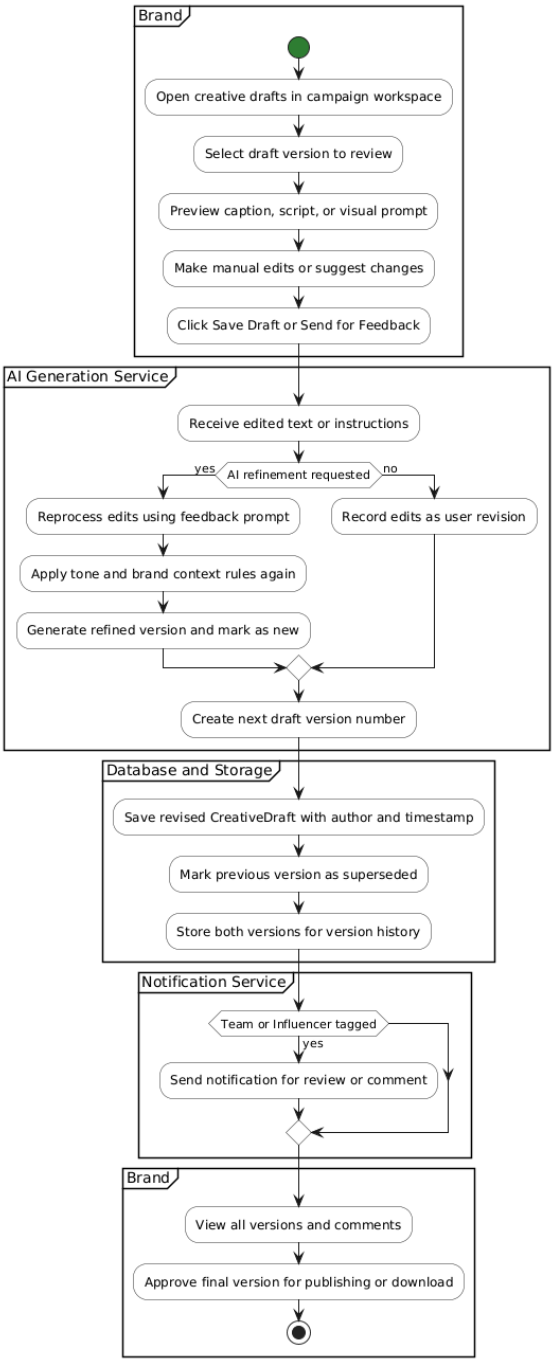


Figure 3.10: Activity Diagram (Review and Edit Creatives)

3.2.1.4 Campaign Management and Chat System

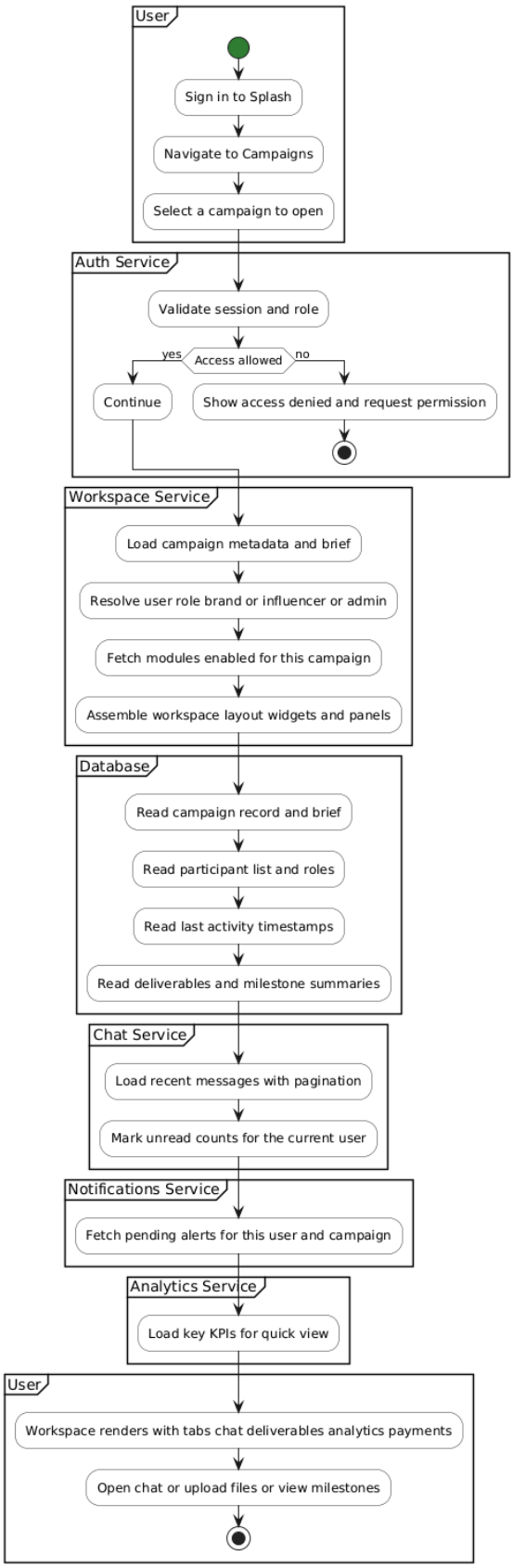


Figure 3.11: Activity Diagram (Open Campaign Workspace)

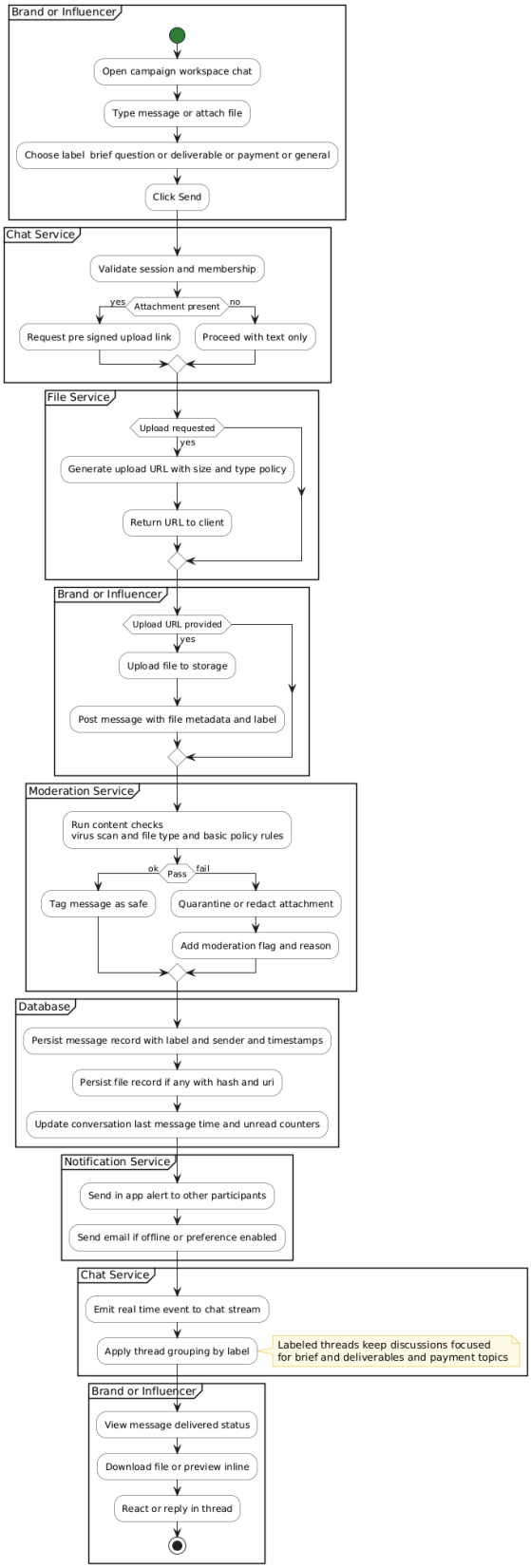


Figure 3.12: Activity Diagram (Chat and File Exchange)

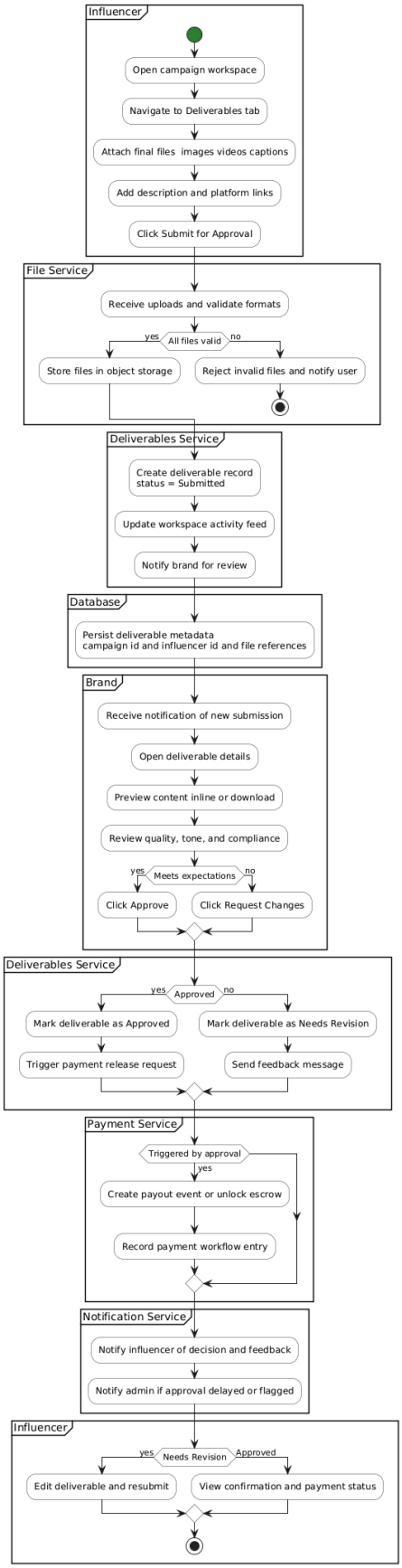


Figure 3.13: Activity Diagram (Submit and Approve Deliverables)

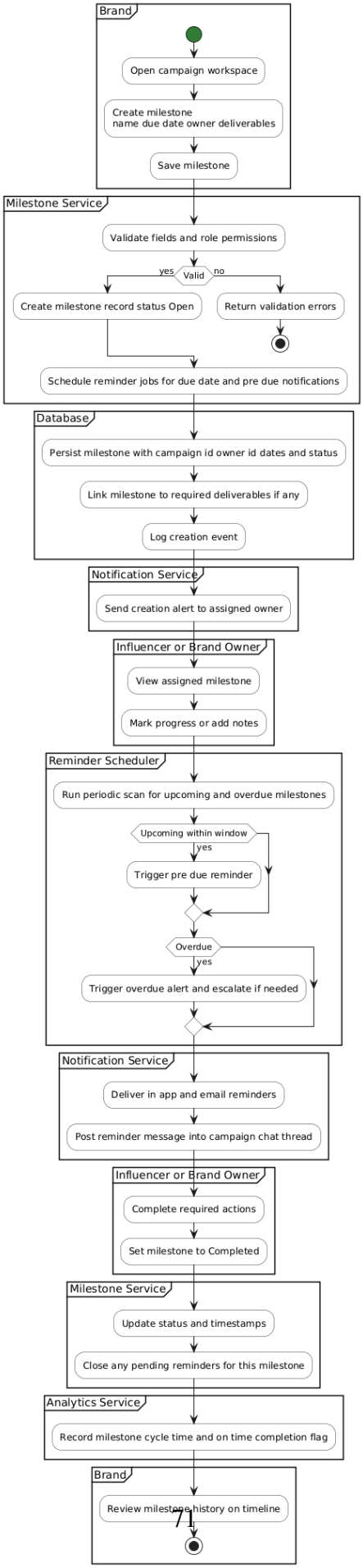


Figure 3.14: Activity Diagram (Milestone Tracking and Reminders)

3.2.1.5 Analytics Dashboard

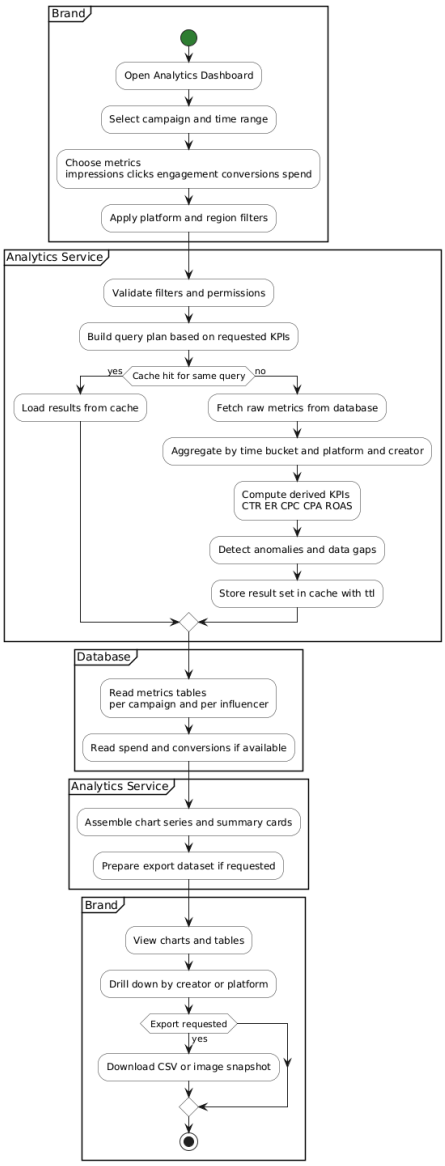


Figure 3.15: Activity Diagram (View Live Campaign Performance)

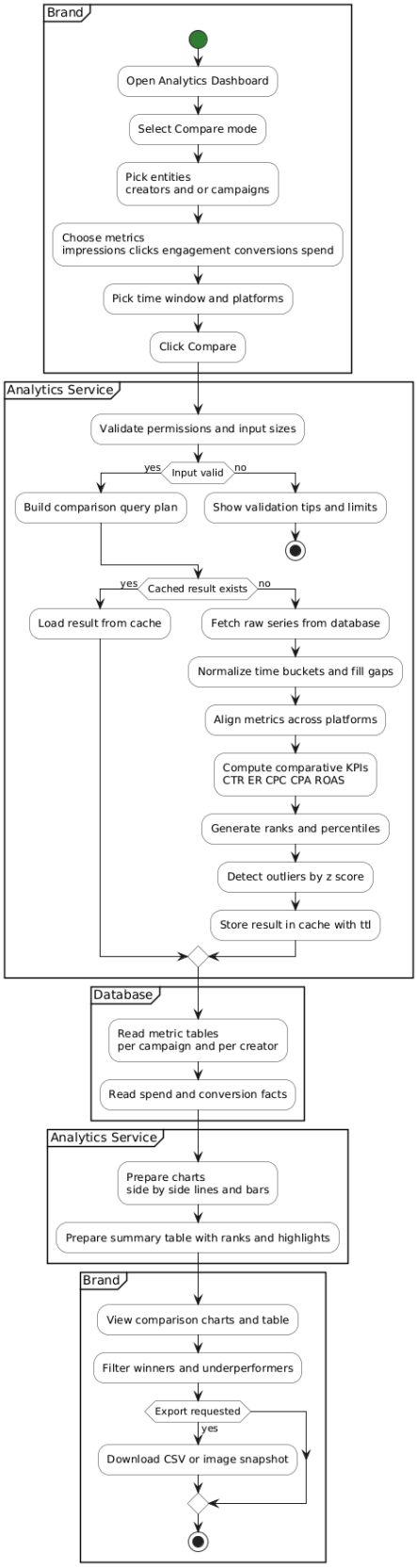


Figure 3.16: Activity Diagram (Compare Creators and Campaigns)

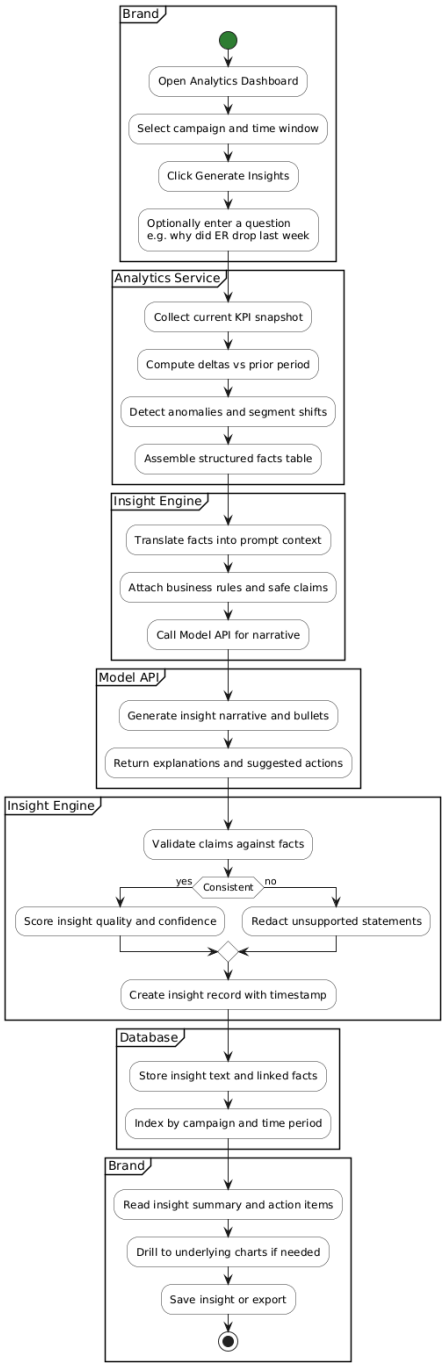


Figure 3.17: Activity Diagram (Generate Insight Prompts)

3.2.1.6 Payment Gateway

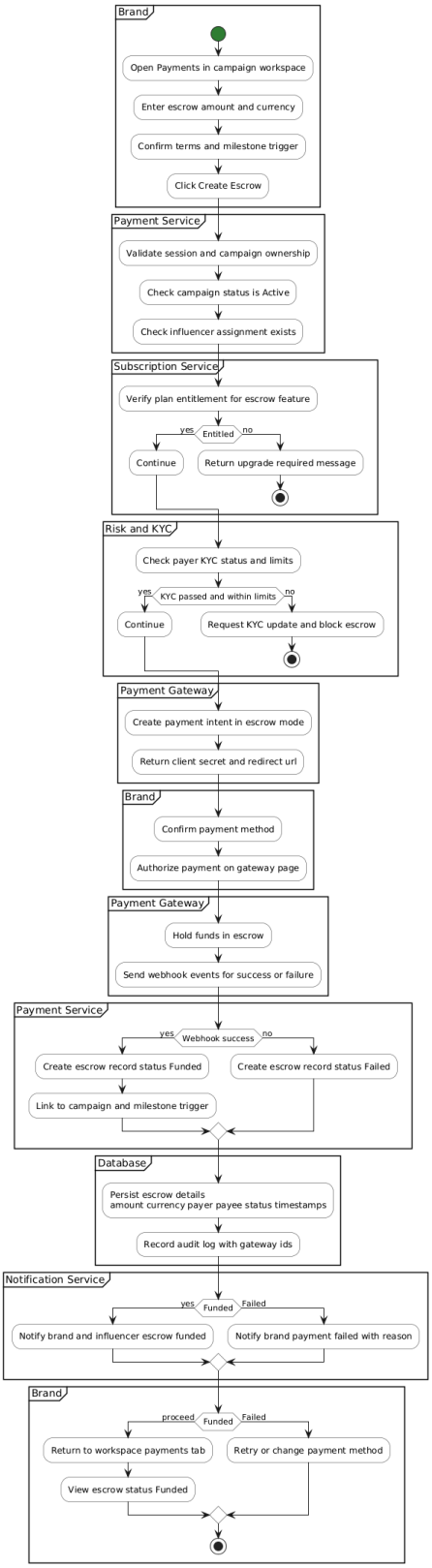


Figure 3.18: Activity Diagram (Create Escrow for Campaign)

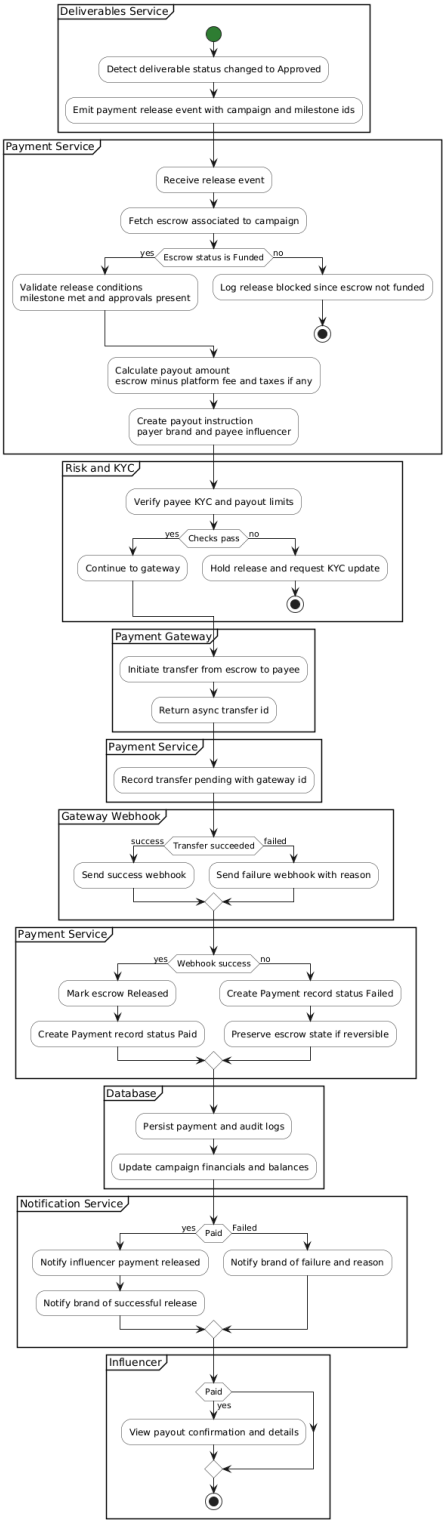


Figure 3.19: Activity Diagram (Release Payment On Approval)

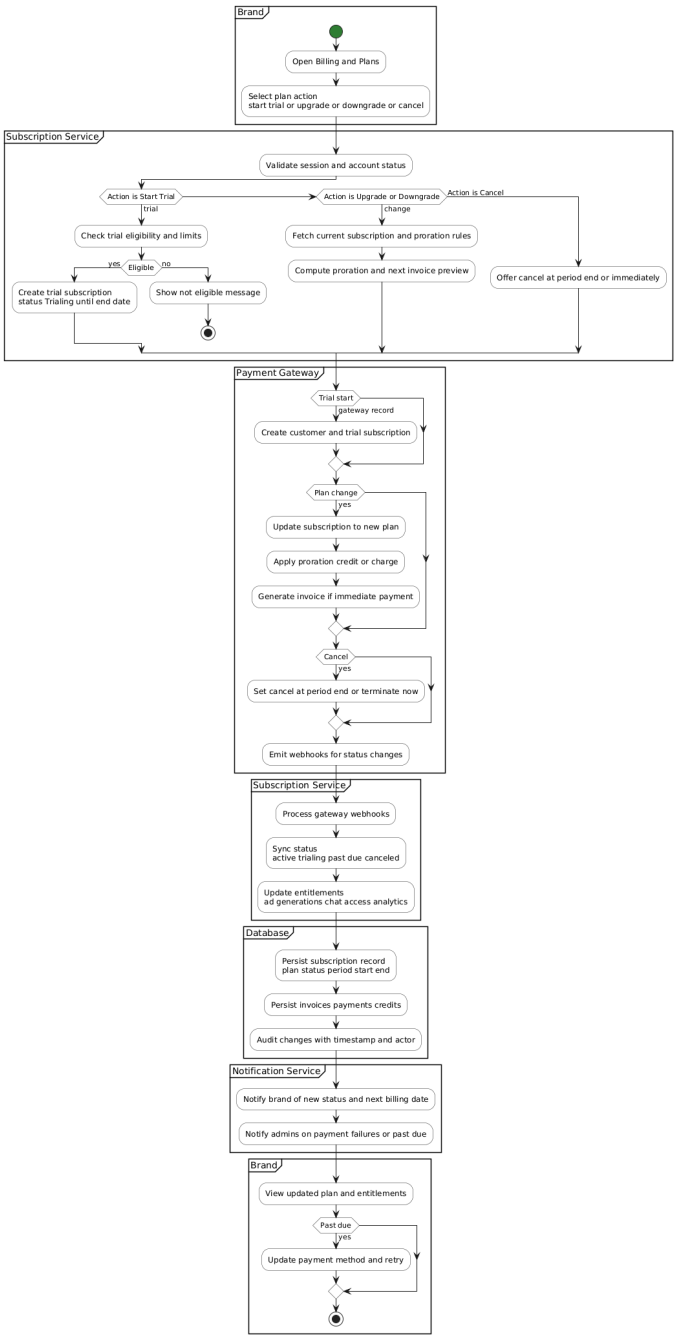


Figure 3.20: Activity Diagram (Manage Subscriptions)

3.2.1.7 Chatbot

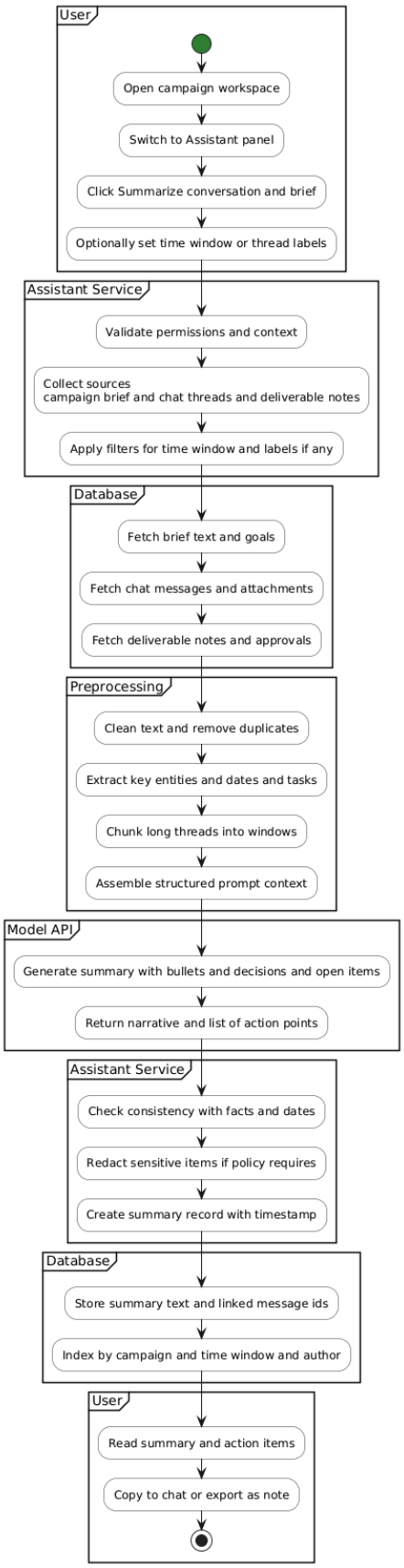


Figure 3.21: Activity Diagram (Summarize Chats and Briefs)

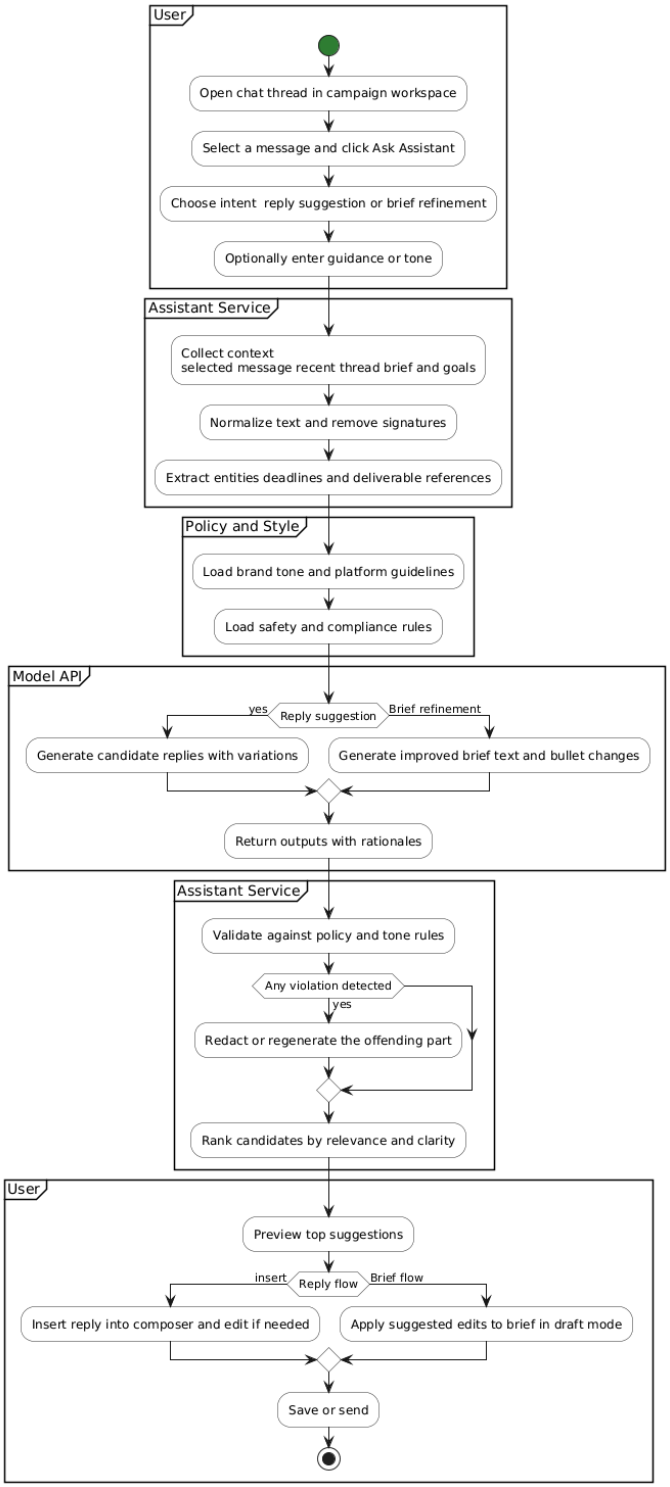


Figure 3.22: Activity Diagram (Suggest Replies and Brief Improvements)

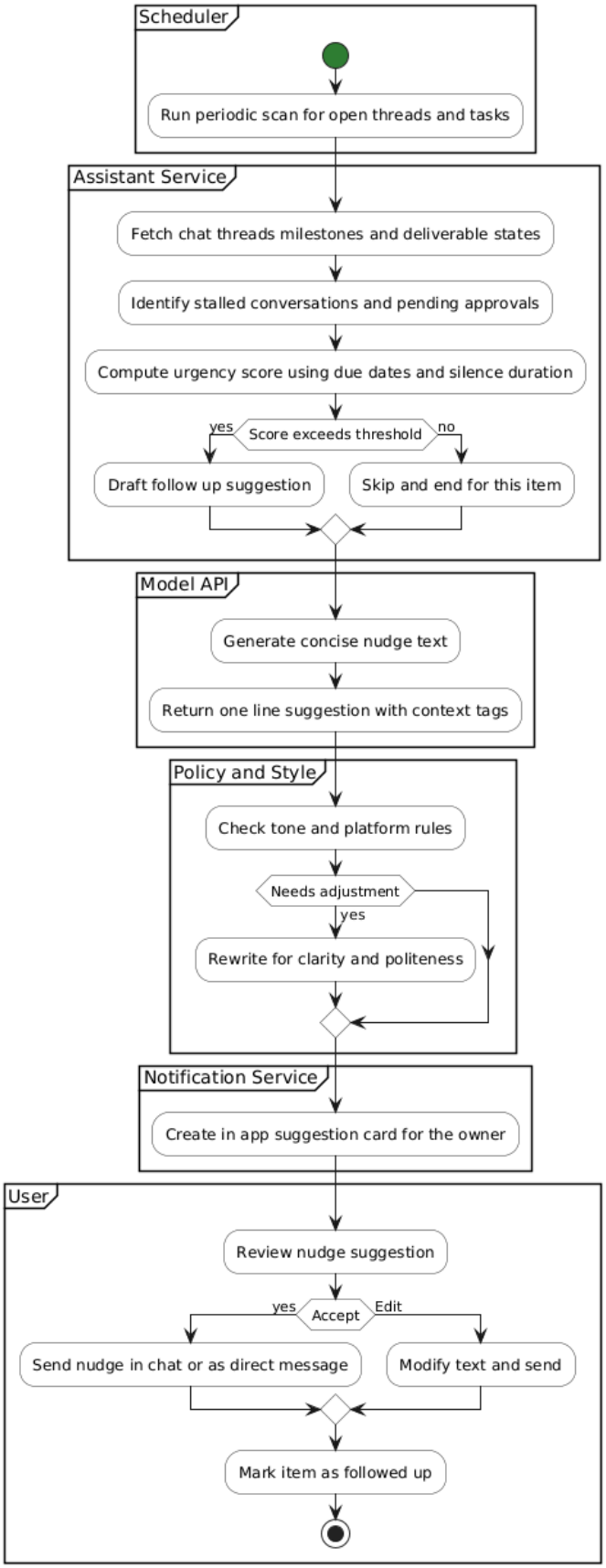


Figure 3.23: Activity Diagram (Follow Up Nudges)

3.2.1.8 Notification and Smart Alerts

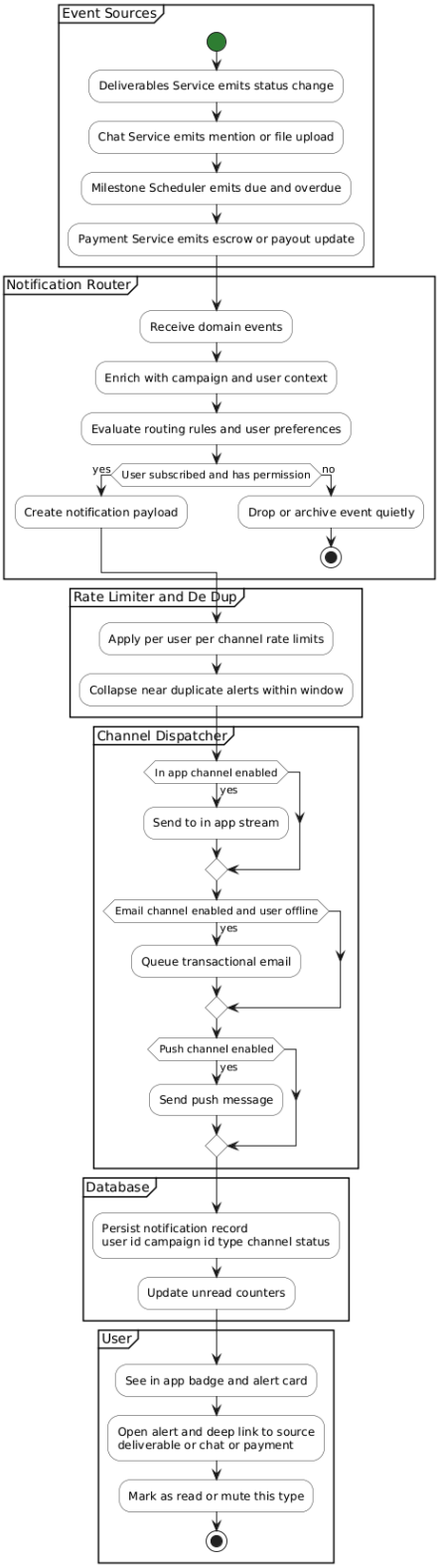


Figure 3.24: Activity Diagram (Real Time Campaign Alerts)

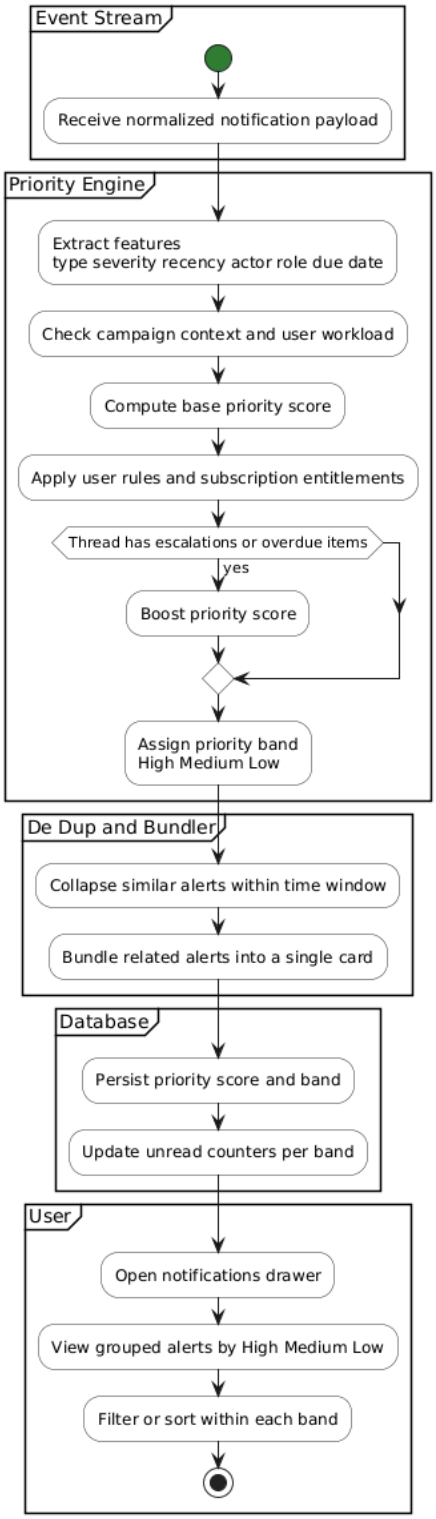


Figure 3.25: Activity Diagram (Priority Sorting)

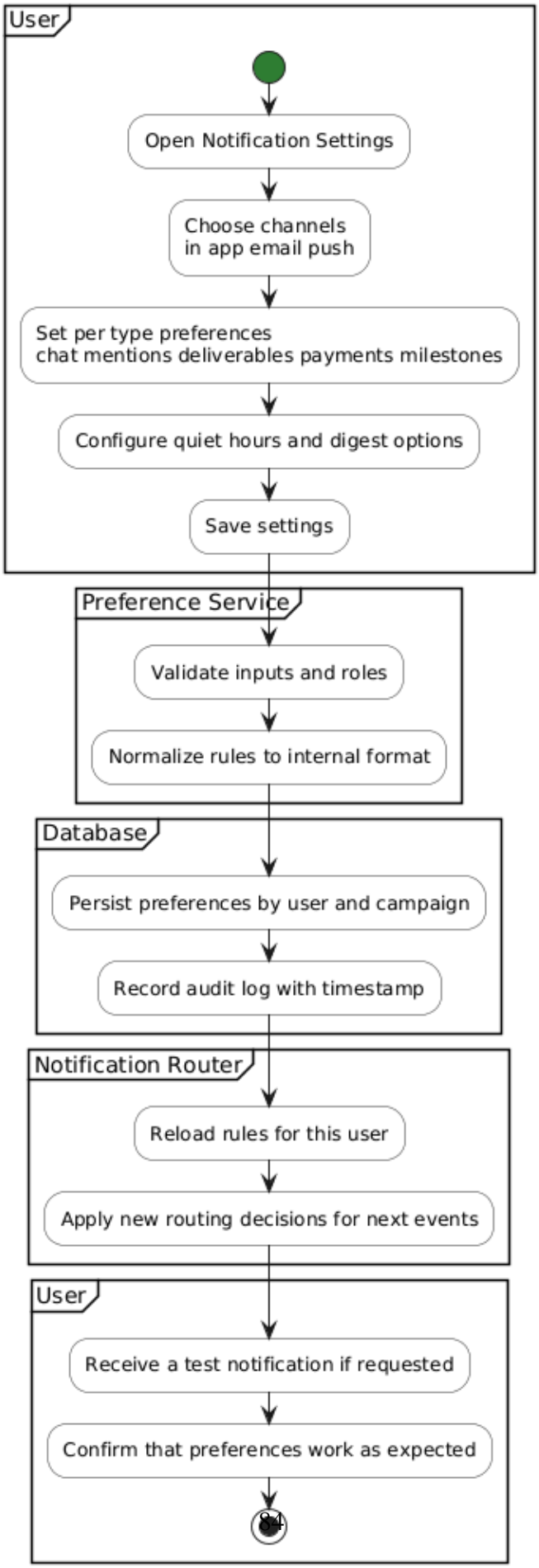


Figure 3.26: Activity Diagram (Preference Management)

3.2.1.9 Admin, User Profile and Personalization

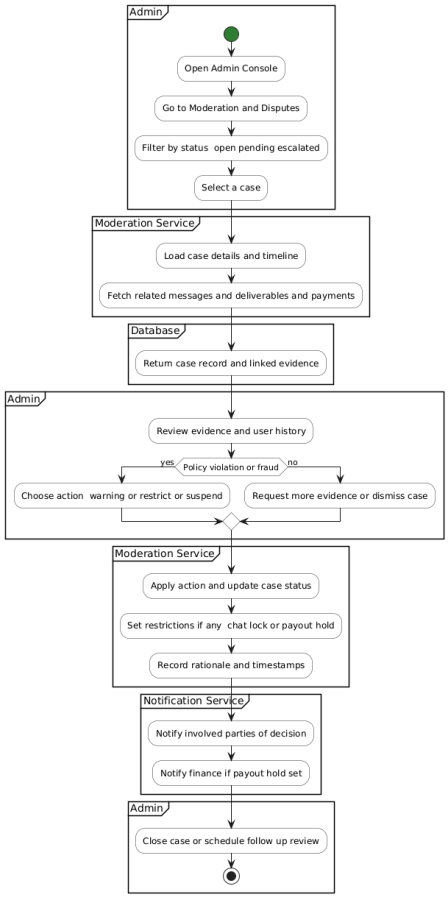


Figure 3.27: Activity Diagram (Moderate Users and Disputes)

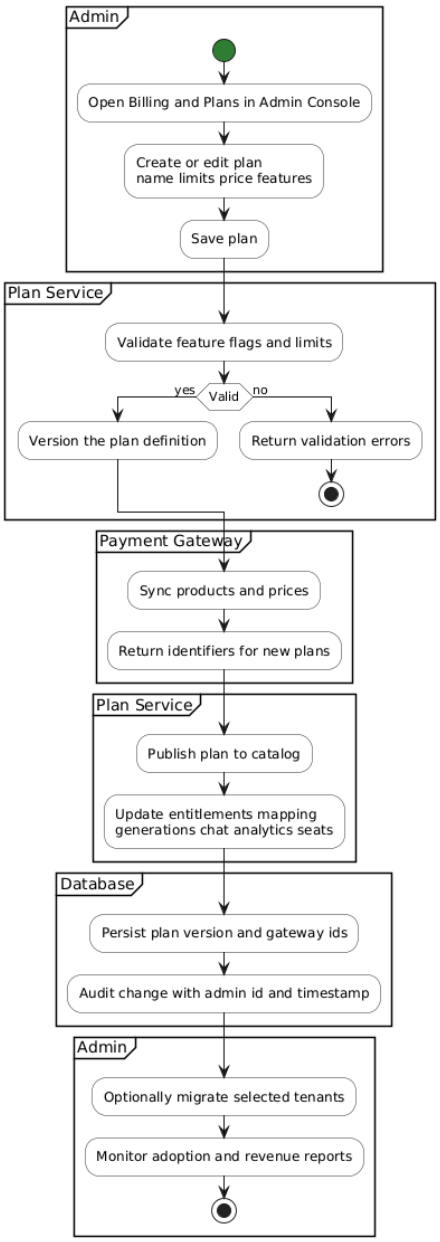


Figure 3.28: Activity Diagram (Manage Plans and Billing)

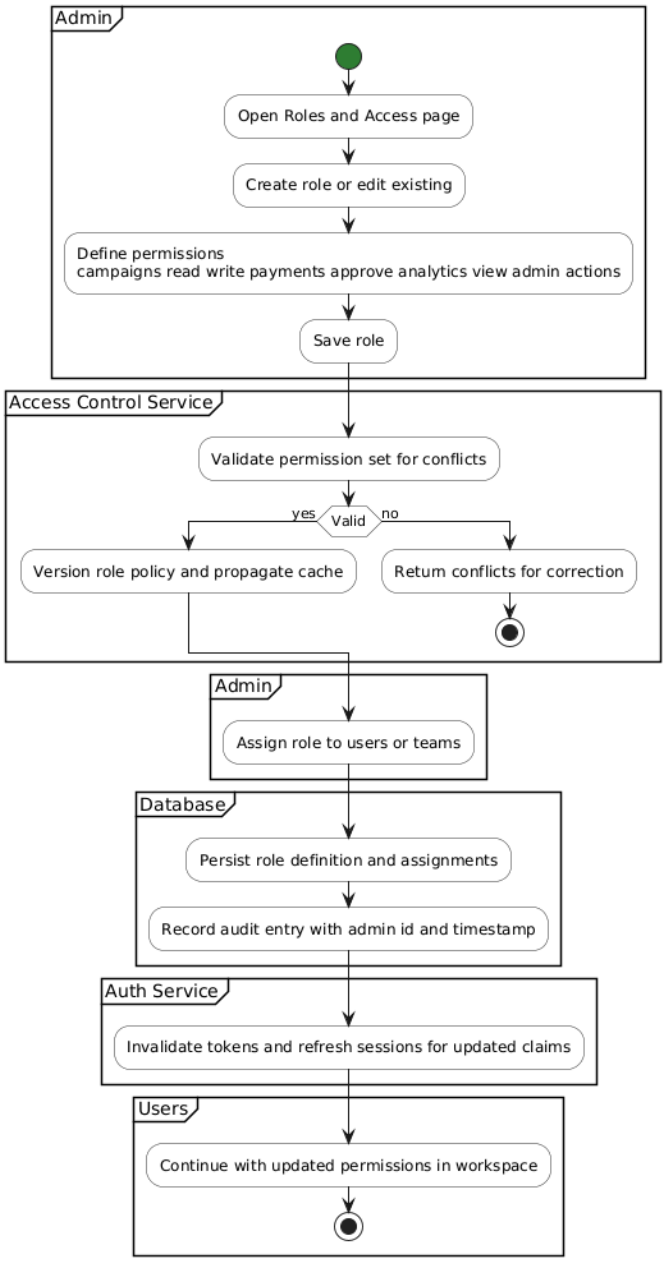


Figure 3.29: Activity Diagram (Configure Roles and Access)

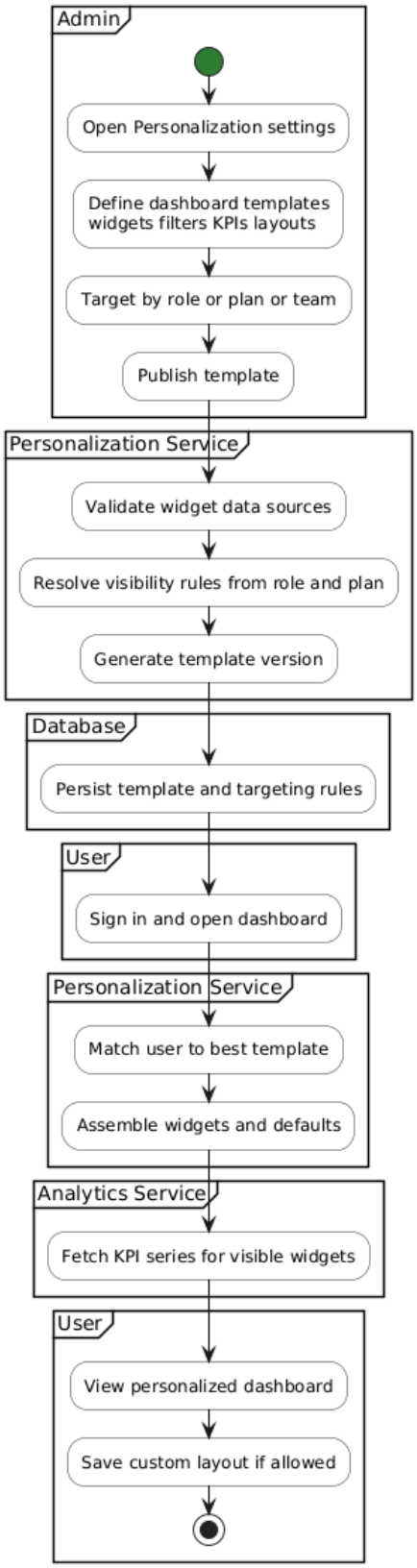


Figure 3.30: Activity Diagram (Generate Personalized Dashboards)

3.2.2 Data Flow Diagram

3.2.2.1 Level 0

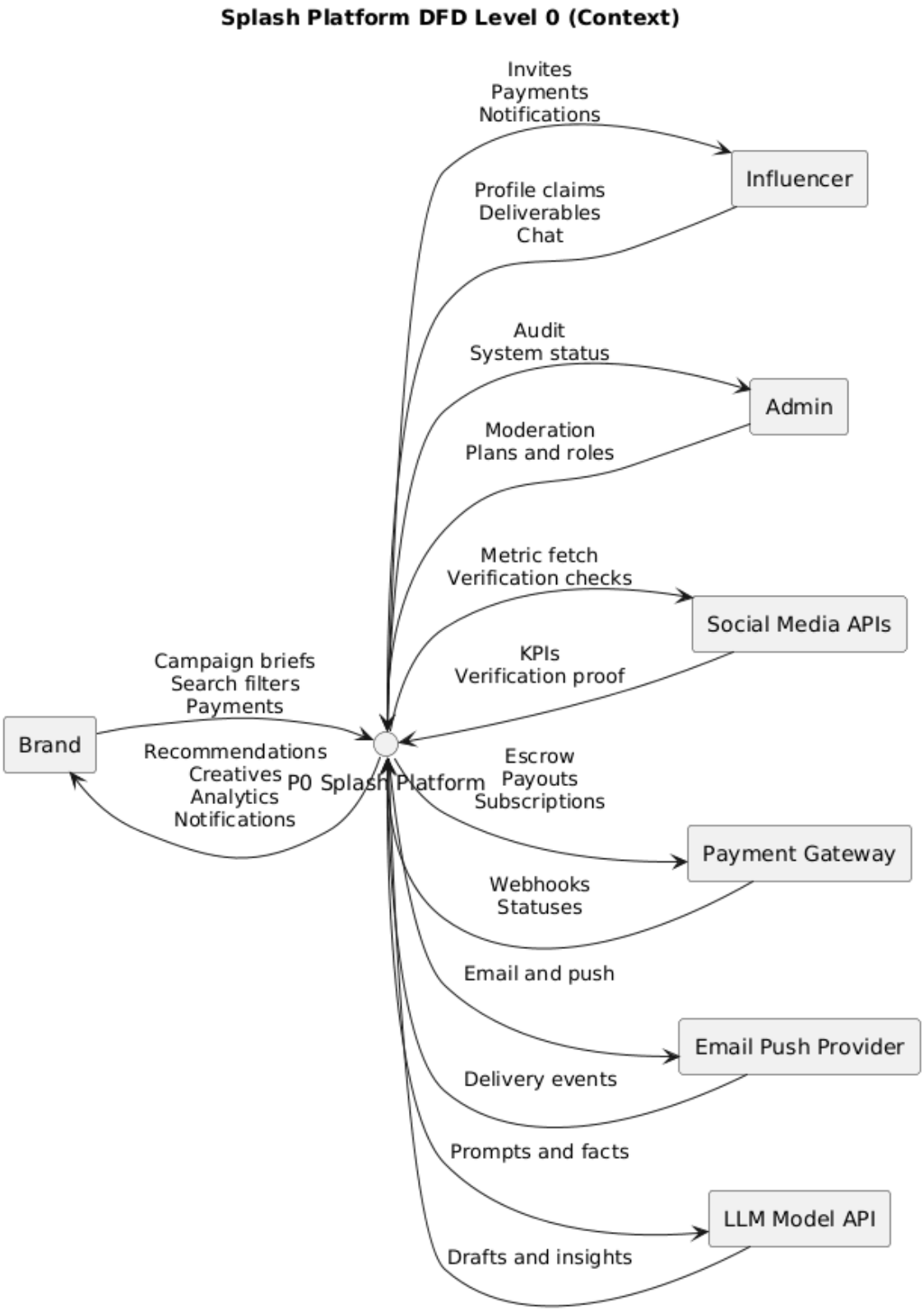


Figure 3.31: Data Flow Diagram- level 0

3.2.2.2 Level 1

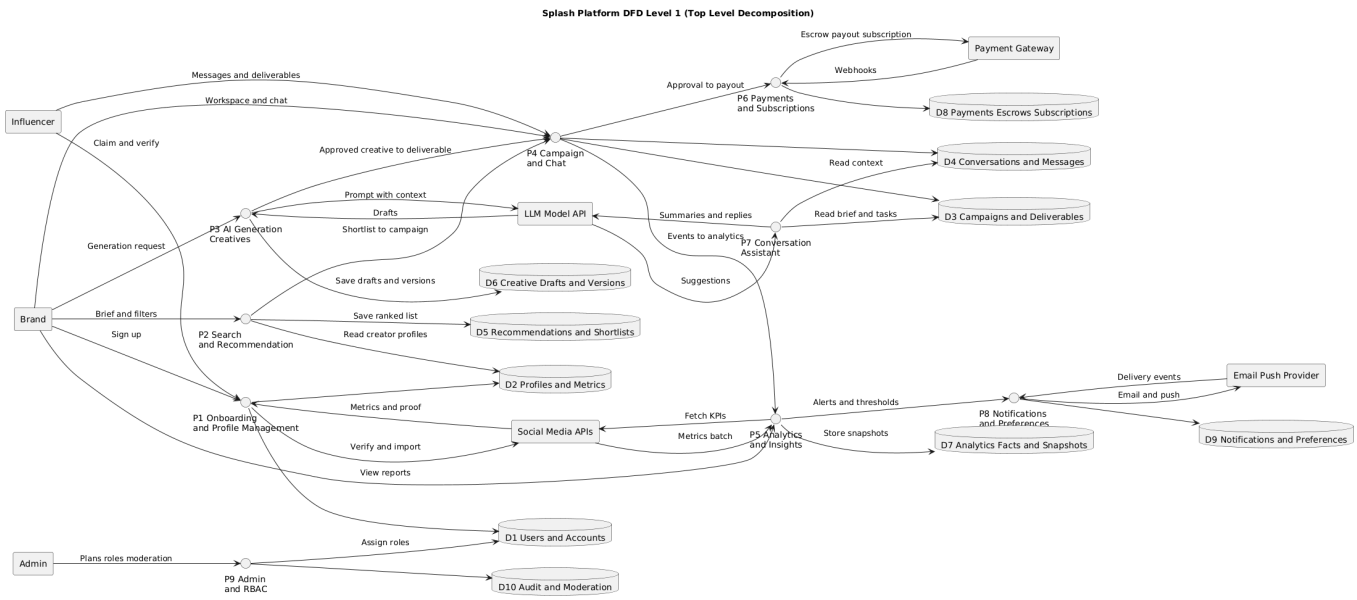


Figure 3.32: Data Flow Diagram level 1

3.2.2.3 Level 2

Data Gathering and Influencer On-boarding

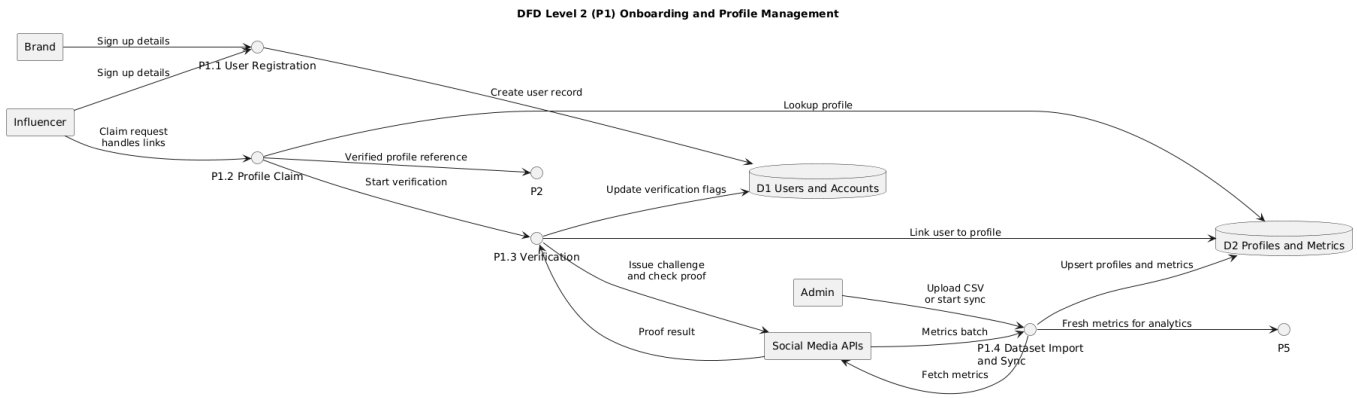


Figure 3.33: Data Flow Diagram level 2

Recommendation System

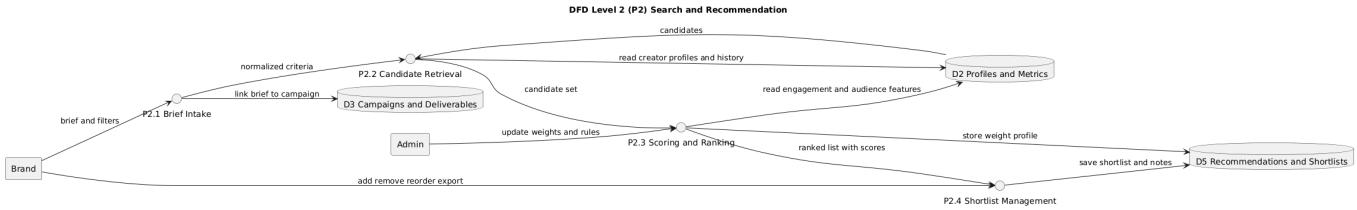


Figure 3.34: Data Flow Diagram level 2

Ad Generation

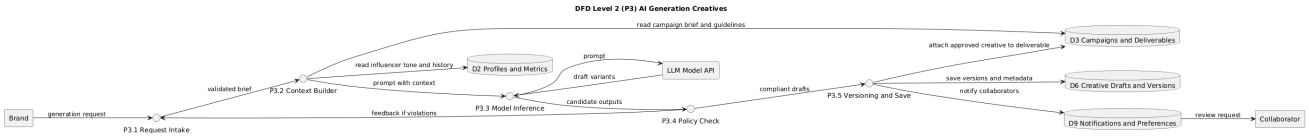


Figure 3.35: Data Flow Diagram level 2

Campaign Management and Chat

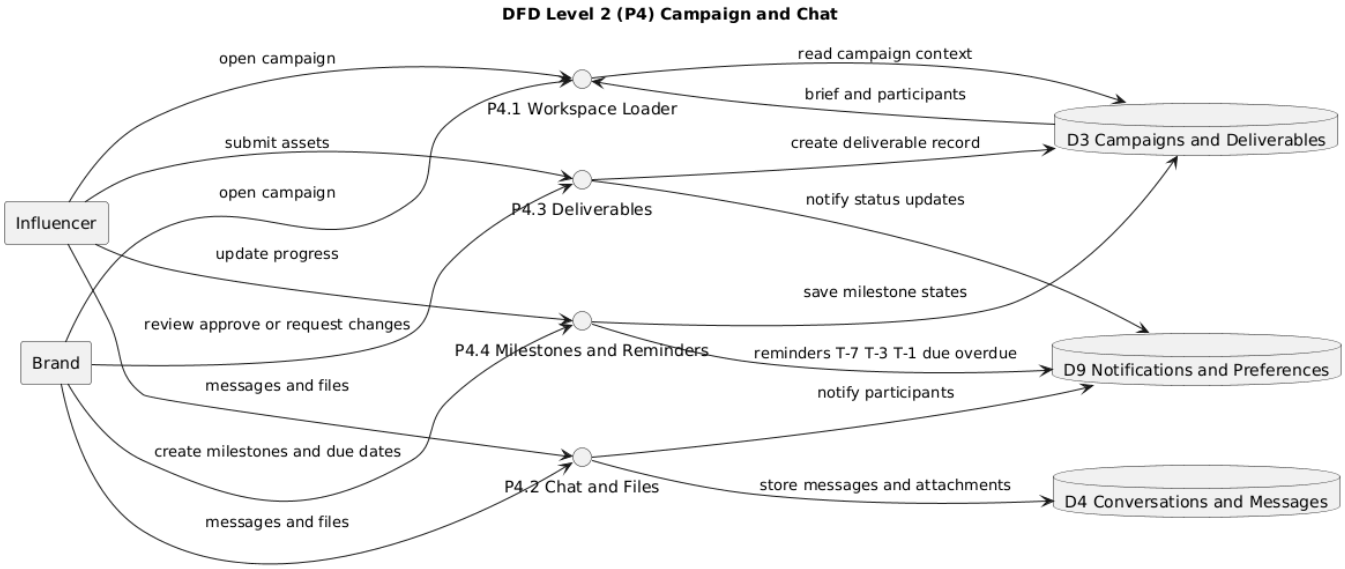


Figure 3.36: Data Flow Diagram level 2

Analytics Dashboard

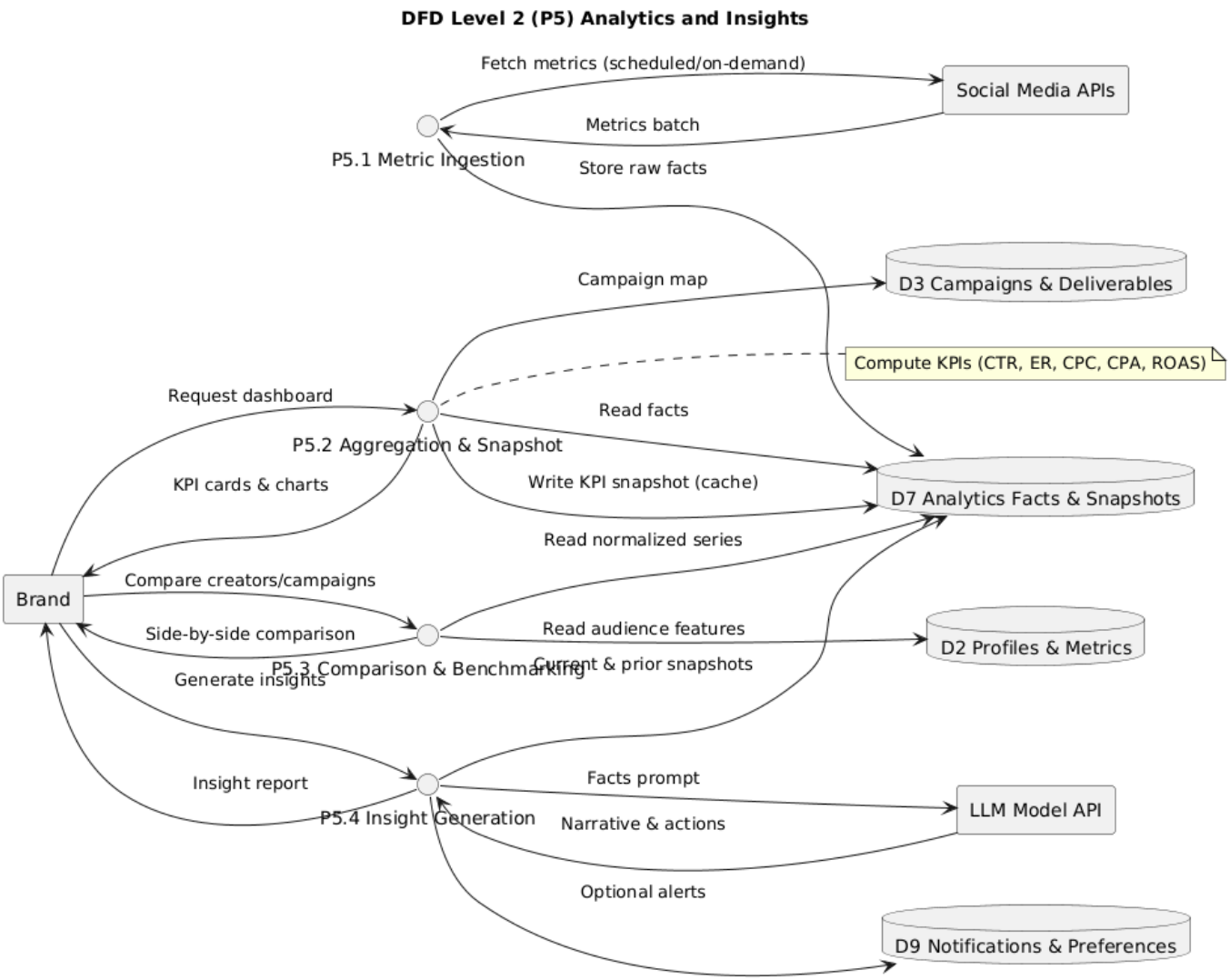


Figure 3.37: Data Flow Diagram level 2

Payment Gateway

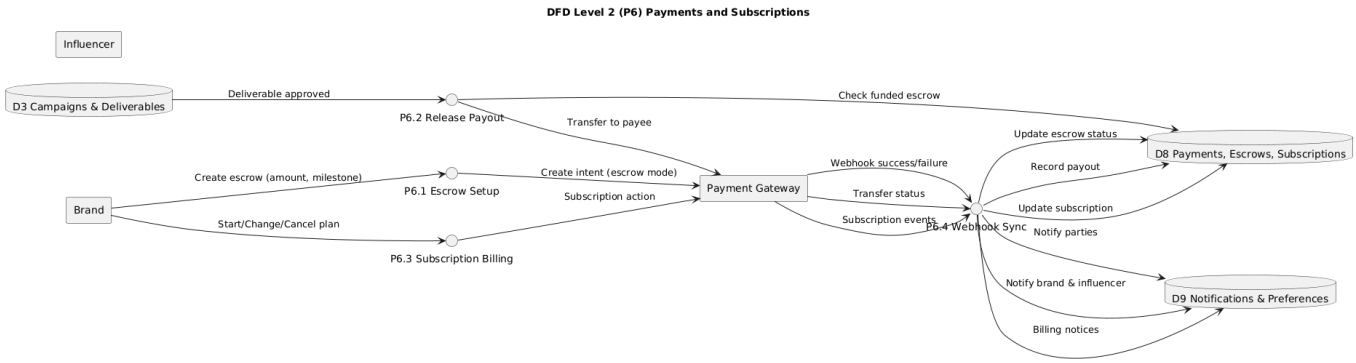


Figure 3.38: Data Flow Diagram level 2

Conversation Intelligence Assistant

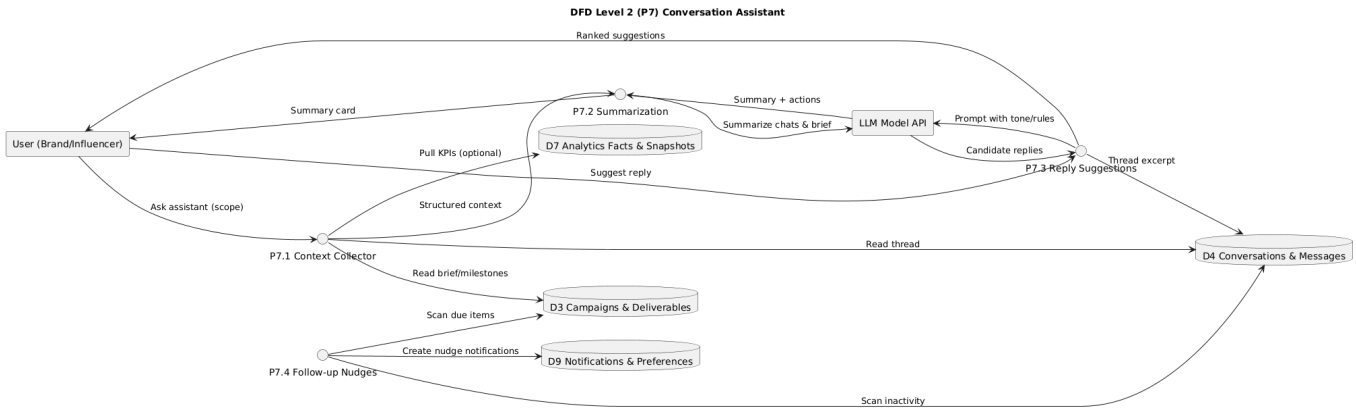


Figure 3.39: Data Flow Diagram level 2

Notification and Smart Alerts

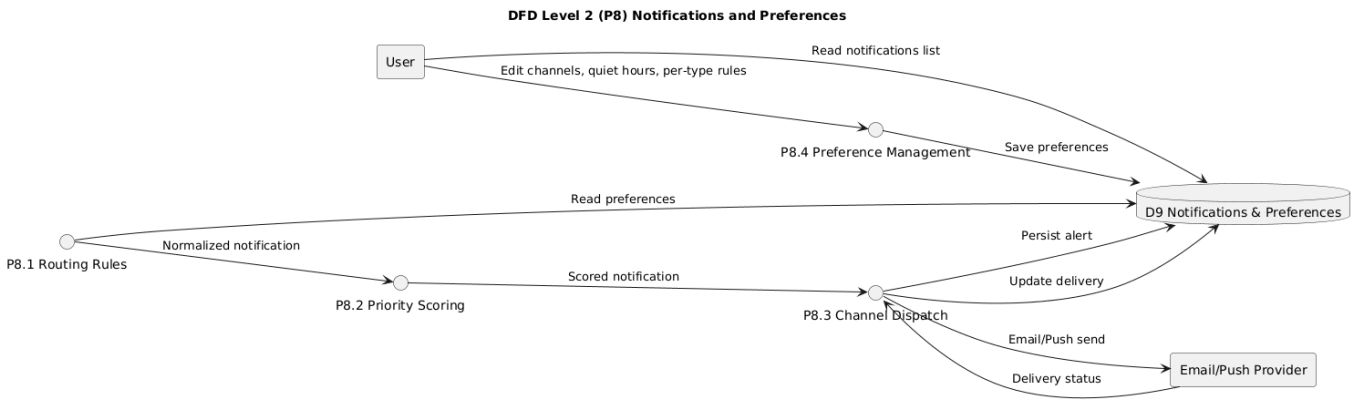


Figure 3.40: Data Flow Diagram level 2

Admin, User Profile and Personalization

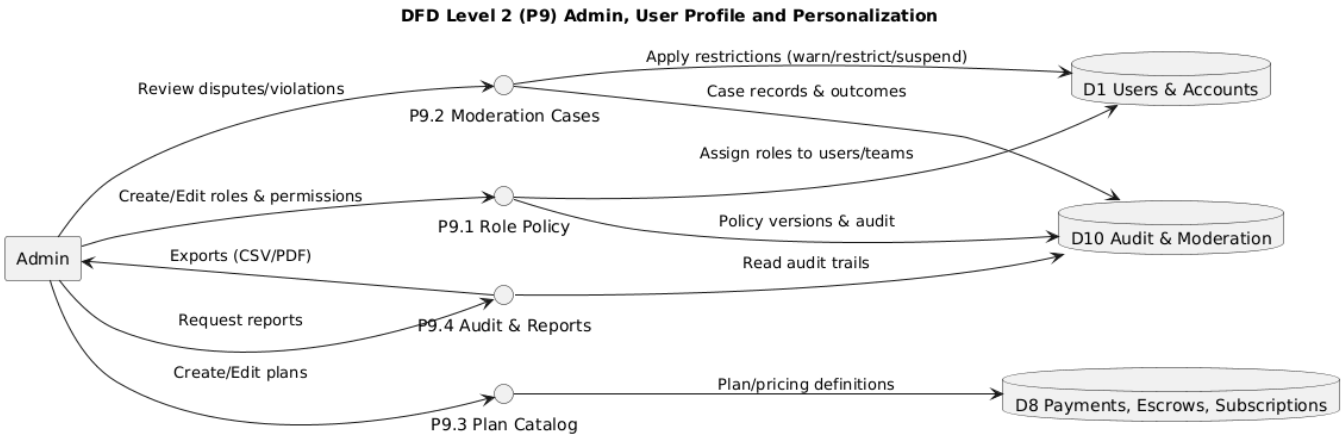


Figure 3.41: Data Flow Diagram level 2

3.2.3 System Level Sequence Diagram

3.2.3.1 Data Gathering and Influencer On-boarding

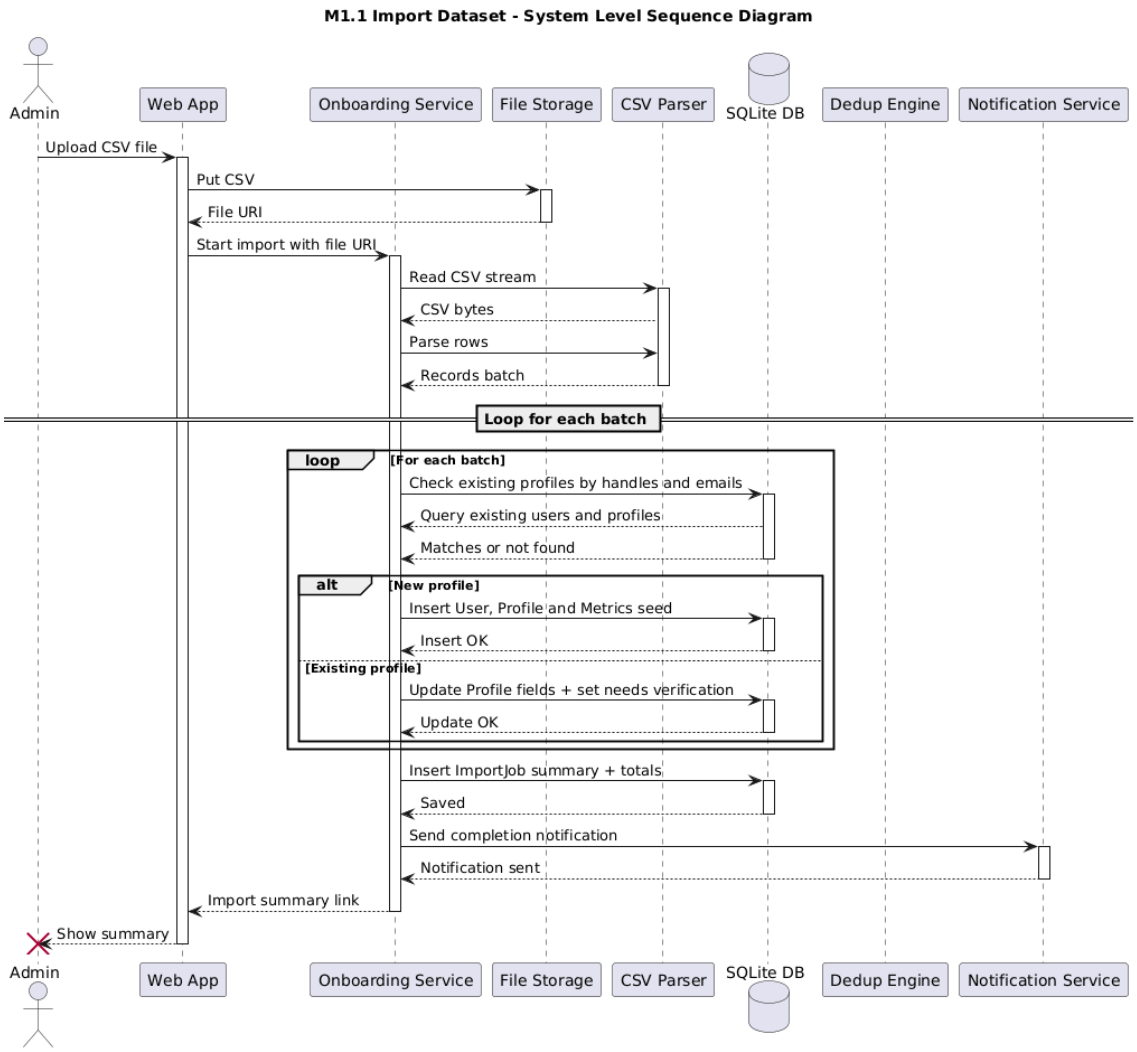


Figure 3.42: Import Dataset

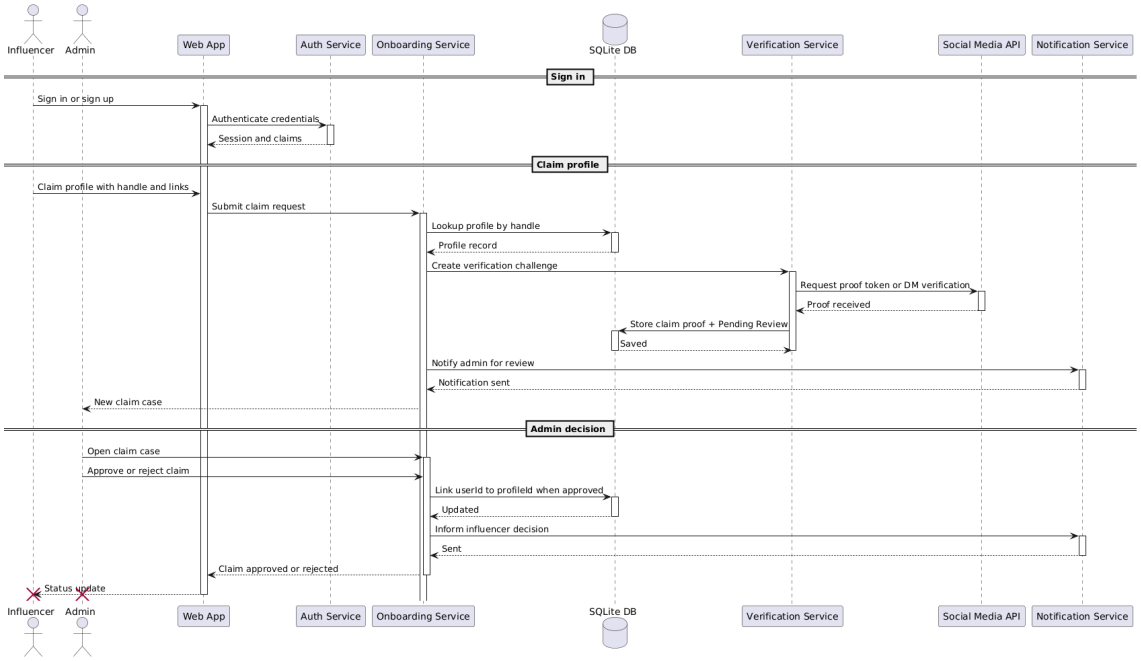


Figure 3.43: Claim Existing Profile

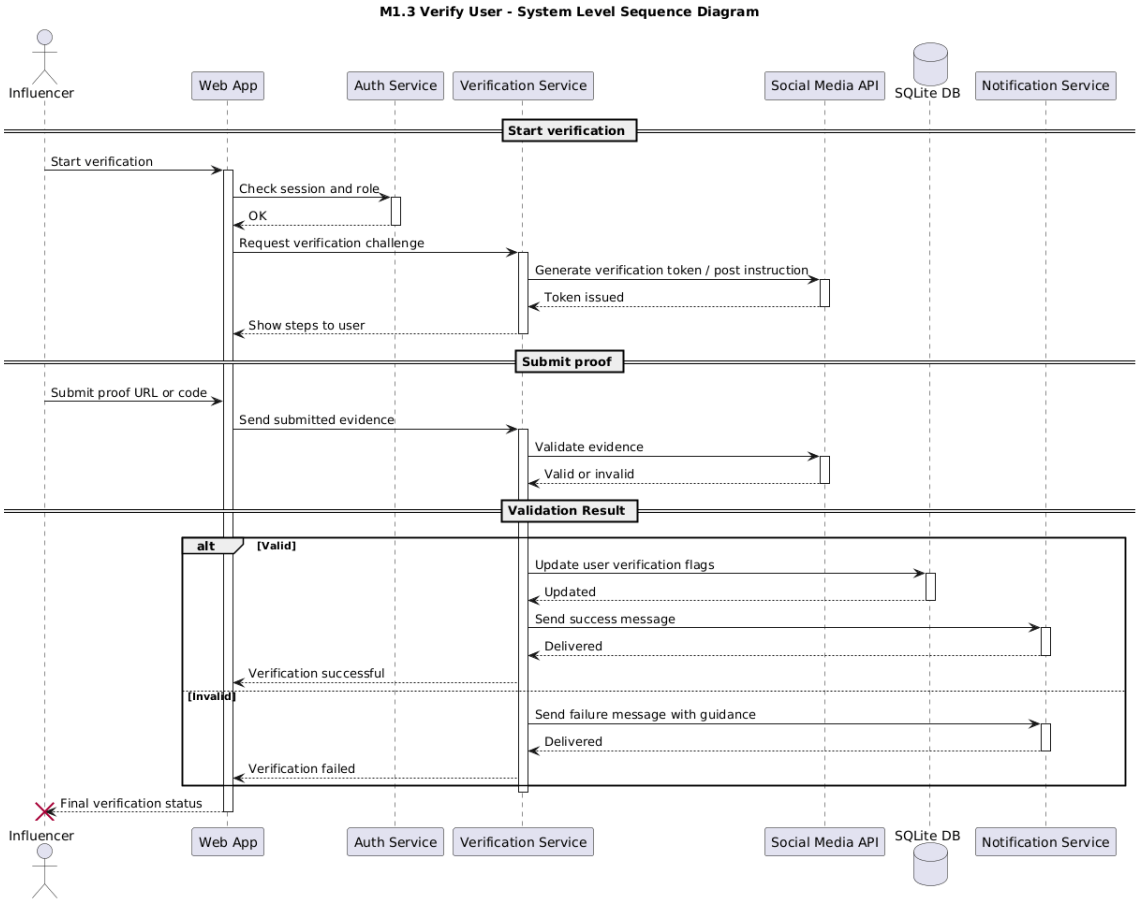


Figure 3.44: Verify User

3.2.3.2 Recommendation System

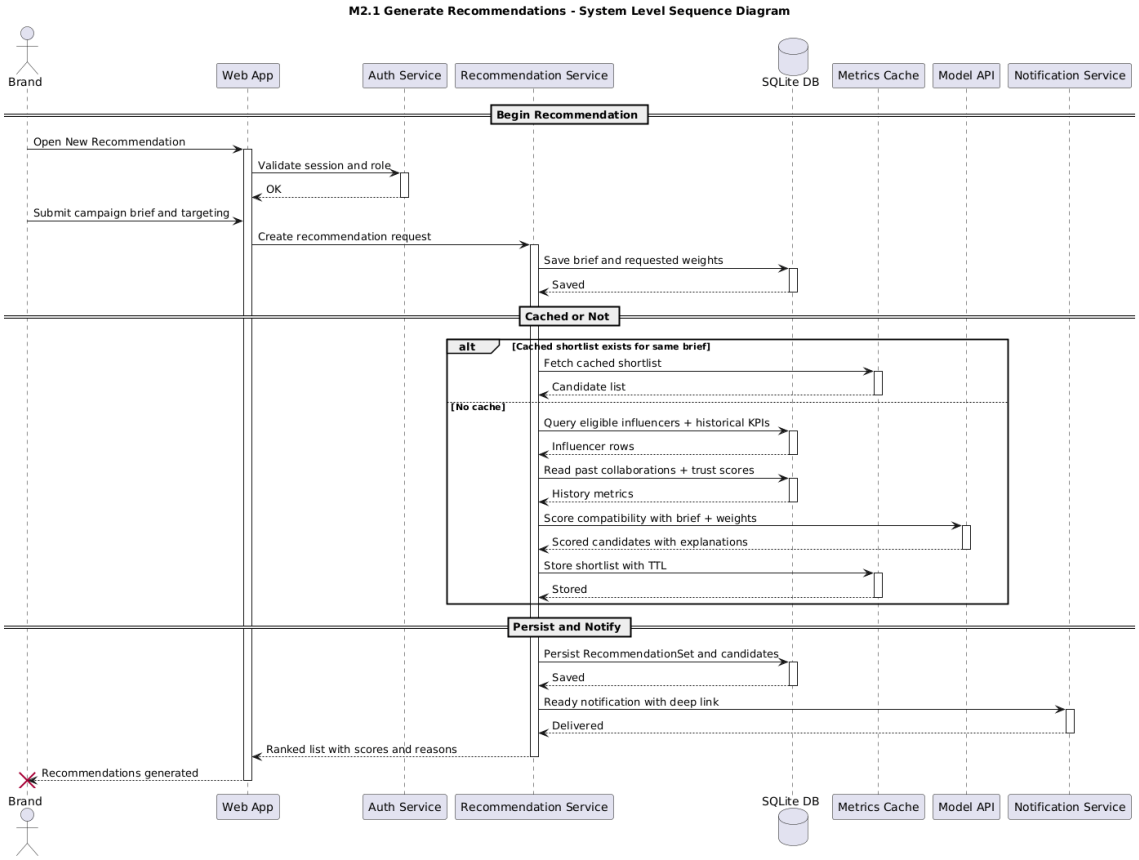


Figure 3.45: Generate Recommendations

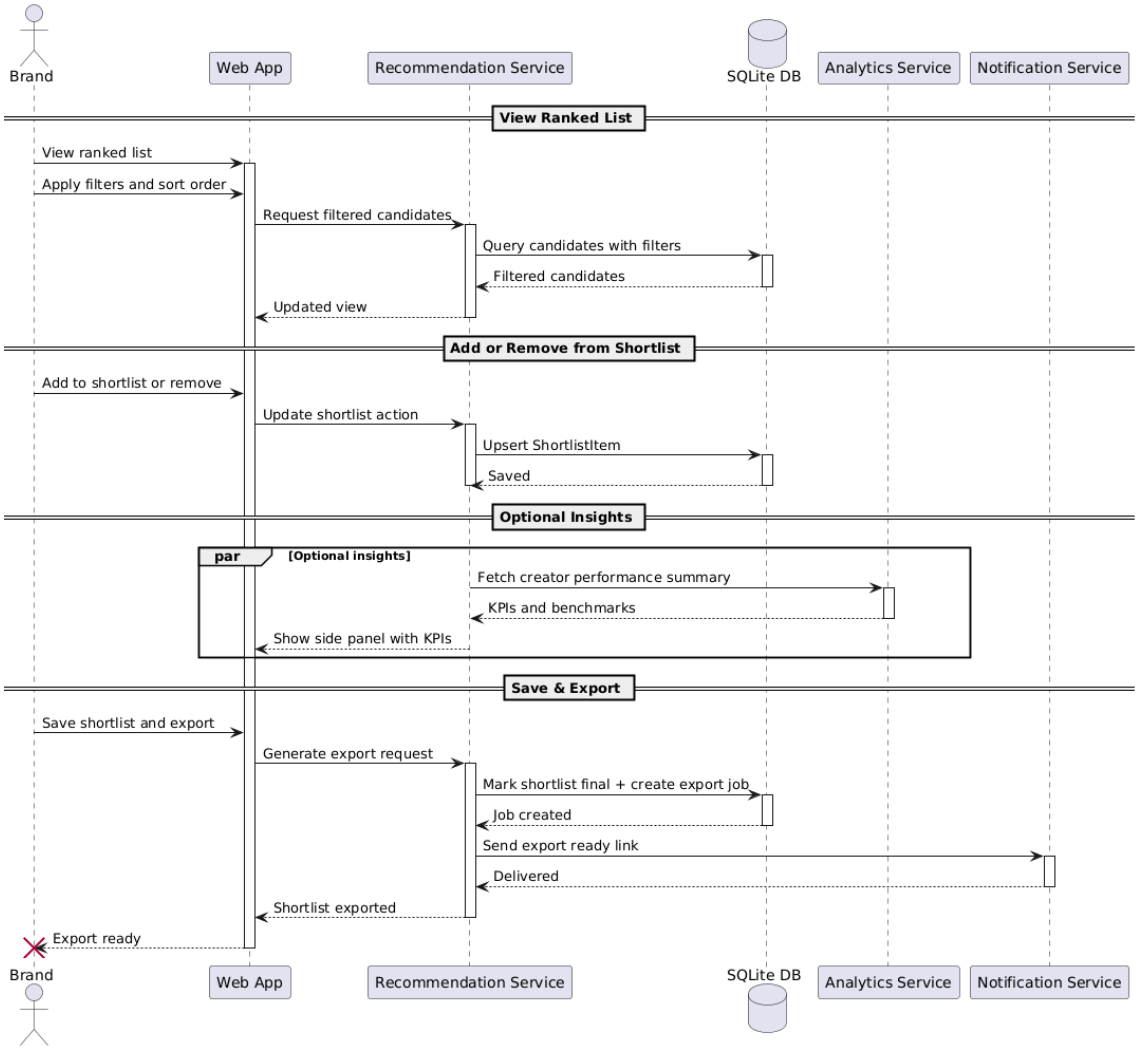


Figure 3.46: Shortlist Influencers

3.2.3.3 Ad Generation

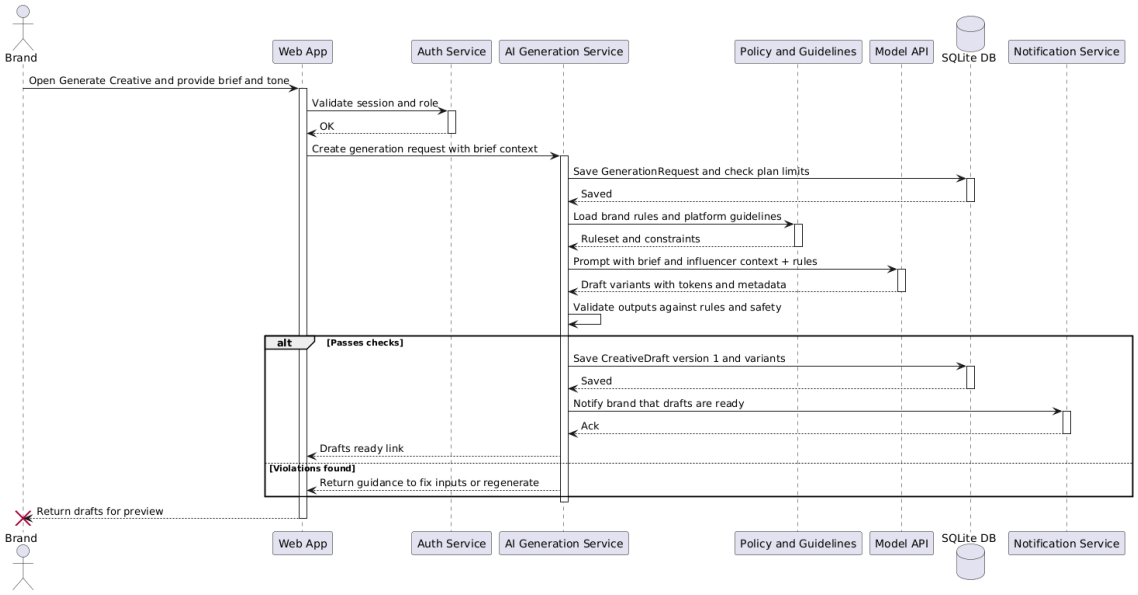


Figure 3.47: Generate Creatives

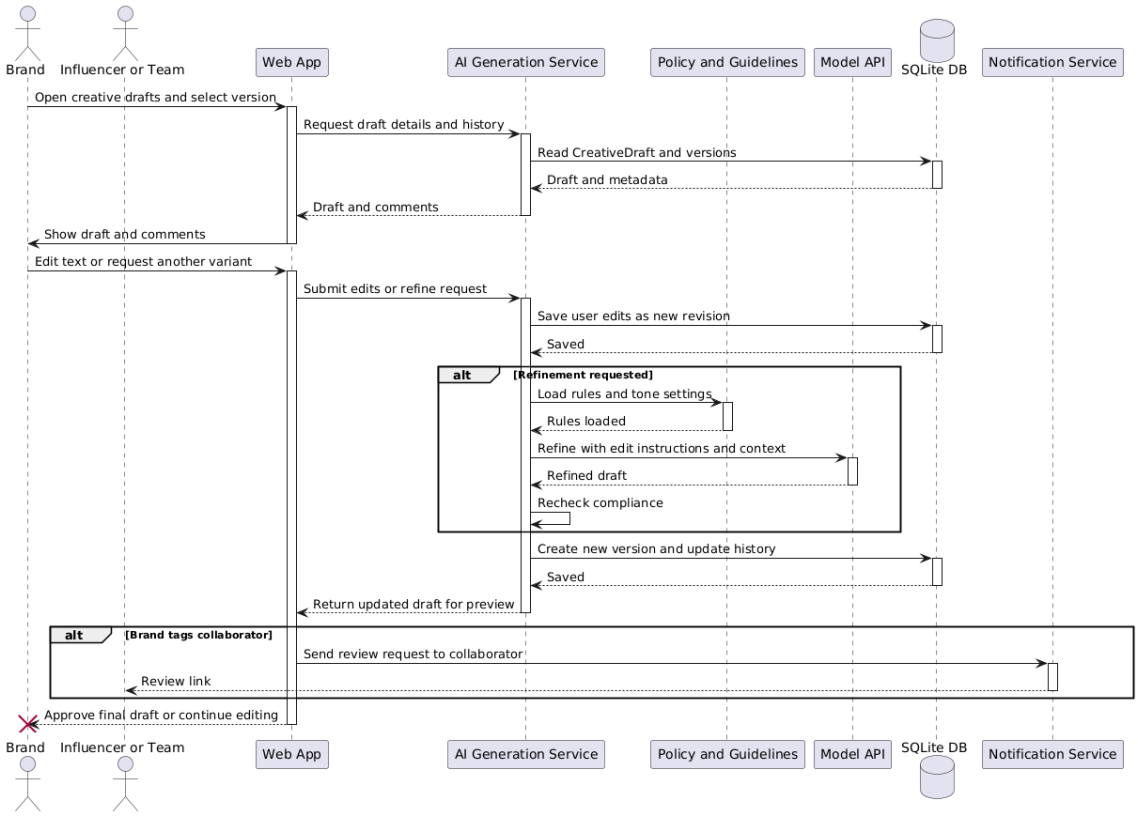


Figure 3.48: Review and Edit Creatives

3.2.3.4 Campaign Management and Chat

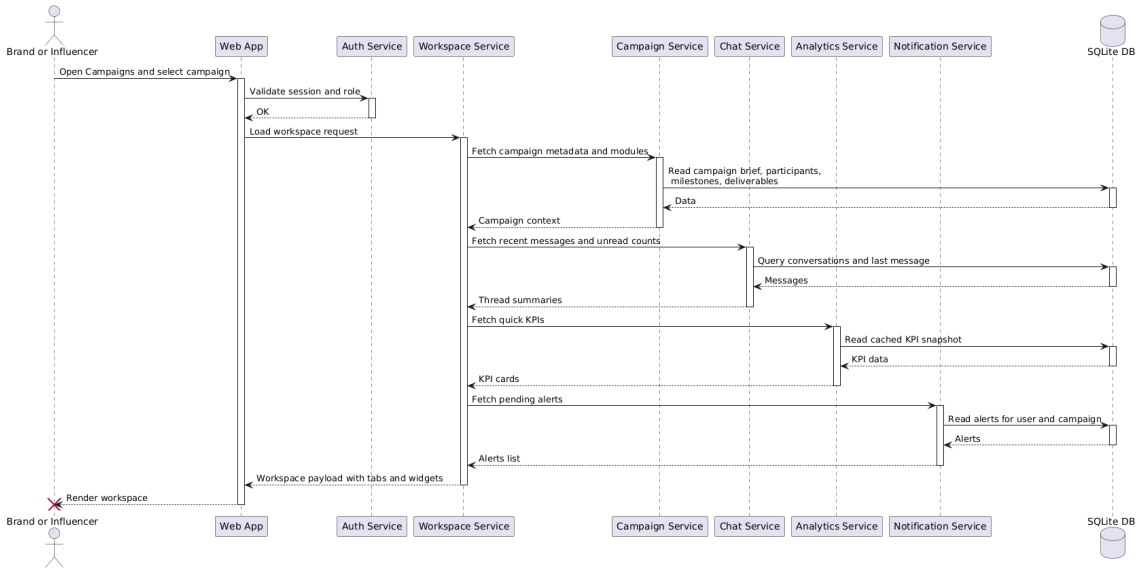


Figure 3.49: Open Campaign Workspace

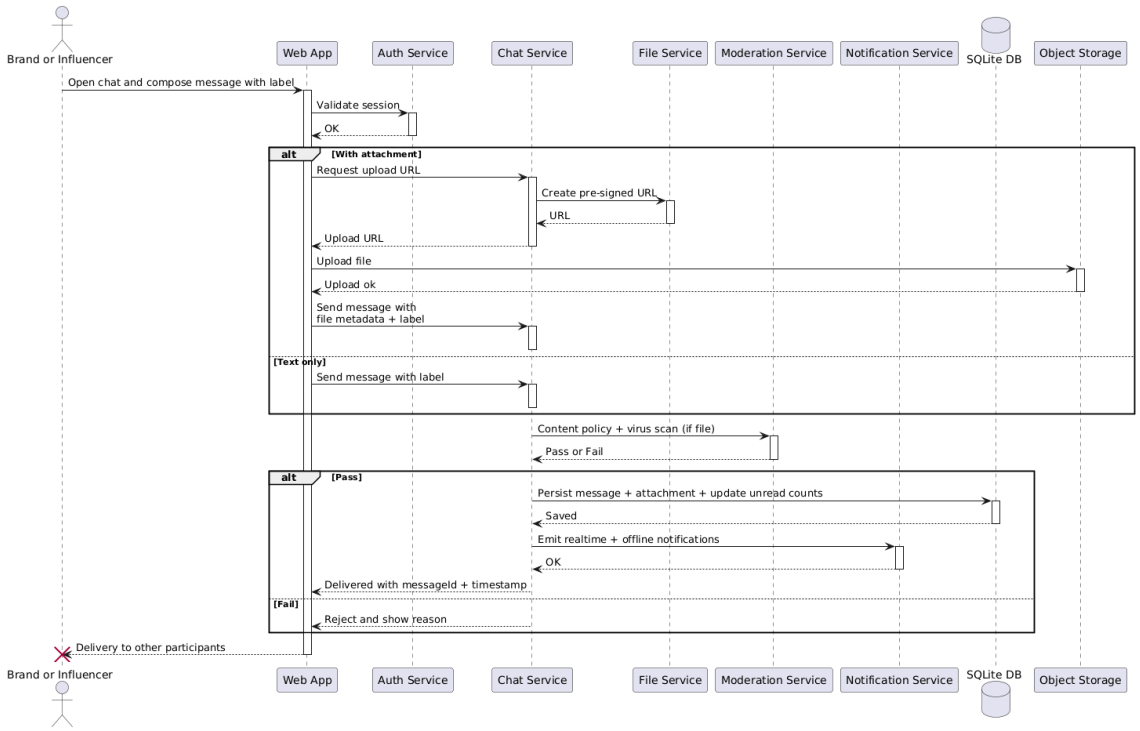


Figure 3.50: Chat and File Exchange

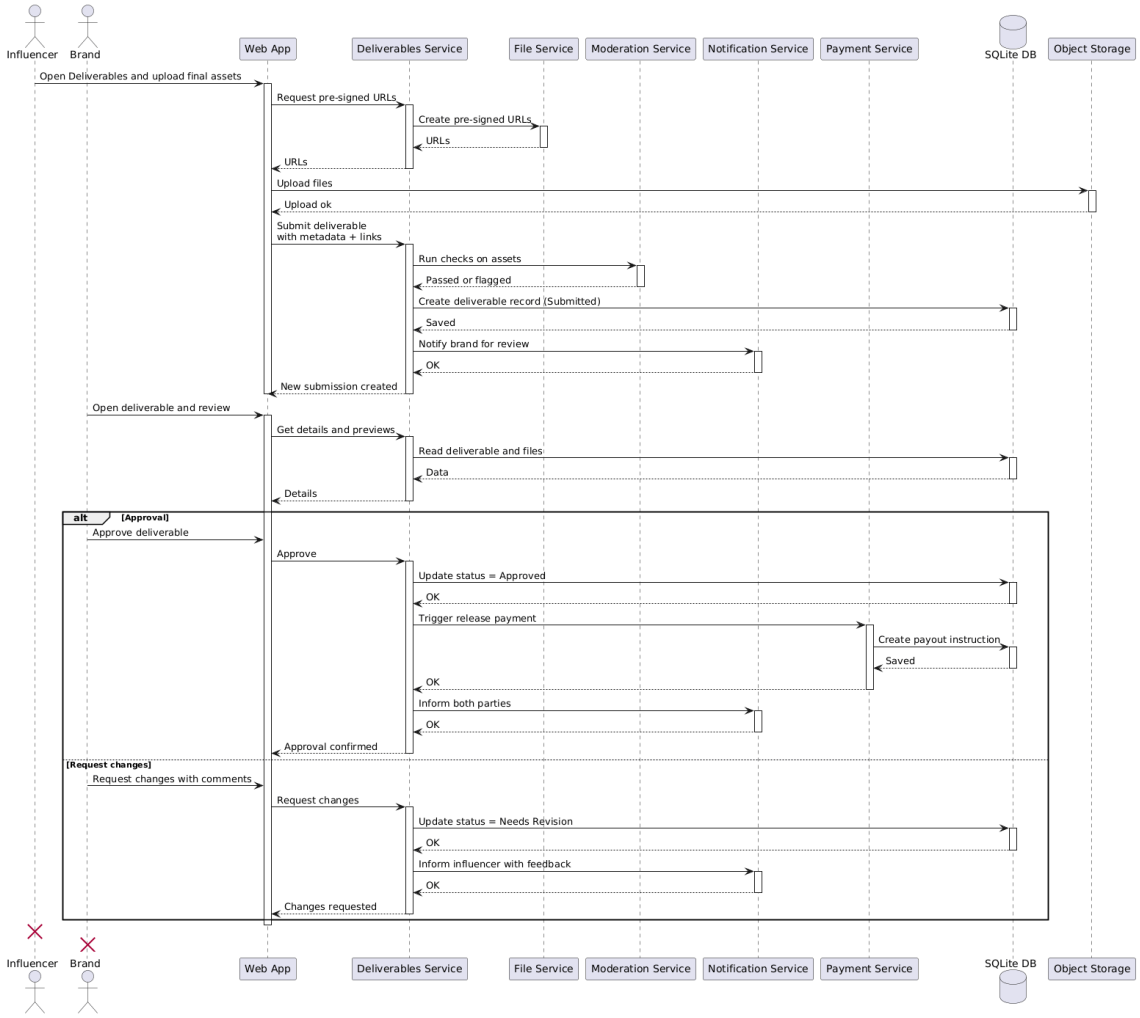


Figure 3.51: Submit and Approve Deliverables

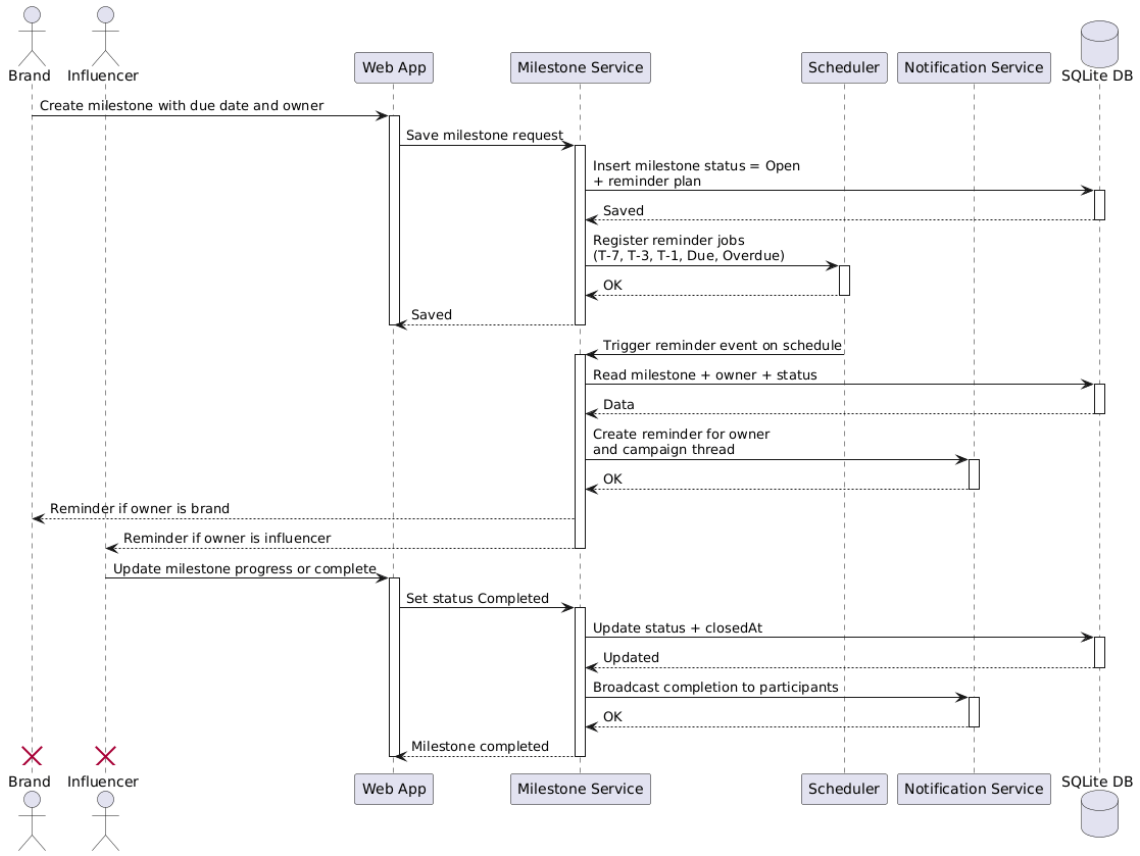


Figure 3.52: Milestone Tracking and Reminders

3.2.3.5 Analytics Dashboard

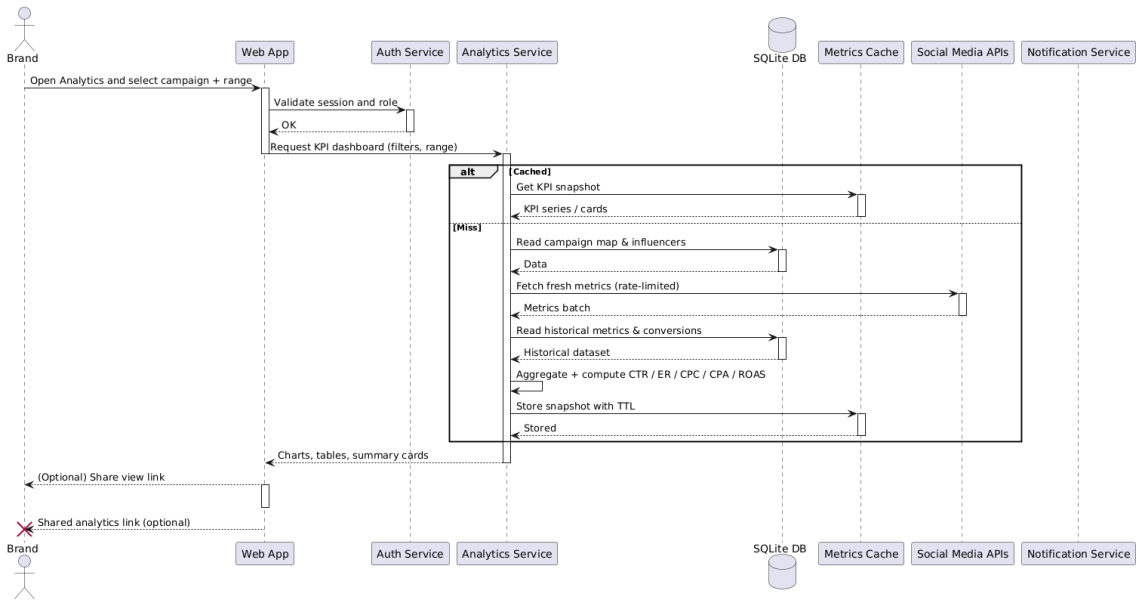


Figure 3.53: View Live Campaign Performance

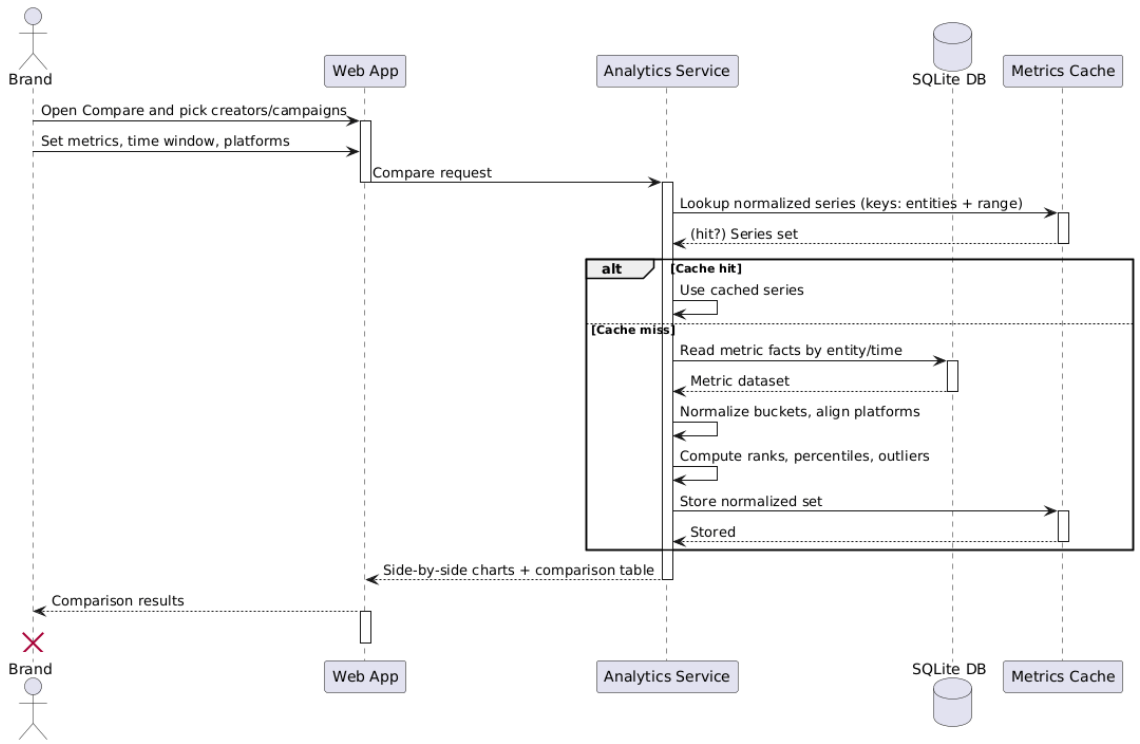


Figure 3.54: Compare Creators and Campaigns

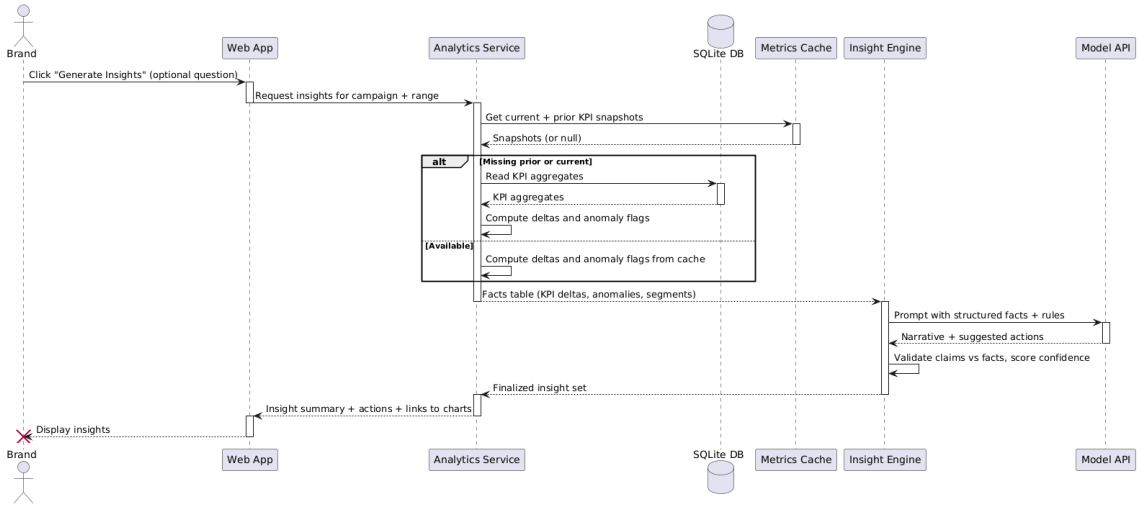


Figure 3.55: Generate Insight Prompts

3.2.3.6 Payment Gateway

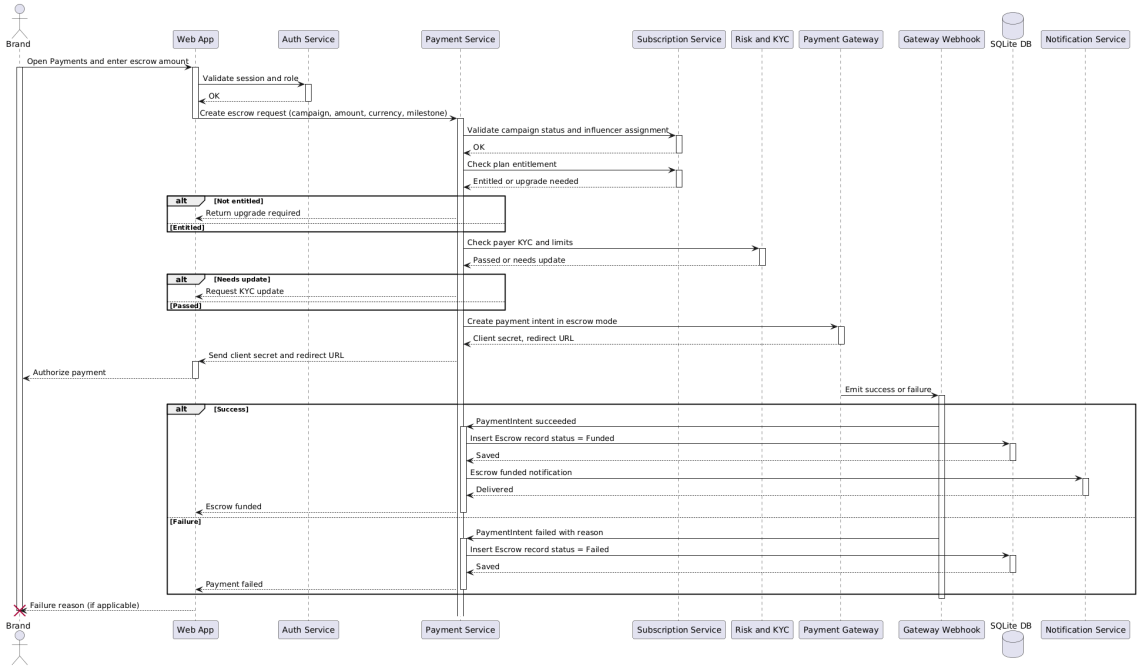


Figure 3.56: Create Escrow for Campaign

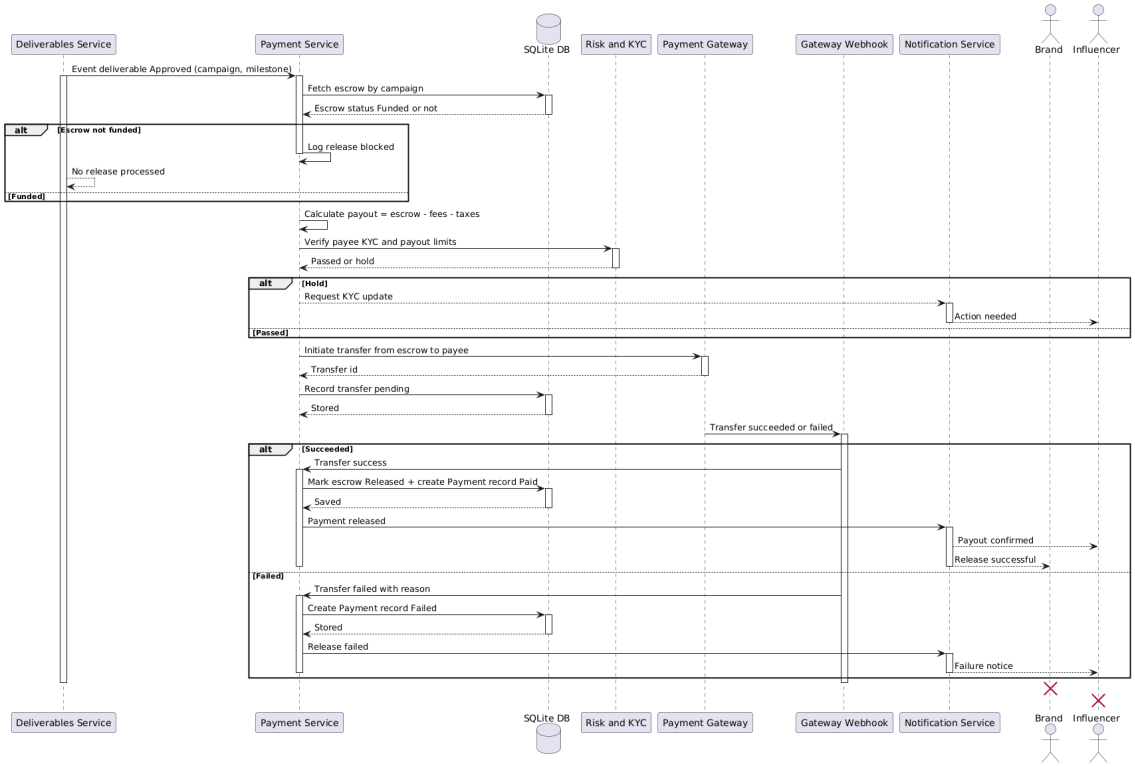


Figure 3.57: Release Payment on Approval

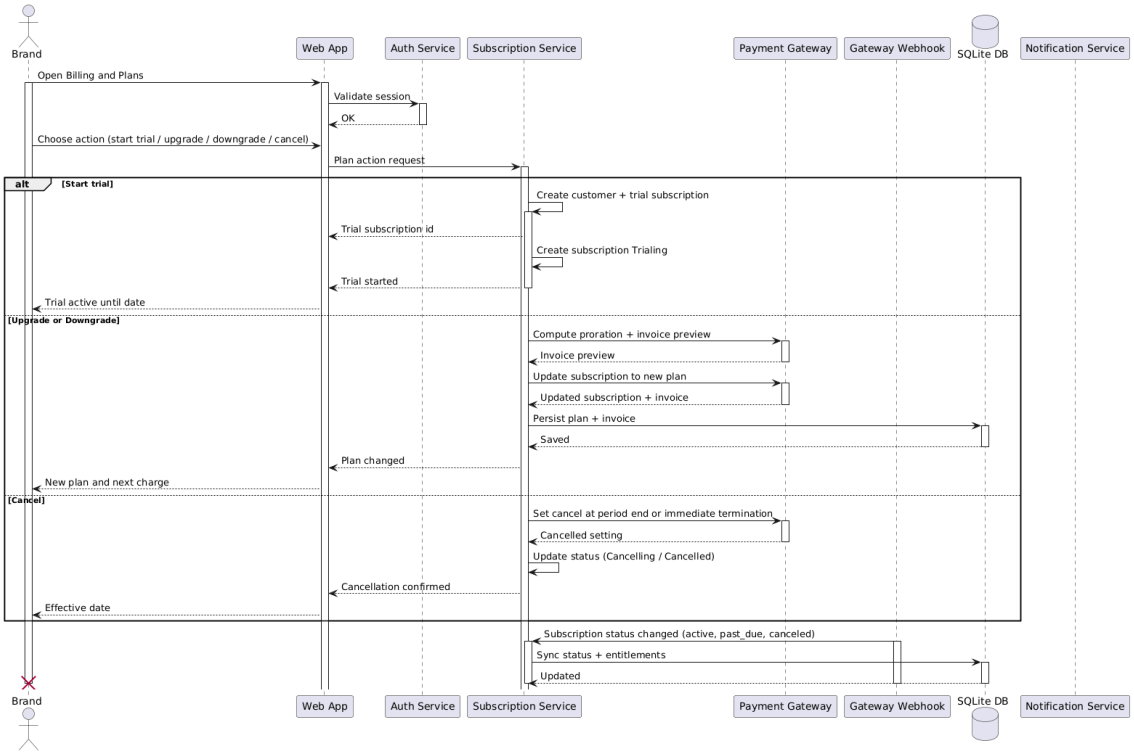


Figure 3.58: Manage Subscriptions

3.2.3.7 Conversation Intelligence Assistant

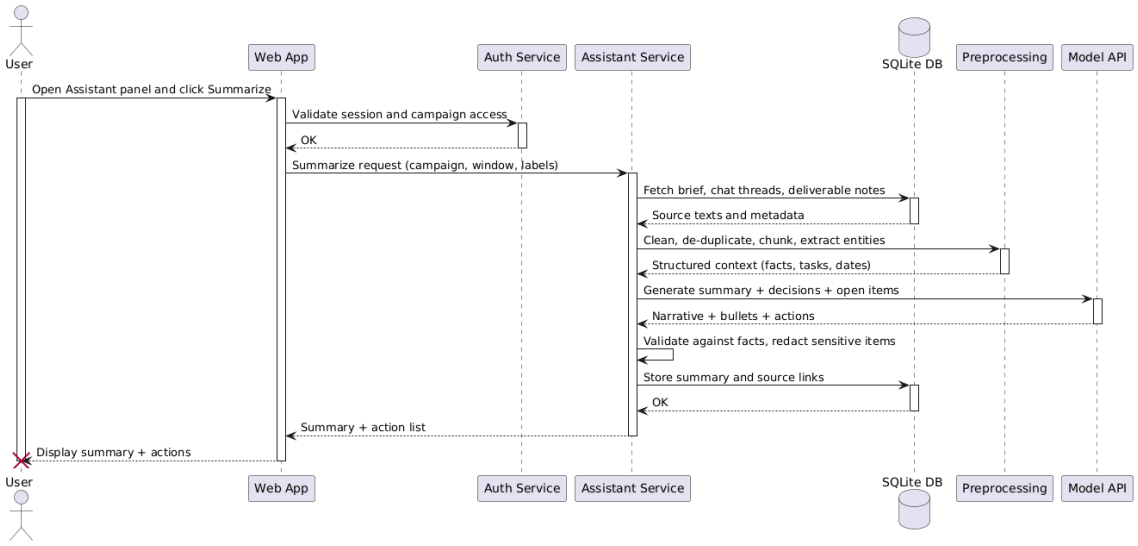


Figure 3.59: Summarize Chats and Briefs

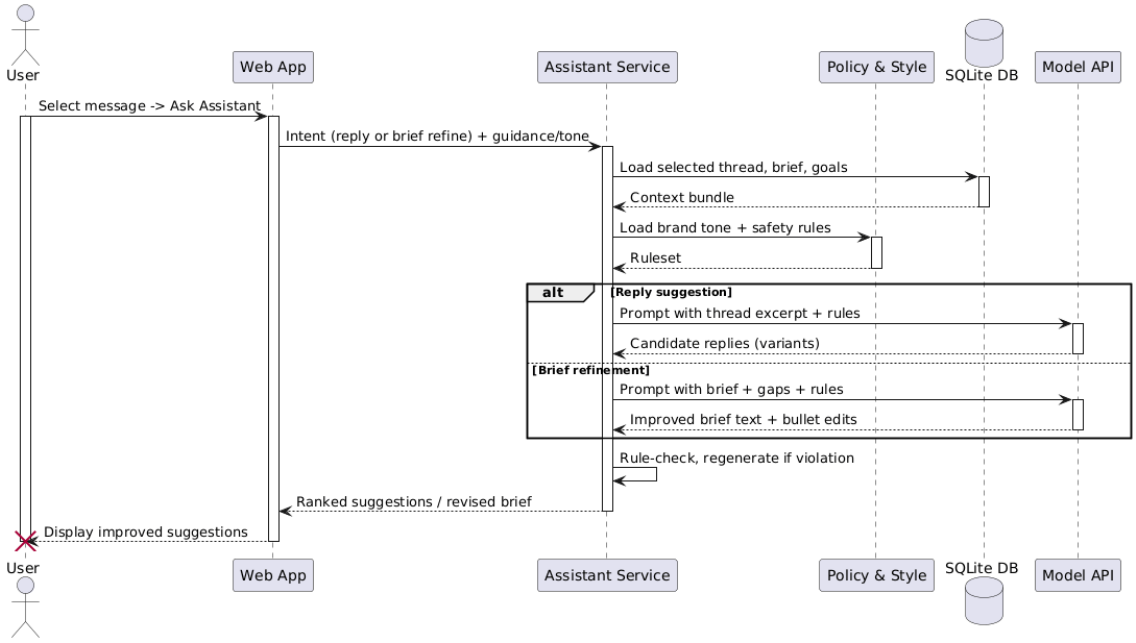


Figure 3.60: Suggest Replies and Brief Improvements

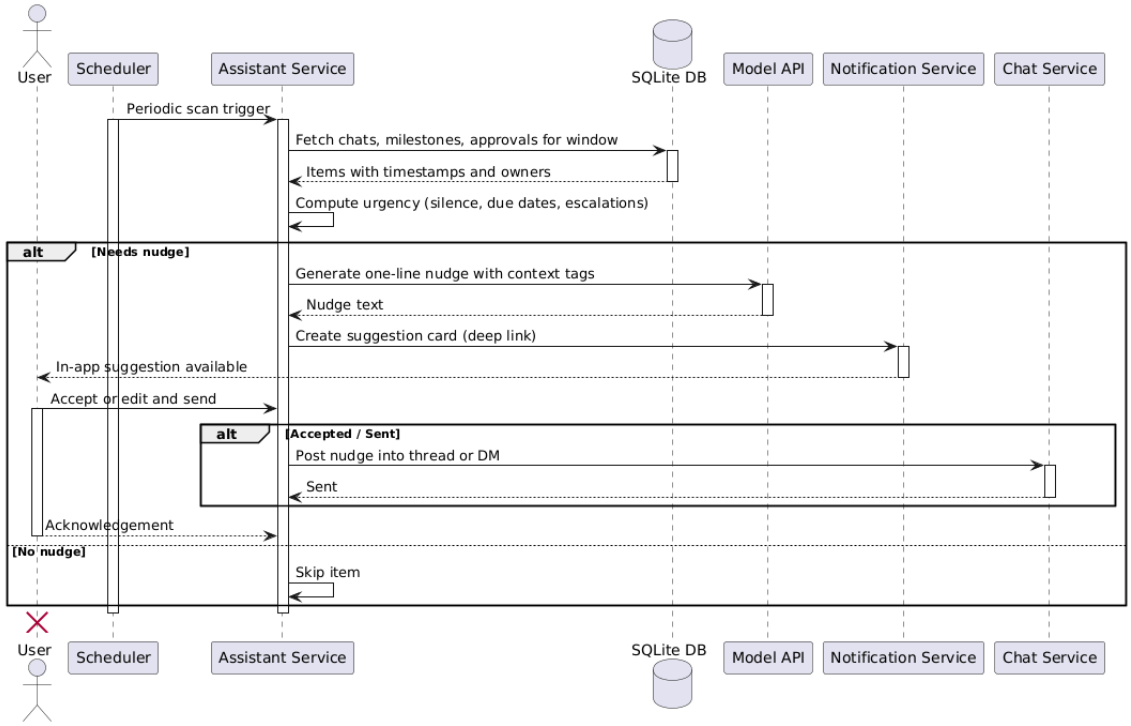


Figure 3.61: Follow Up Nudges

3.2.3.8 Notification and Smart Alerts

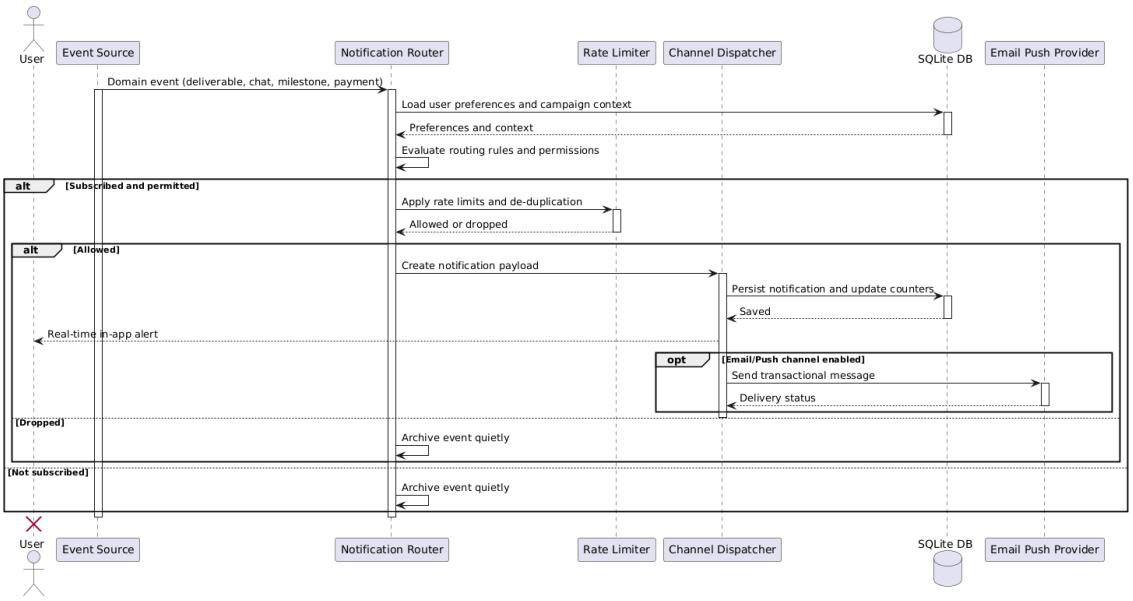


Figure 3.62: Real Time Campaign Alerts

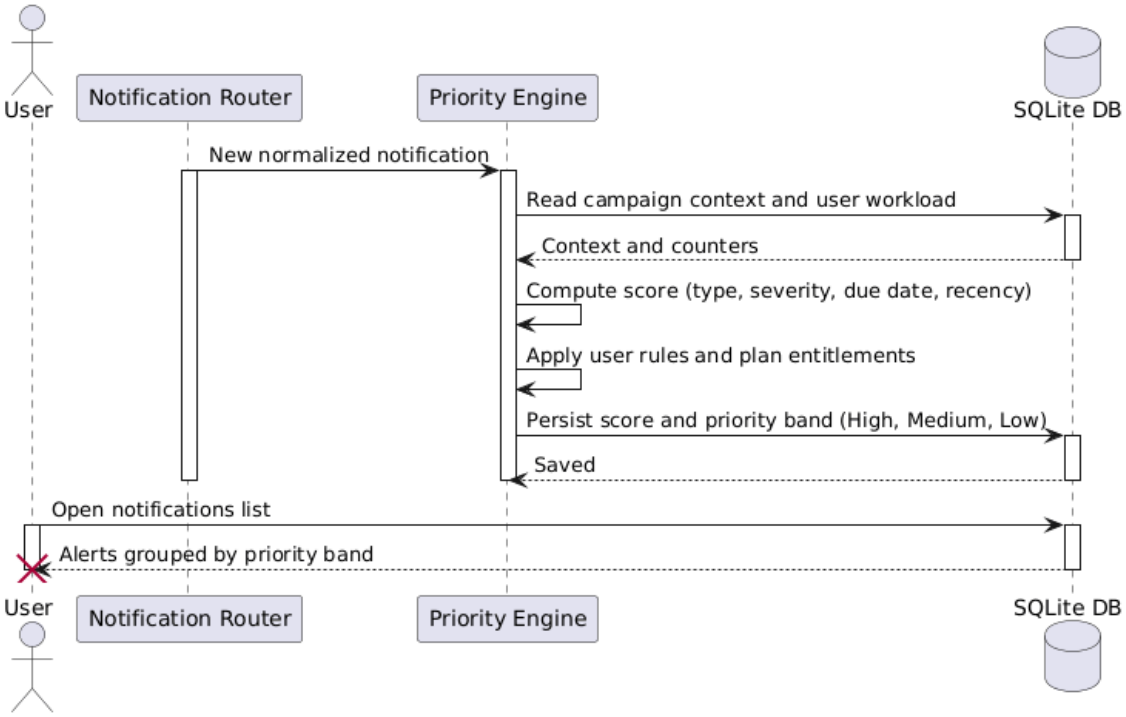


Figure 3.63: Priority Sorting

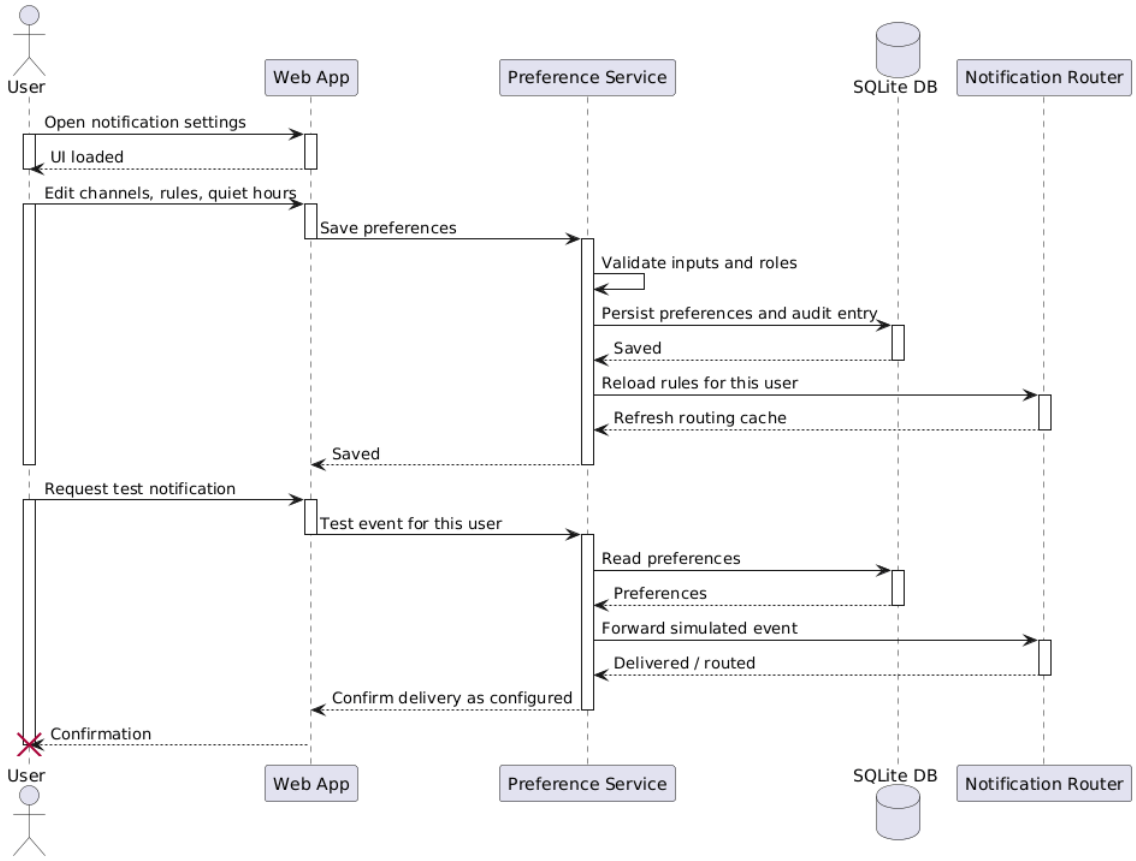


Figure 3.64: Preference Management

3.2.3.9 Admin, User Profile and Personalization

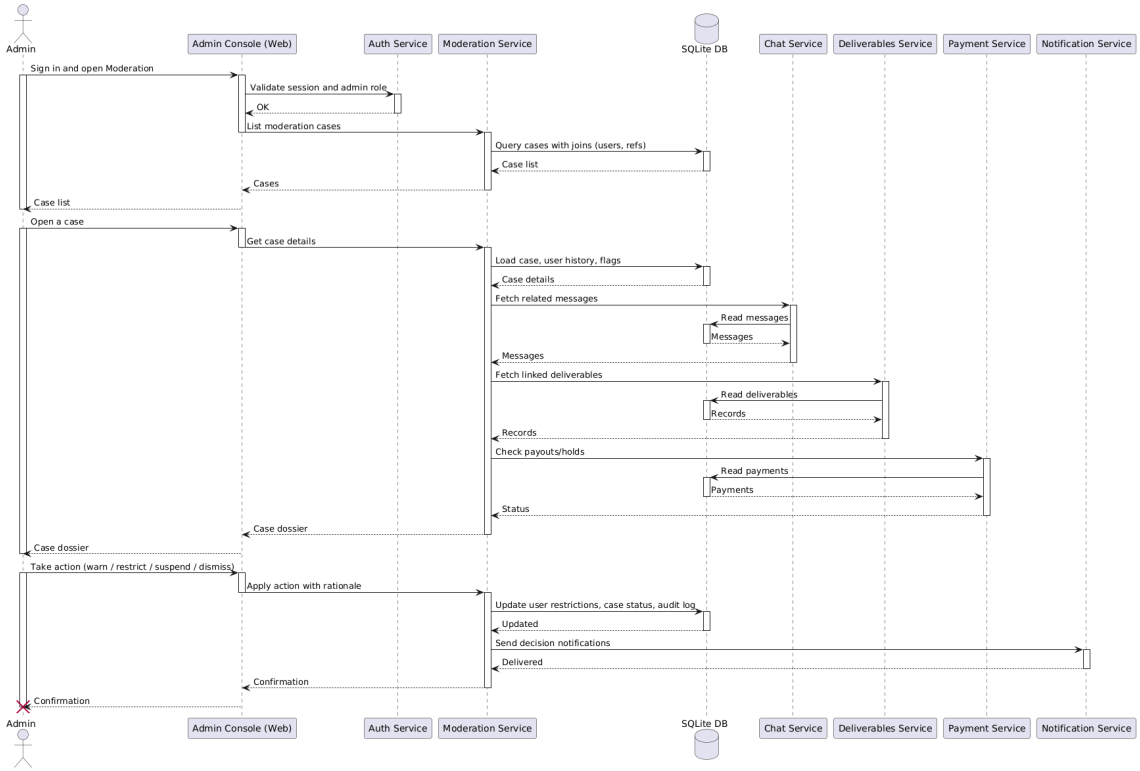


Figure 3.65: Moderate Users and Disputes

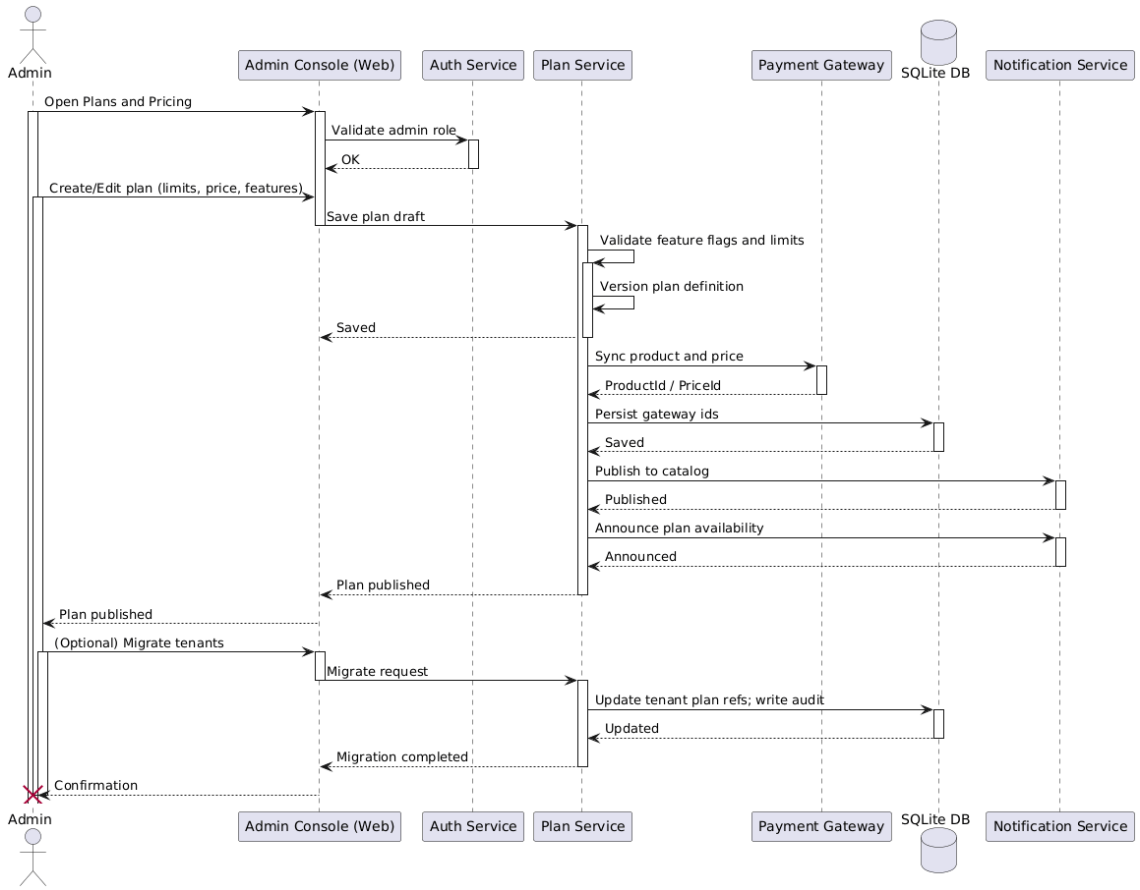


Figure 3.66: Manage Plans and Billing

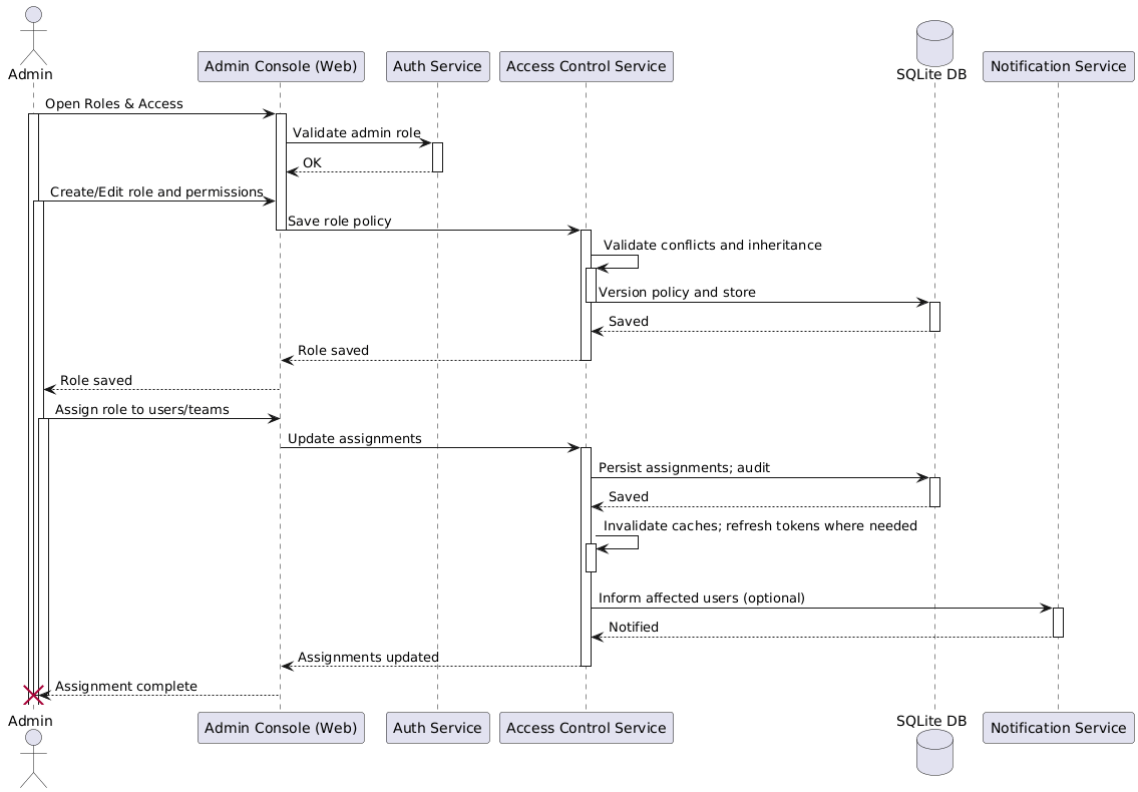


Figure 3.67: Configure Roles and Access

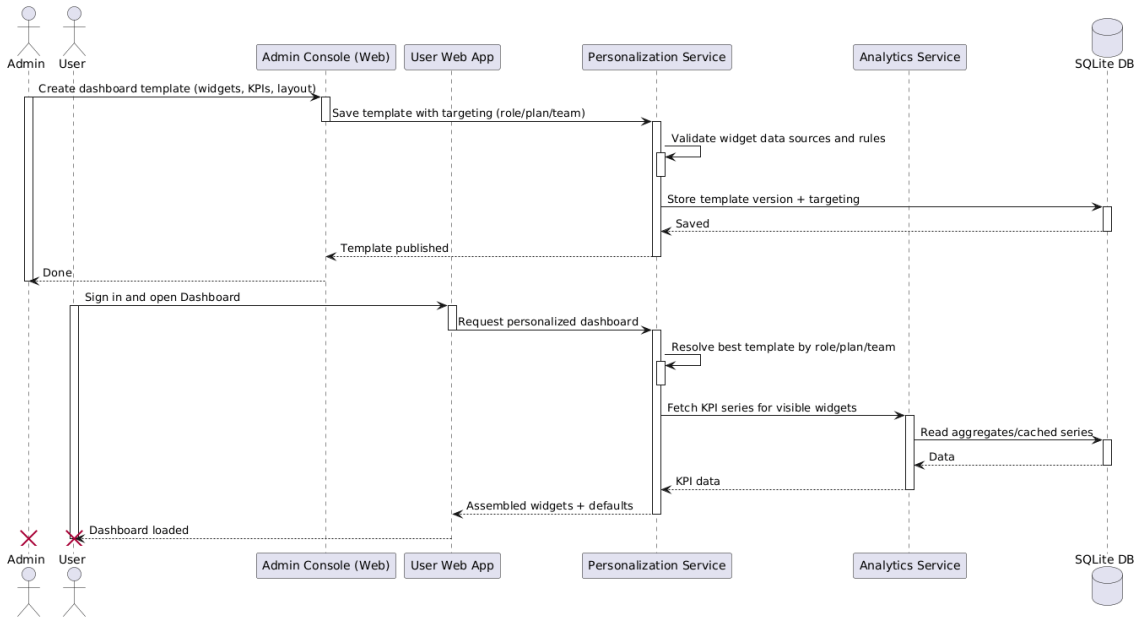
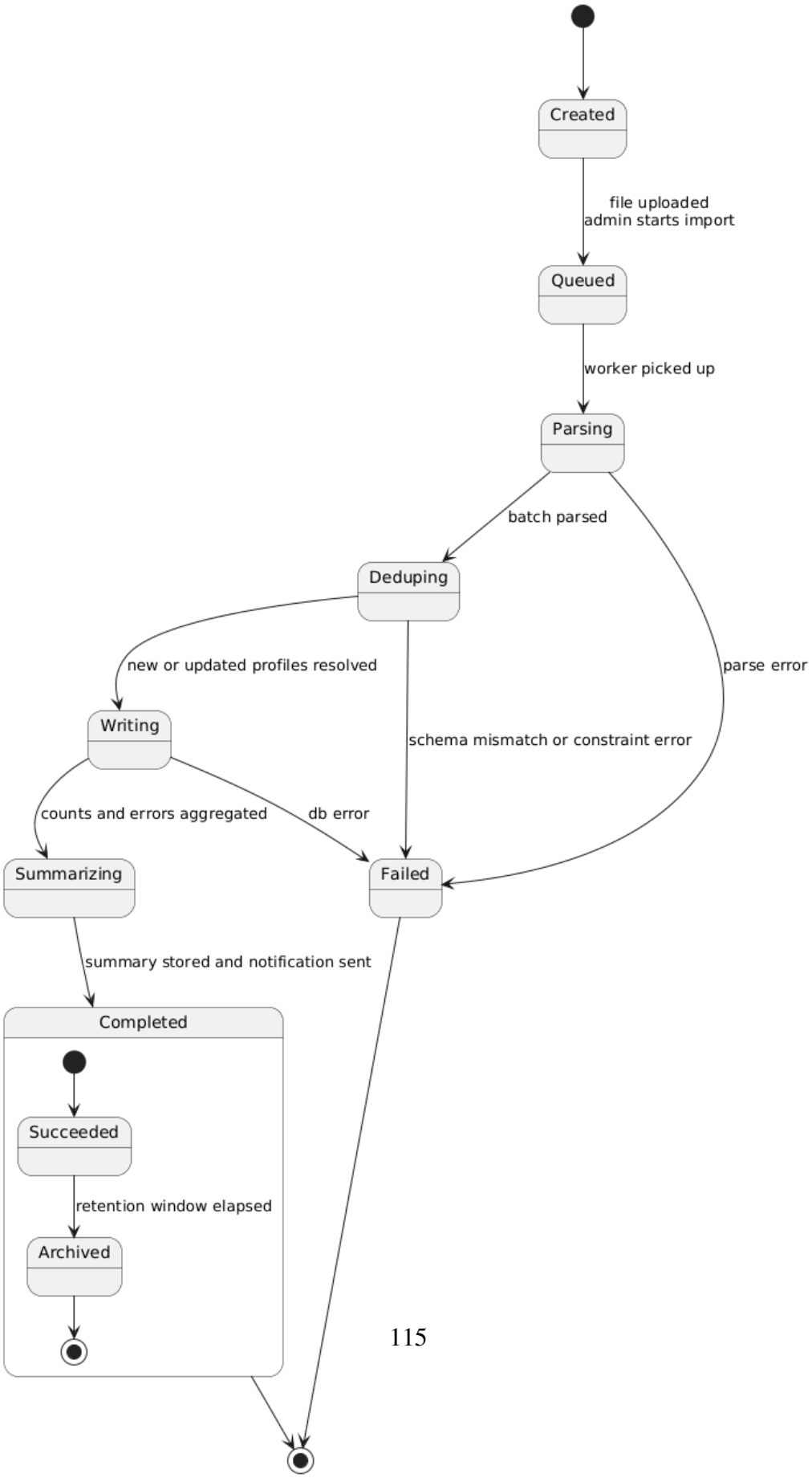


Figure 3.68: Generate Personalized Dashboards

3.2.4 State Transition Diagram

3.2.4.1 Data Gathering and Influencer On-boarding



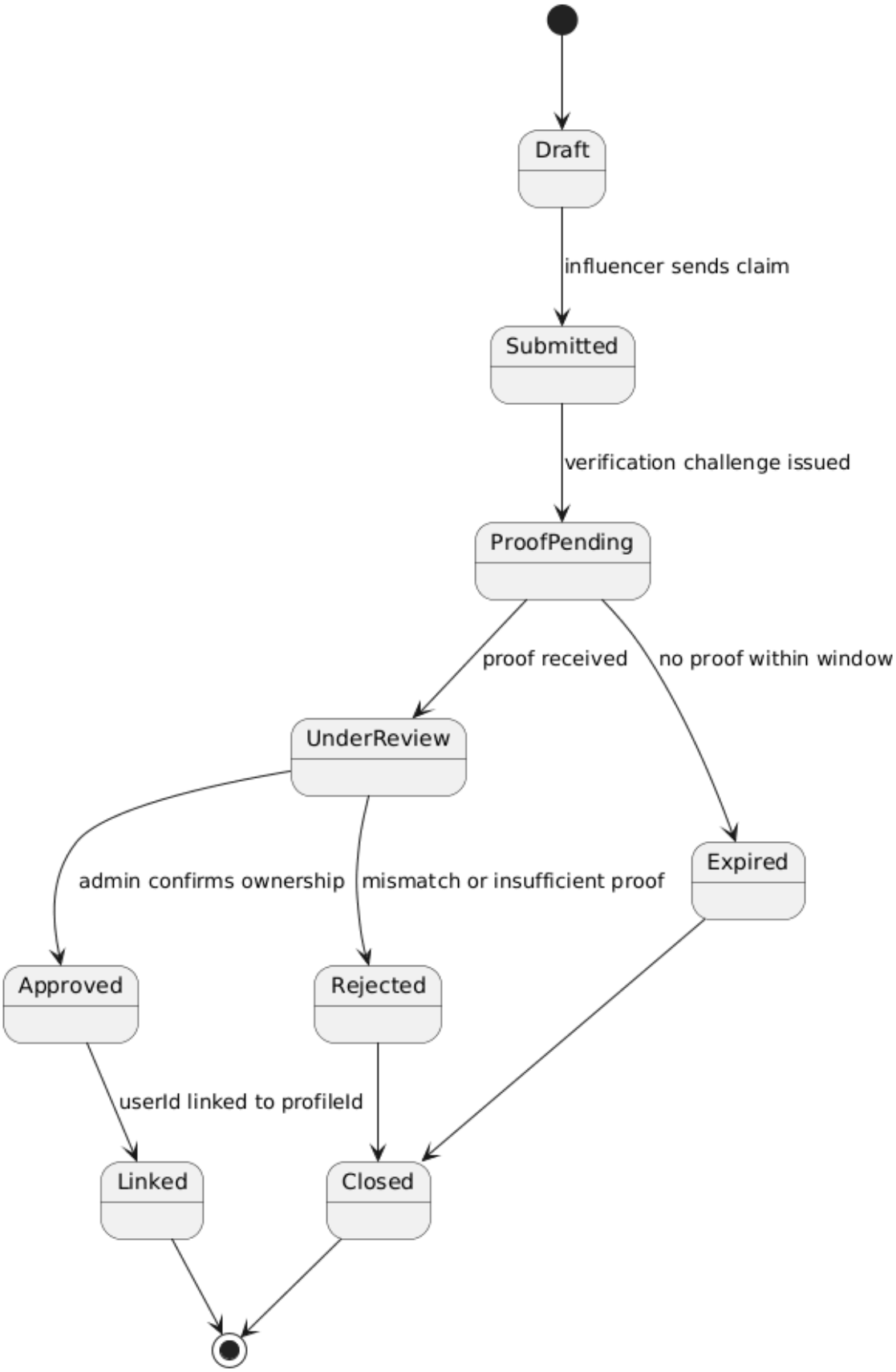


Figure 3.70: Claim Existing Profile

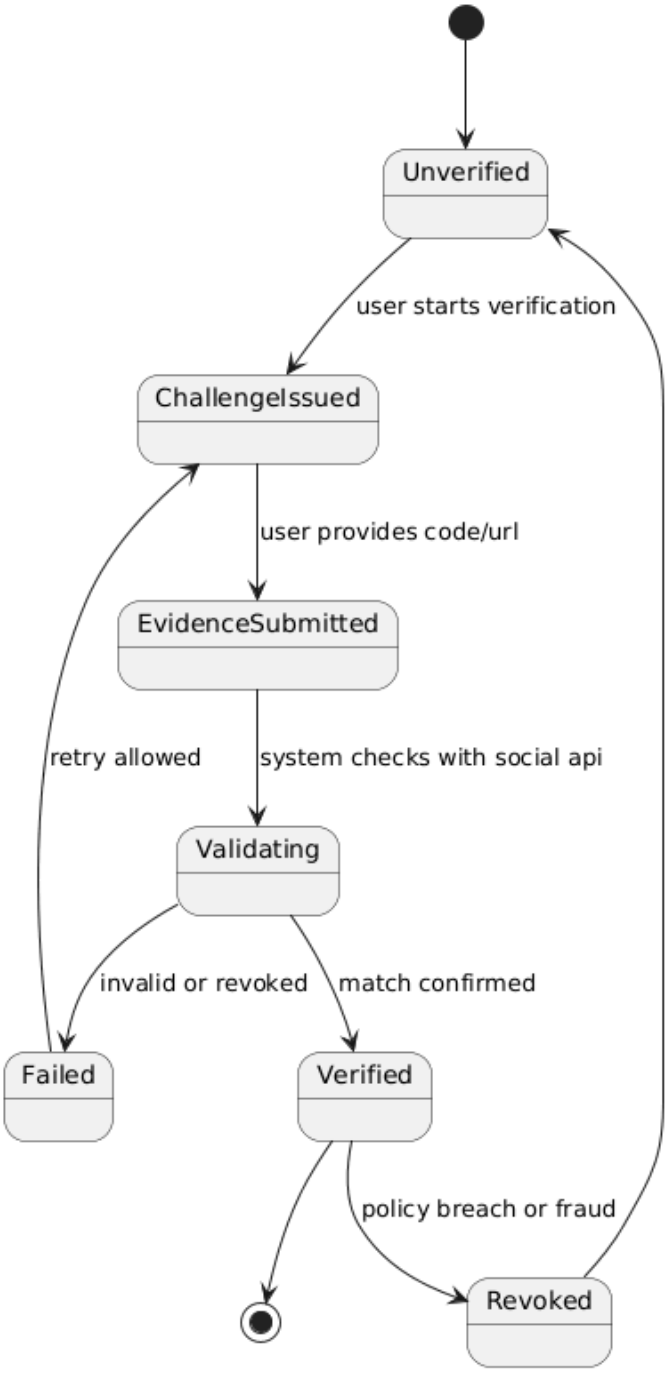


Figure 3.71: Verify User

3.2.4.2 Recommendation System

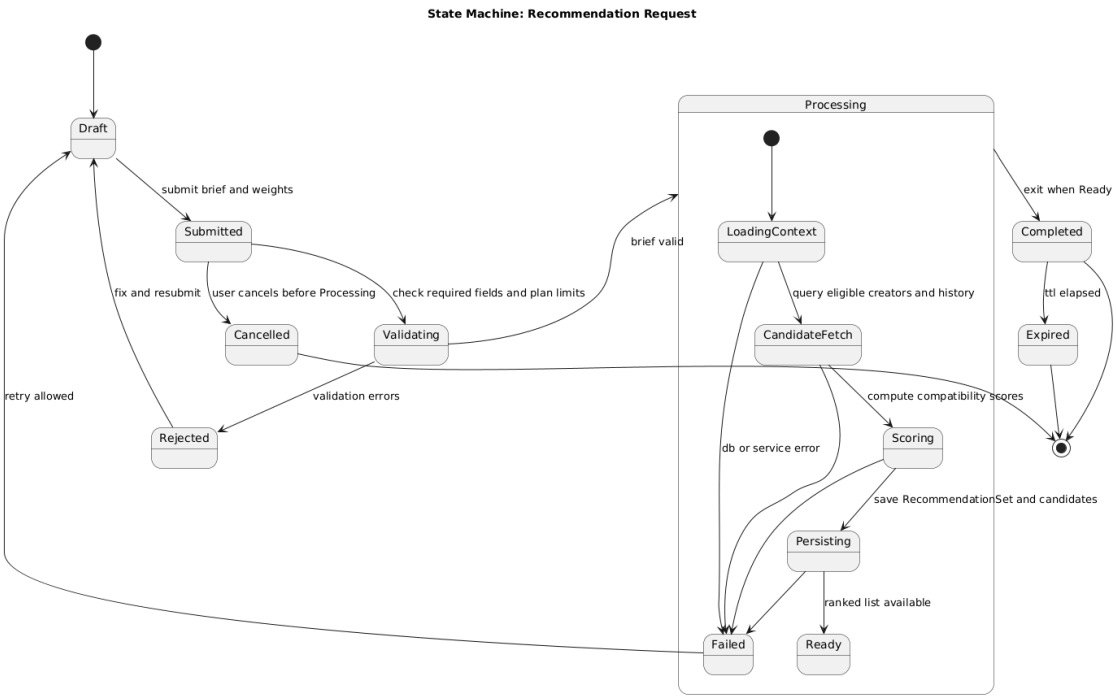


Figure 3.72: Generate Recommendations

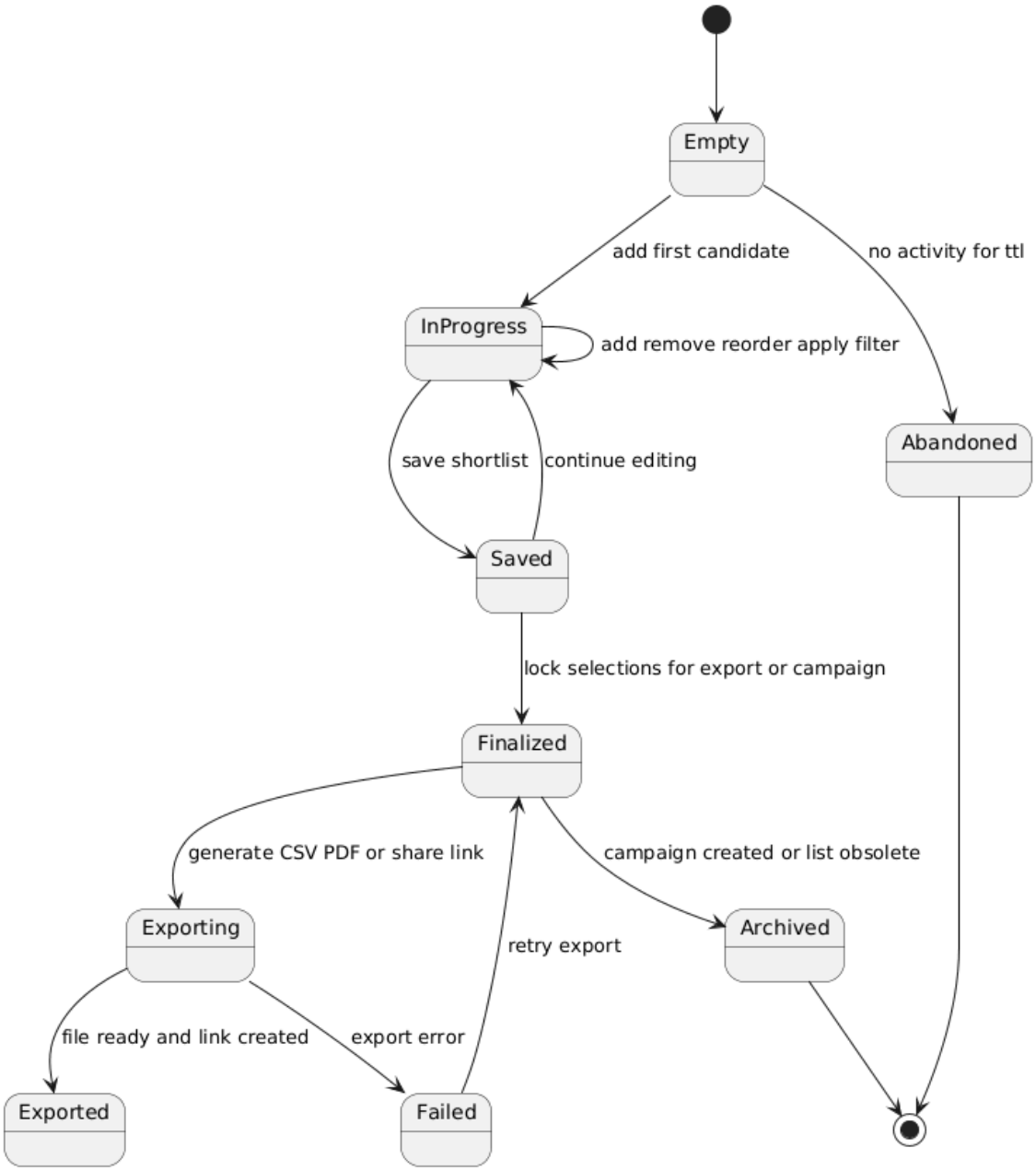


Figure 3.73: Shortlist Influencers

3.2.4.3 Ad Generation

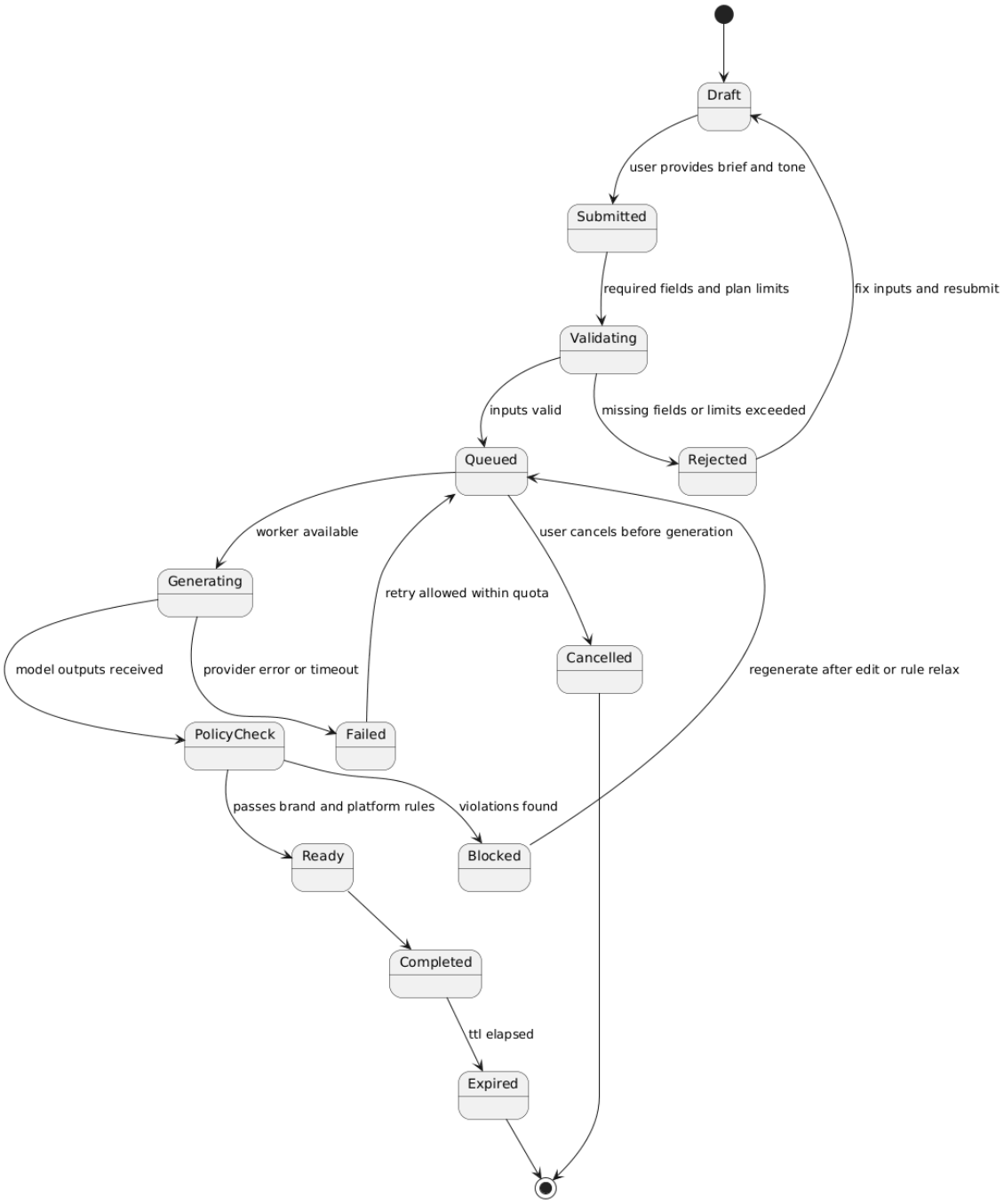


Figure 3.74: Generate Creatives

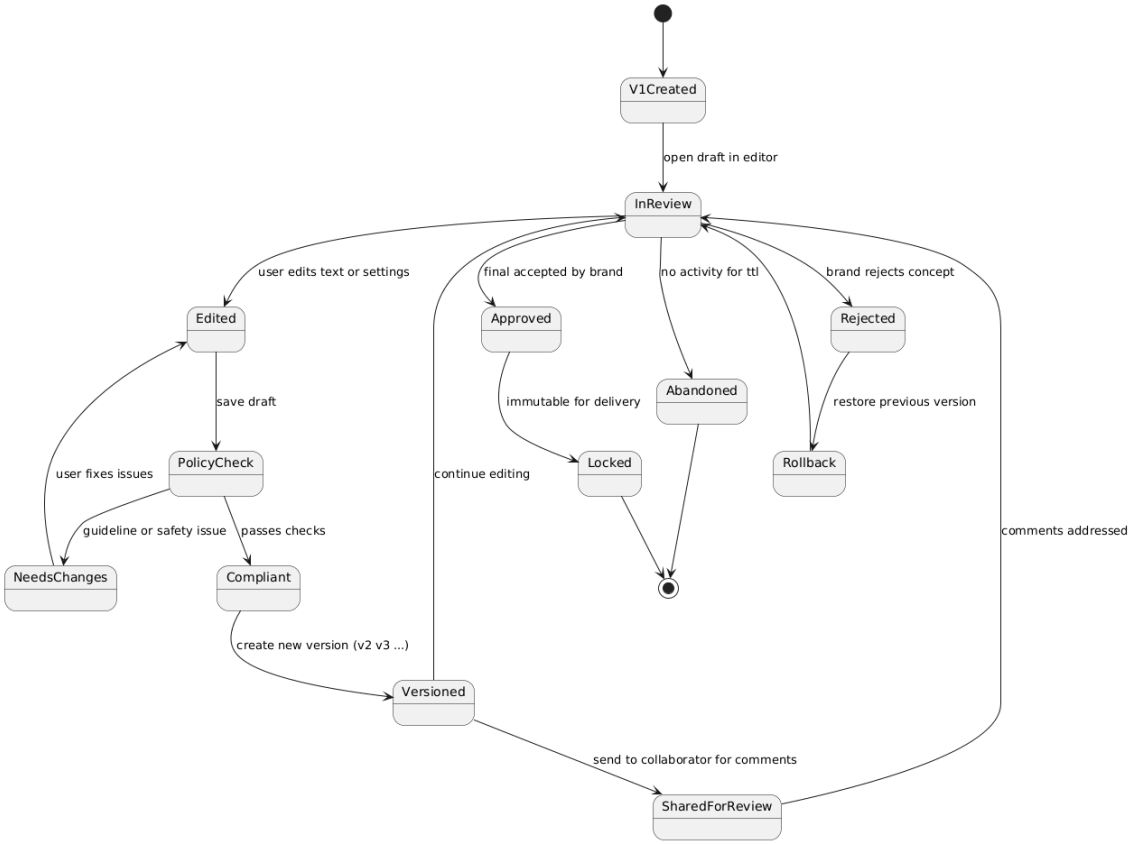


Figure 3.75: Review and Edit Creatives

3.2.4.4 Campaign Management and Chat

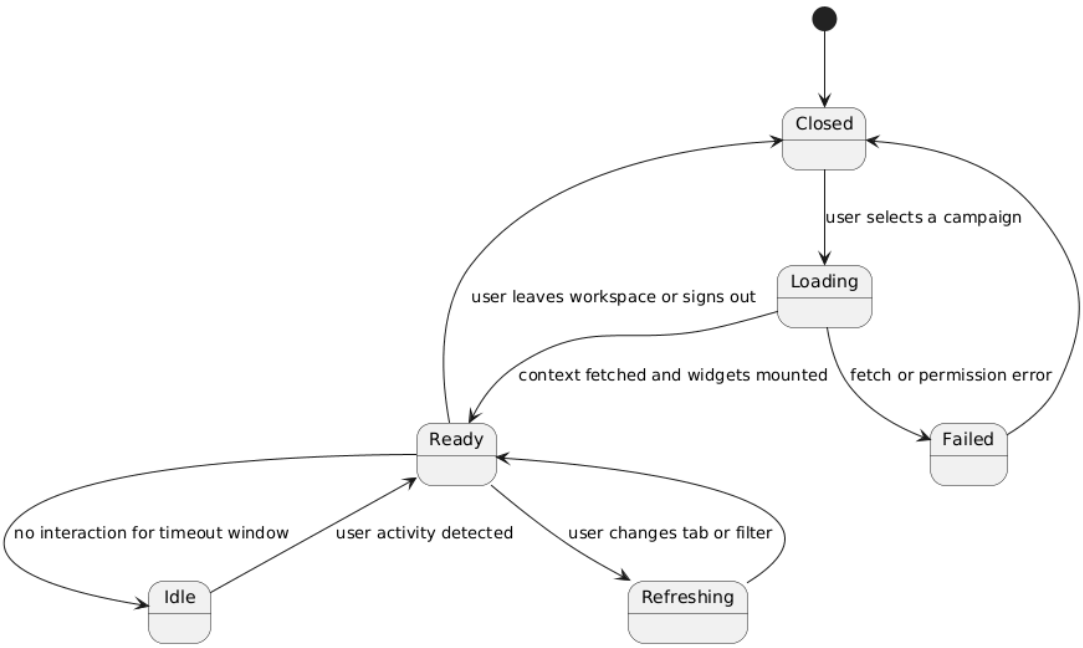


Figure 3.76: Open Campaign Workspace

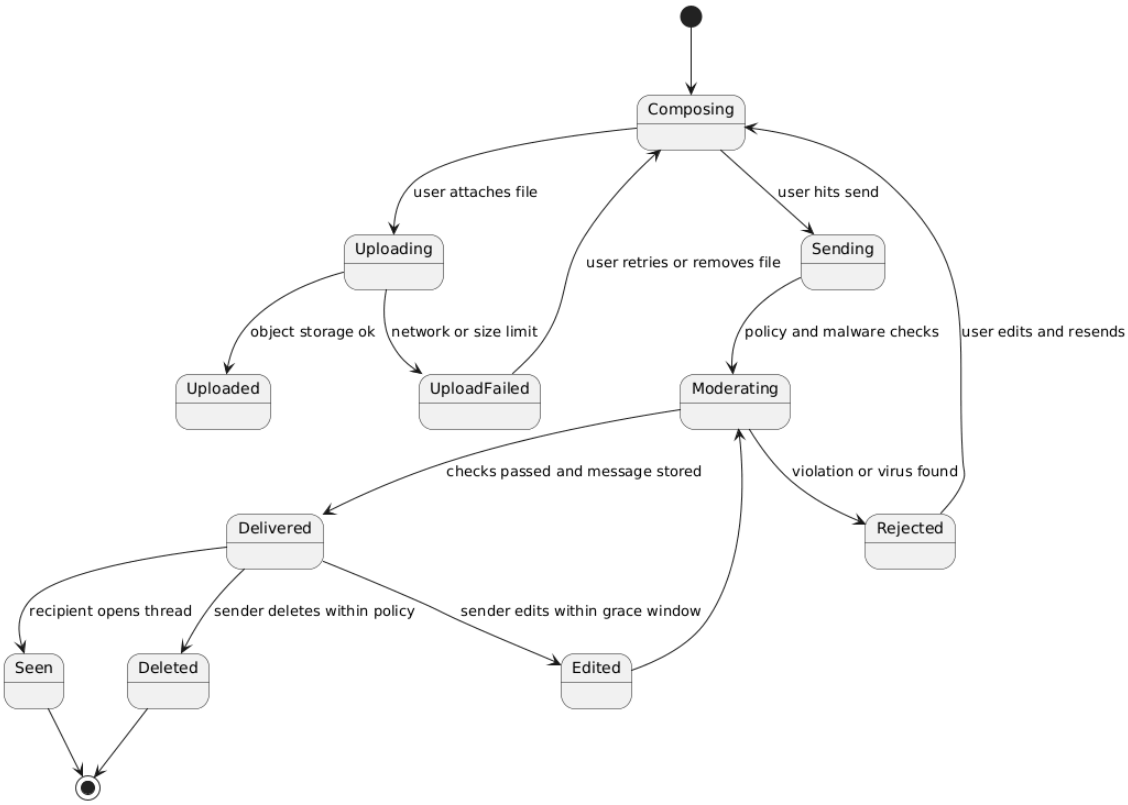


Figure 3.77: Chat and File Exchange

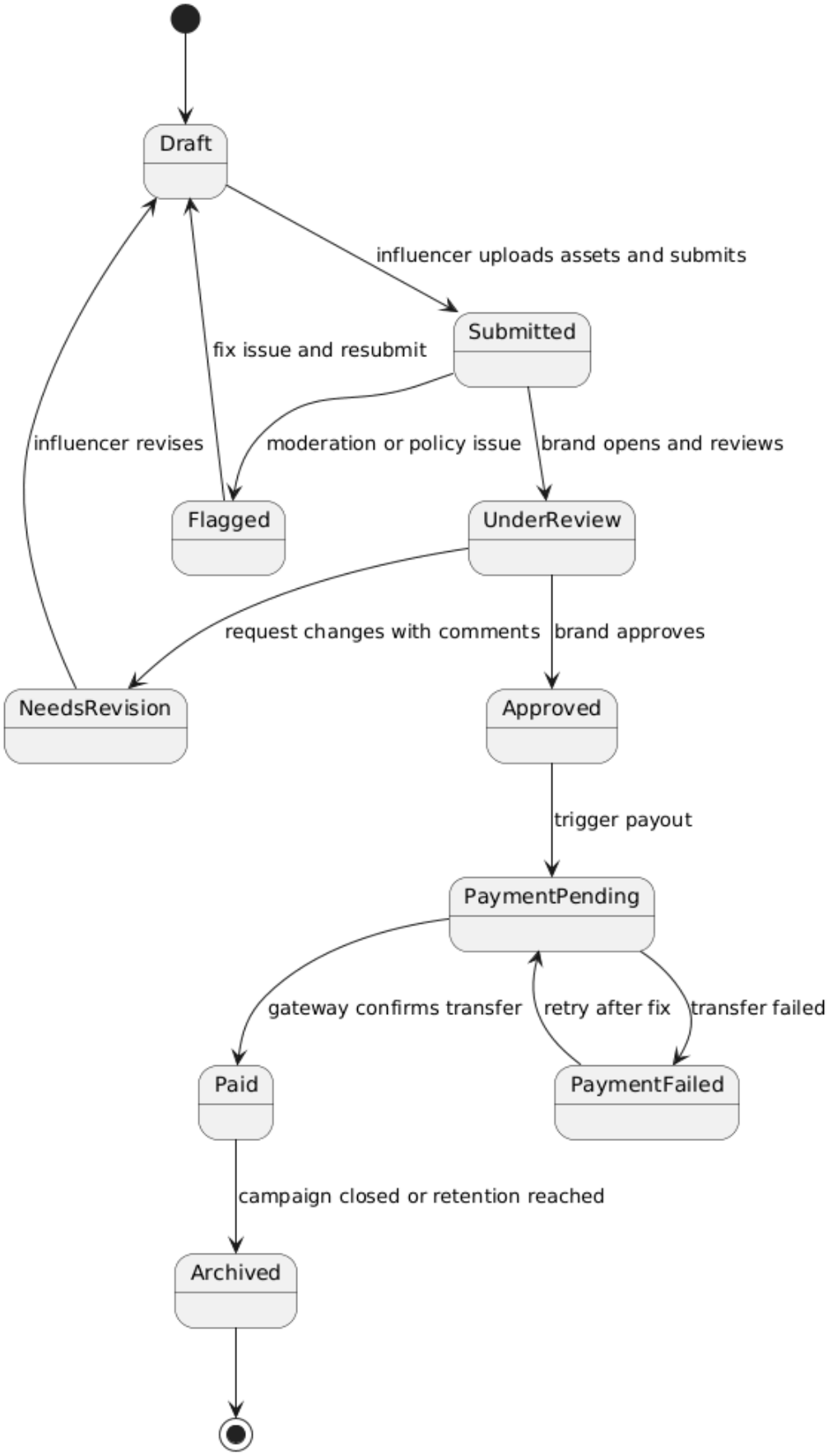


Figure 3.78: Submit and Approve Deliverables

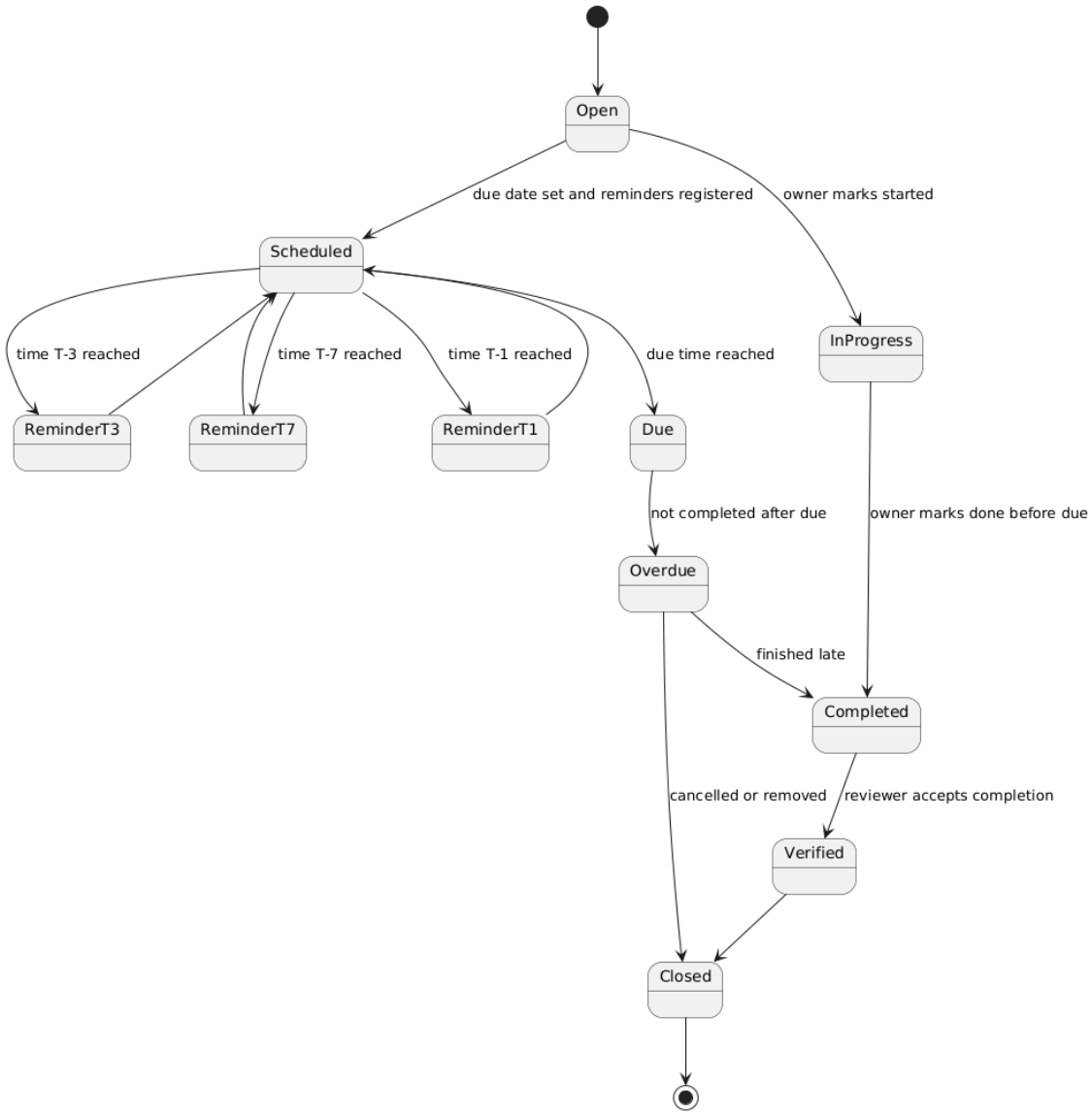


Figure 3.79: Milestone Tracking and Reminders

3.2.4.5 Analytics Dashboard

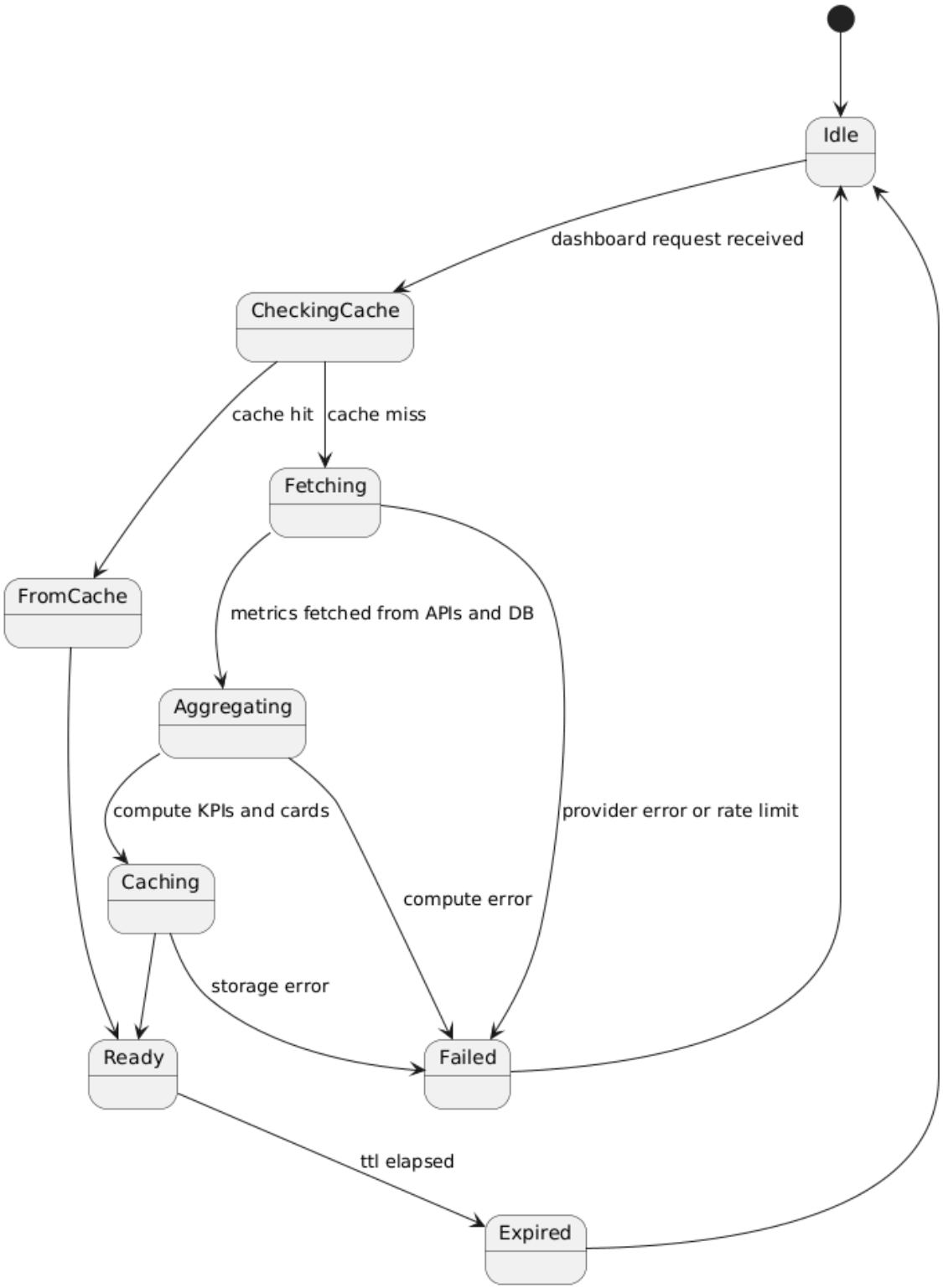


Figure 3.80: View Live Campaign Performance

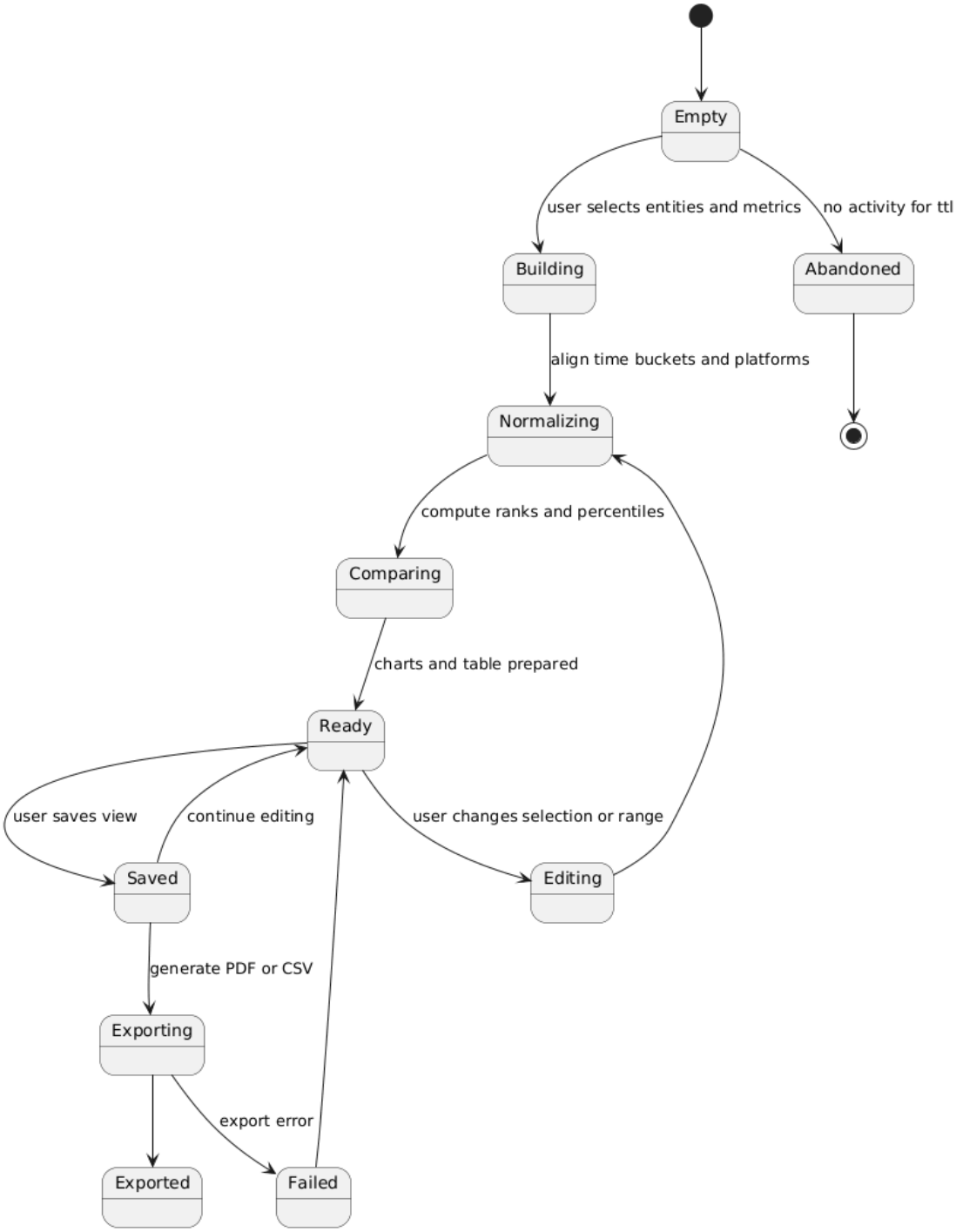


Figure 3.81: Compare Creators and Campaigns

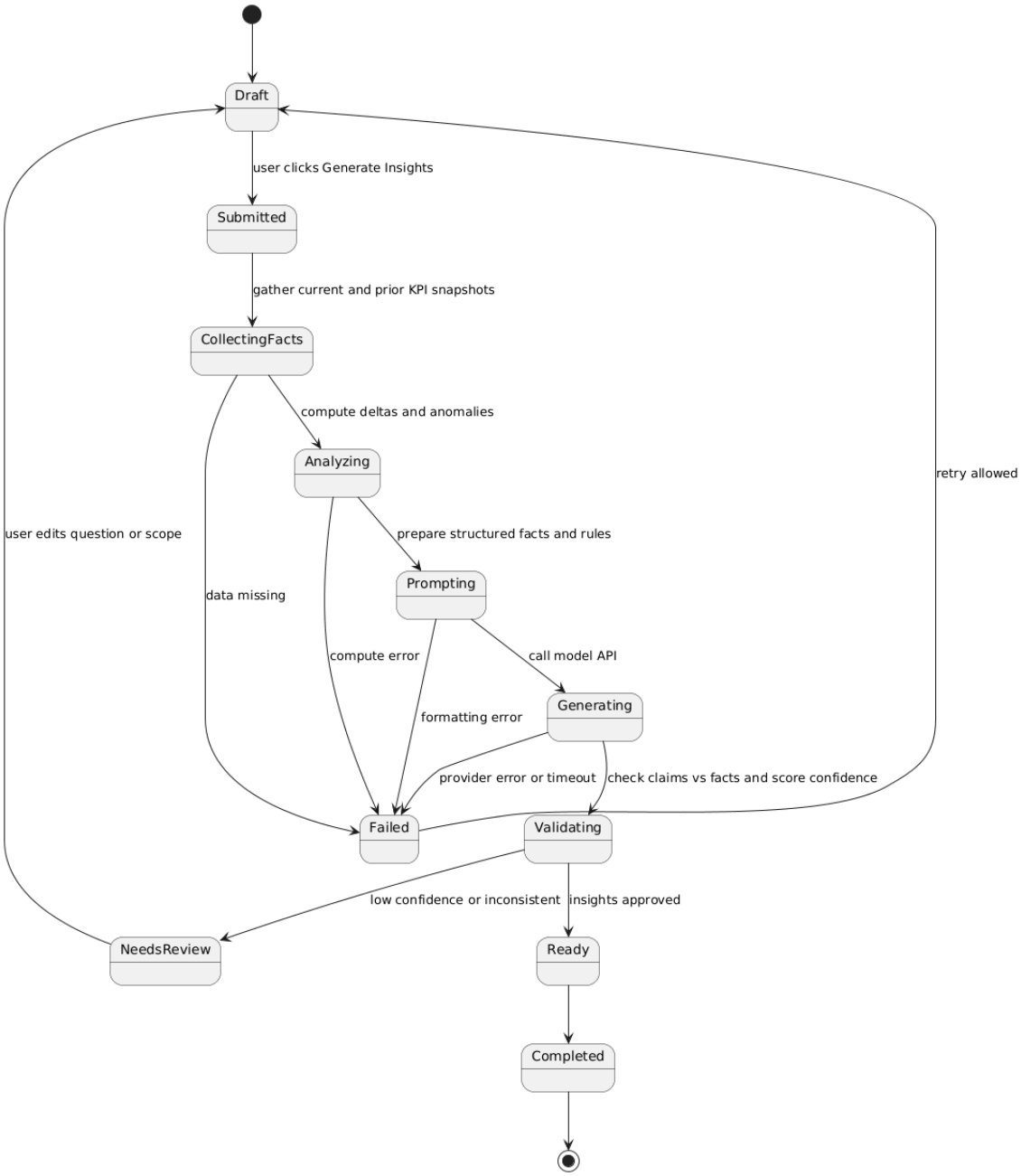


Figure 3.82: Generate Insight Prompts

3.2.4.6 Payment Gateway

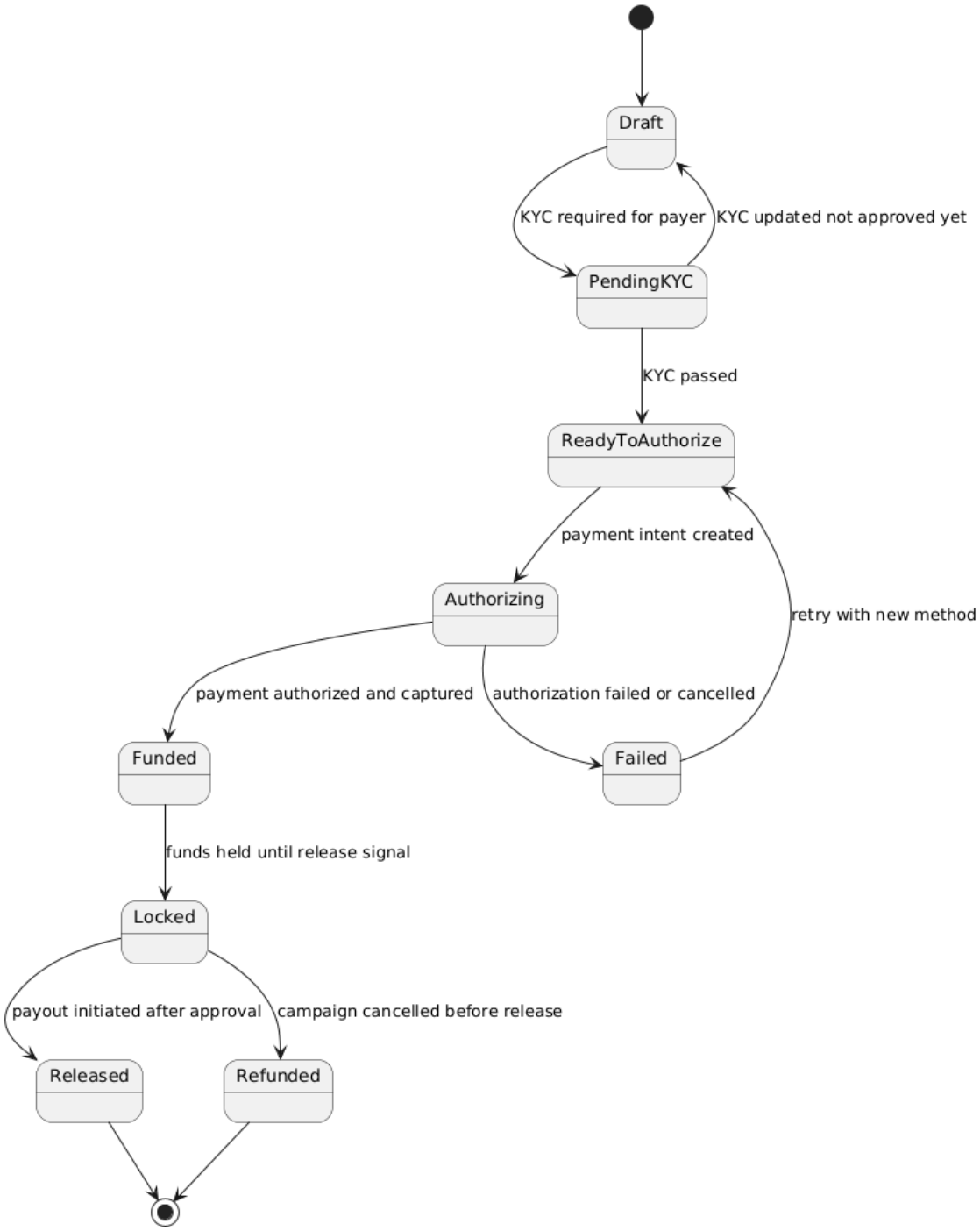


Figure 3.83: Create Escrow for Campaign

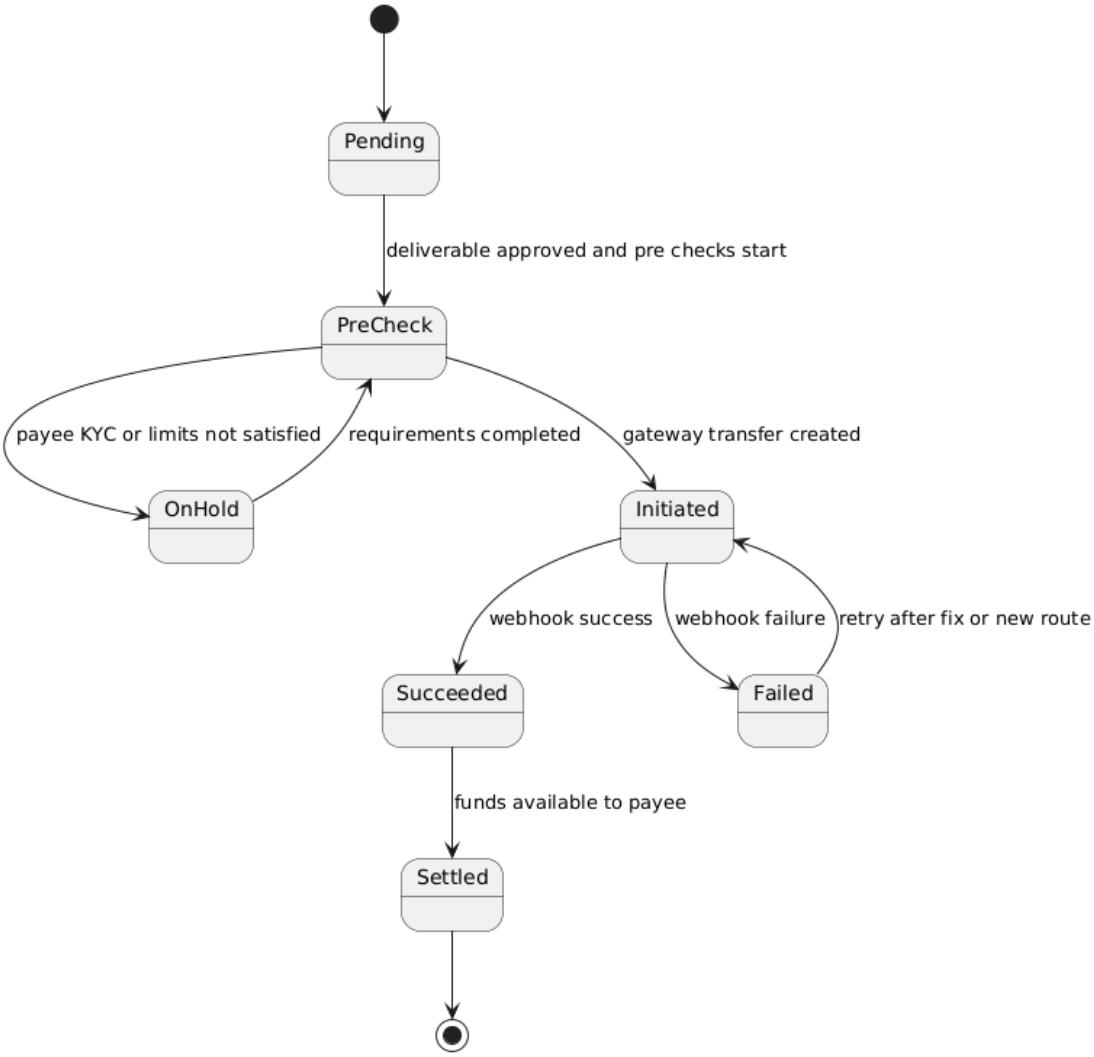


Figure 3.84: Release Payment on Approval

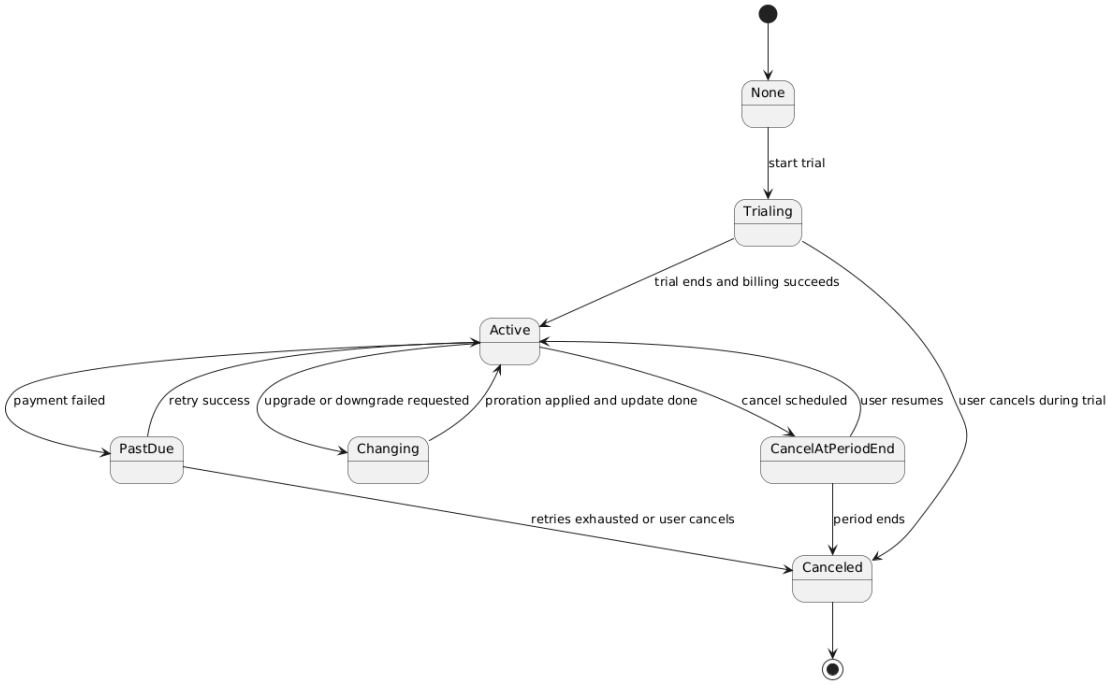


Figure 3.85: Manage Subscriptions

3.2.4.7 Conversation Intelligence Assistant

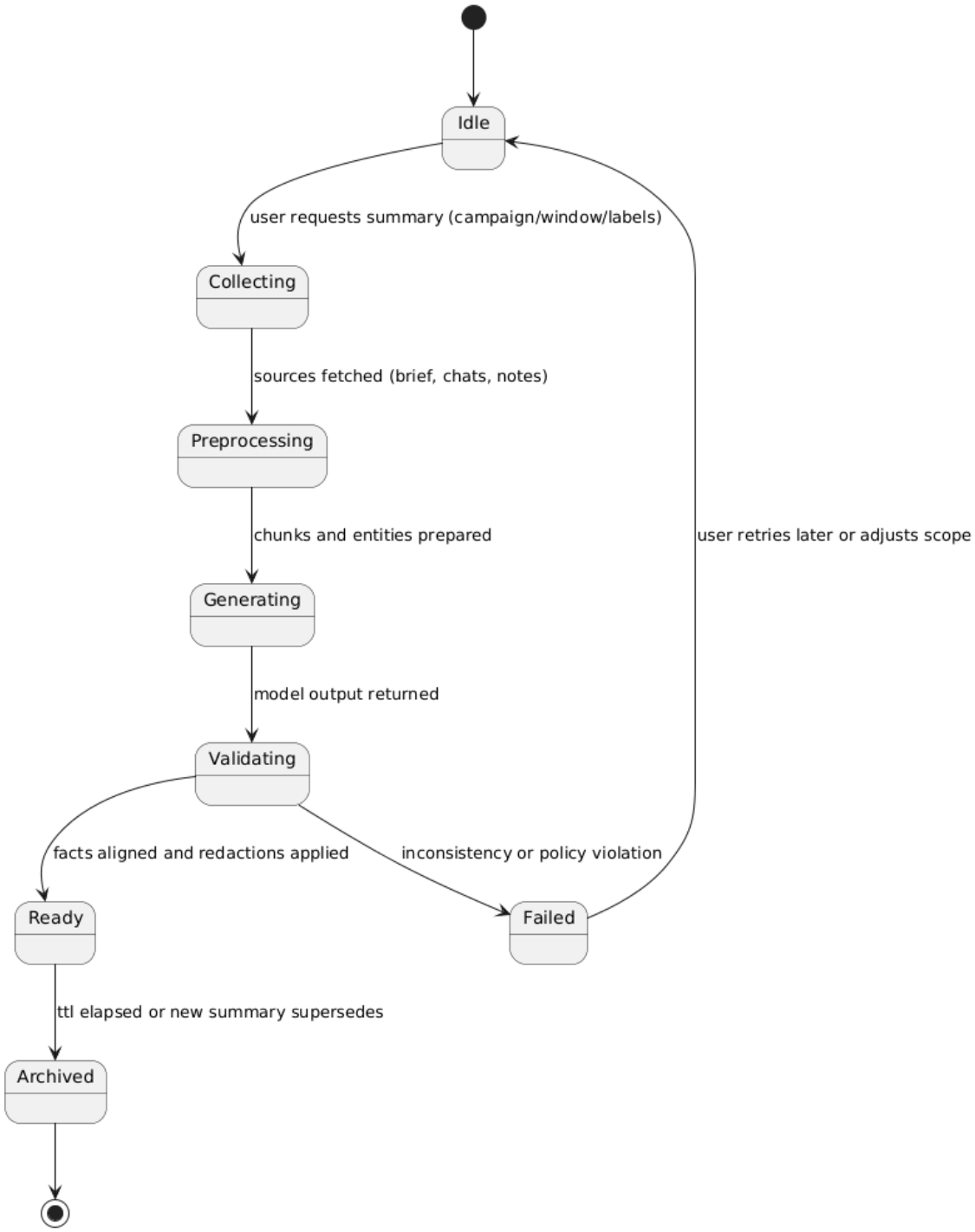


Figure 3.86: Summarize Chats and Briefs

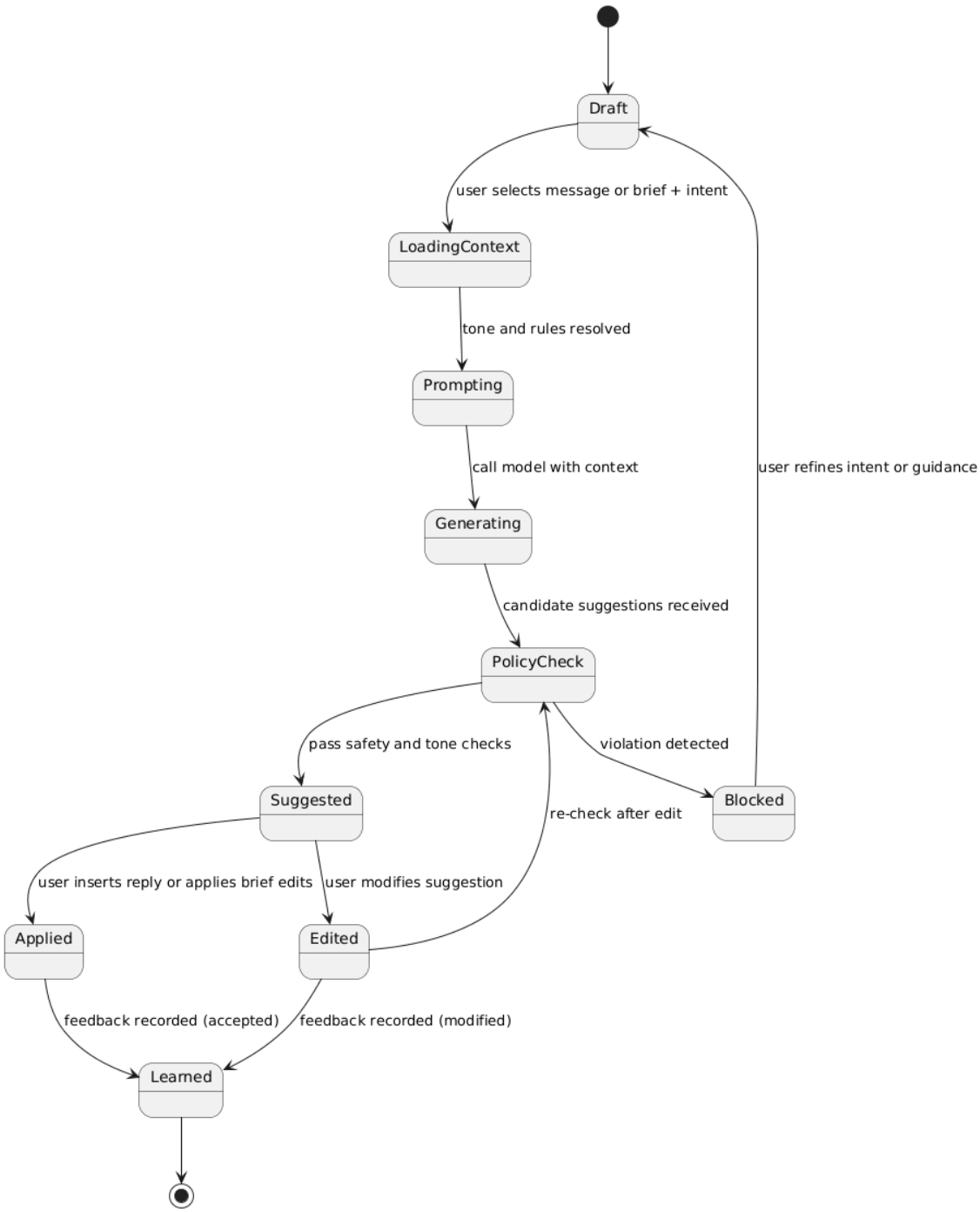


Figure 3.87: Suggest Replies and Brief Improvements

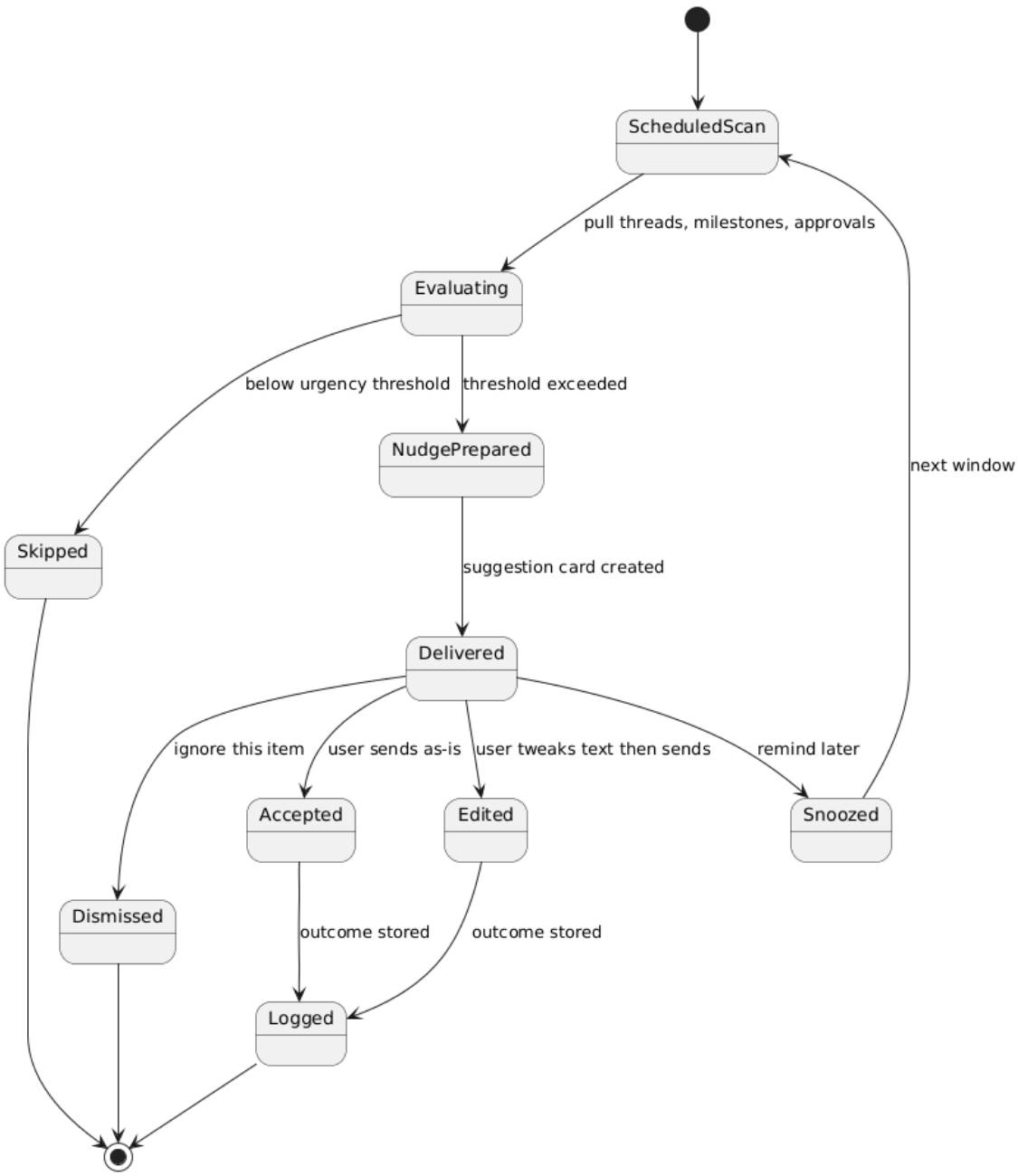


Figure 3.88: Follow Up Nudges

3.2.4.8 Notification and Smart Alerts

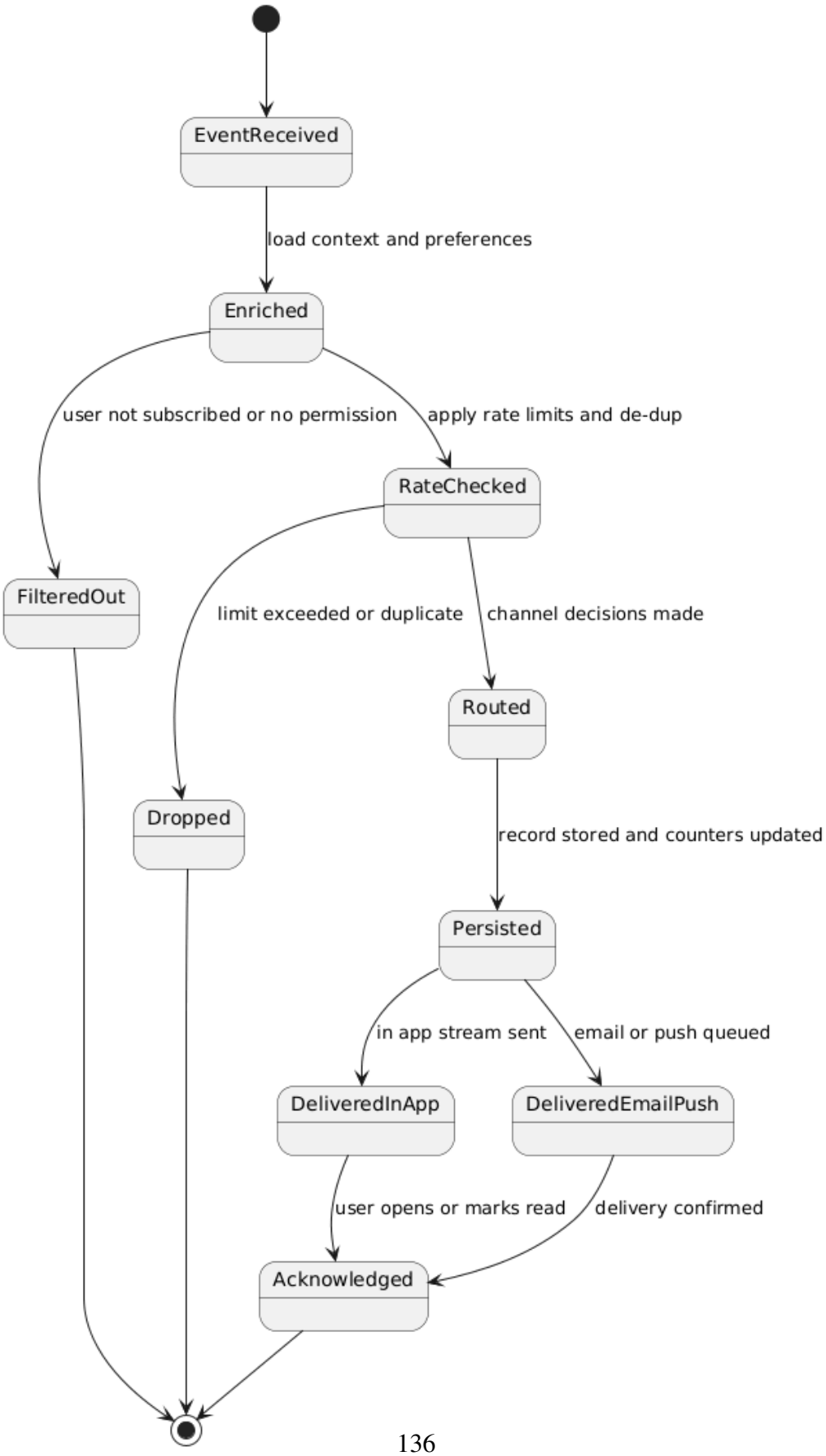


Figure 3.89: Real Time Campaign Alerts

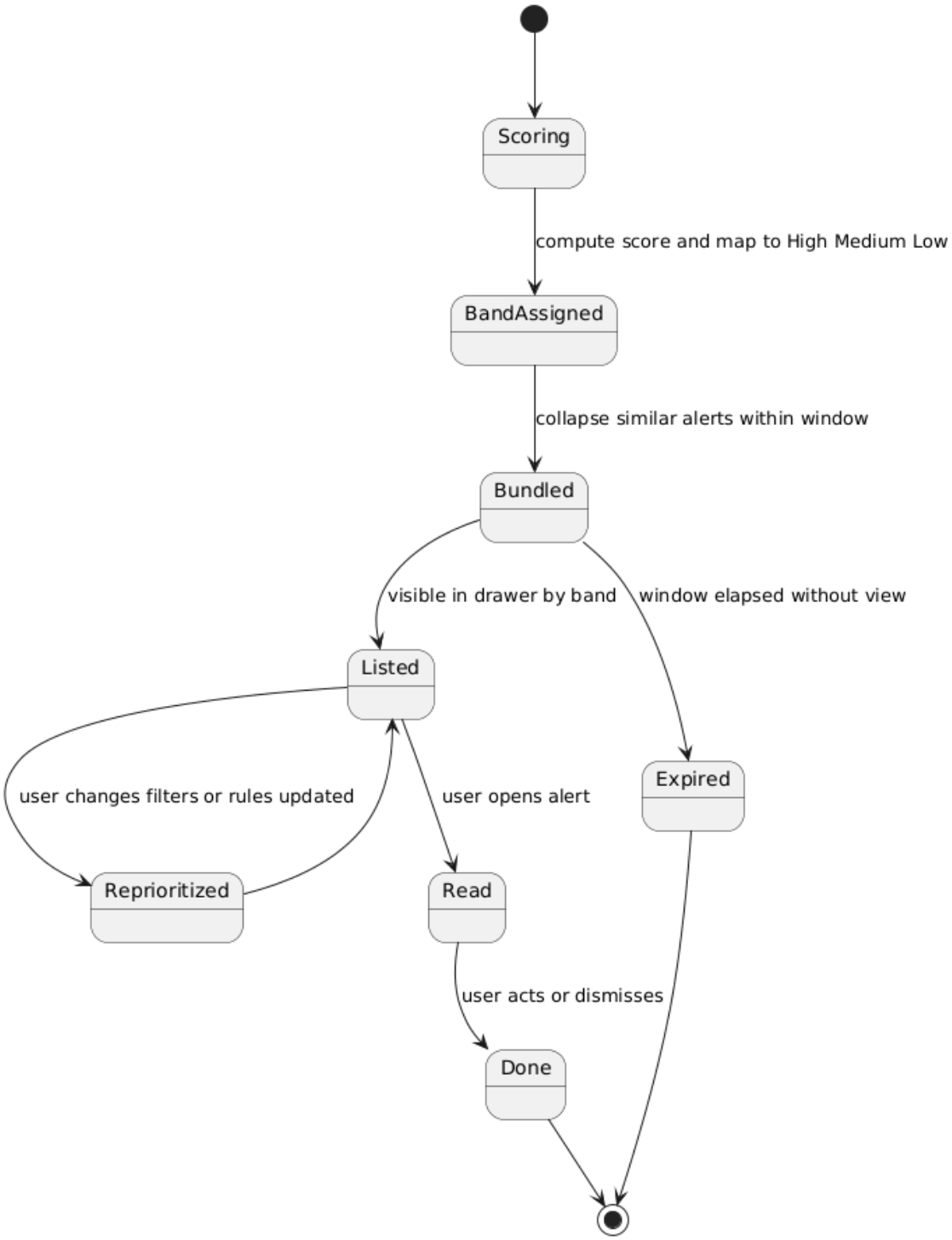


Figure 3.90: Priority Sorting

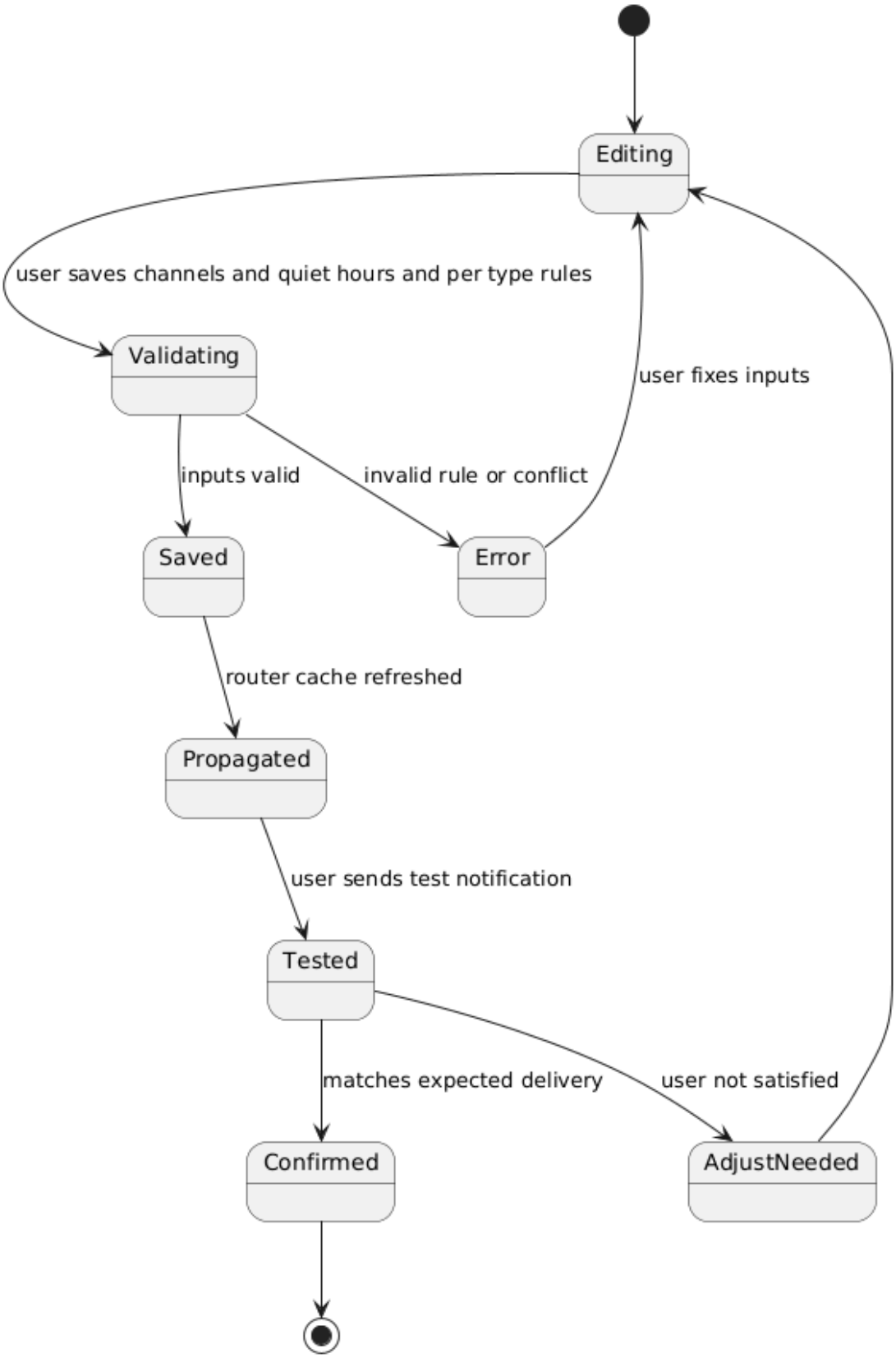


Figure 3.91: Preference Management

3.2.4.9 Admin, User Profile and Personalization

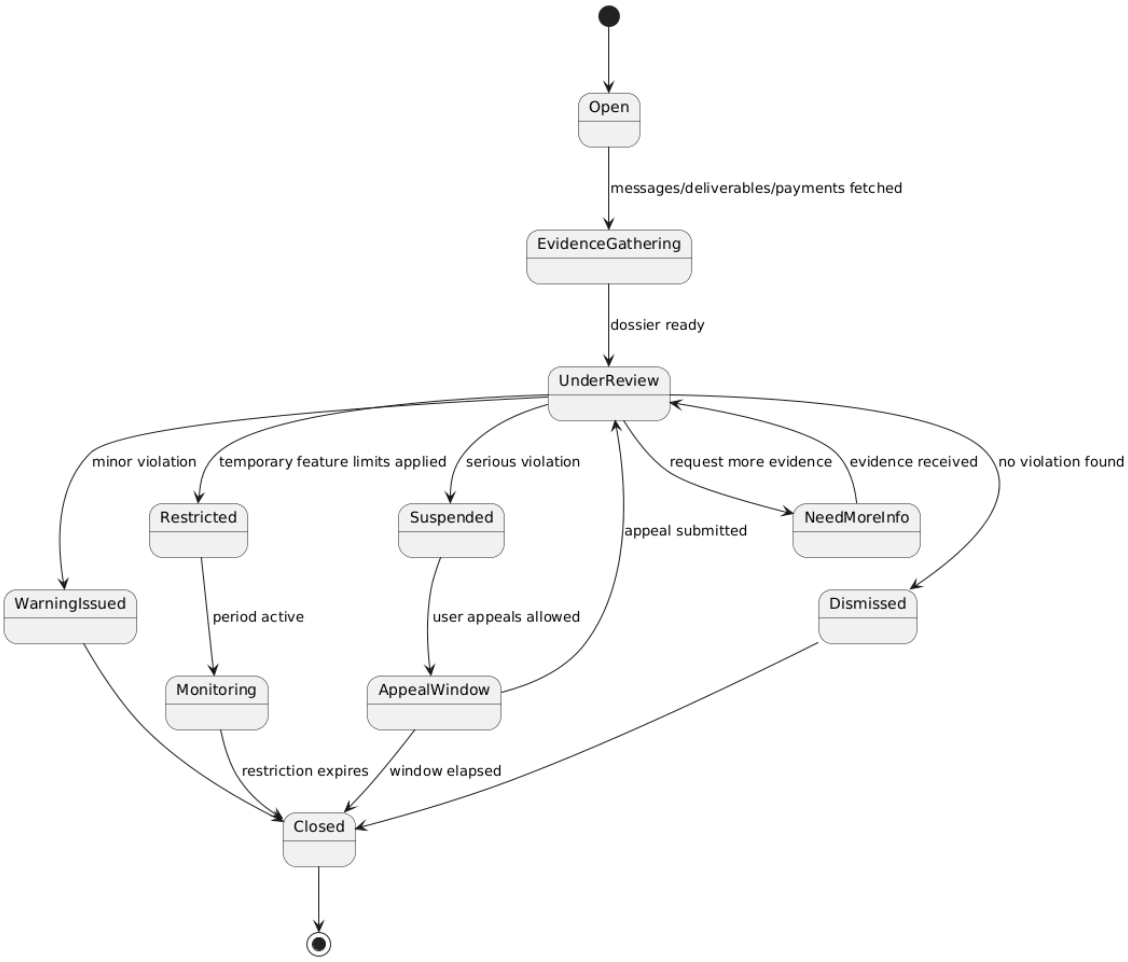


Figure 3.92: Moderate Users and Disputes

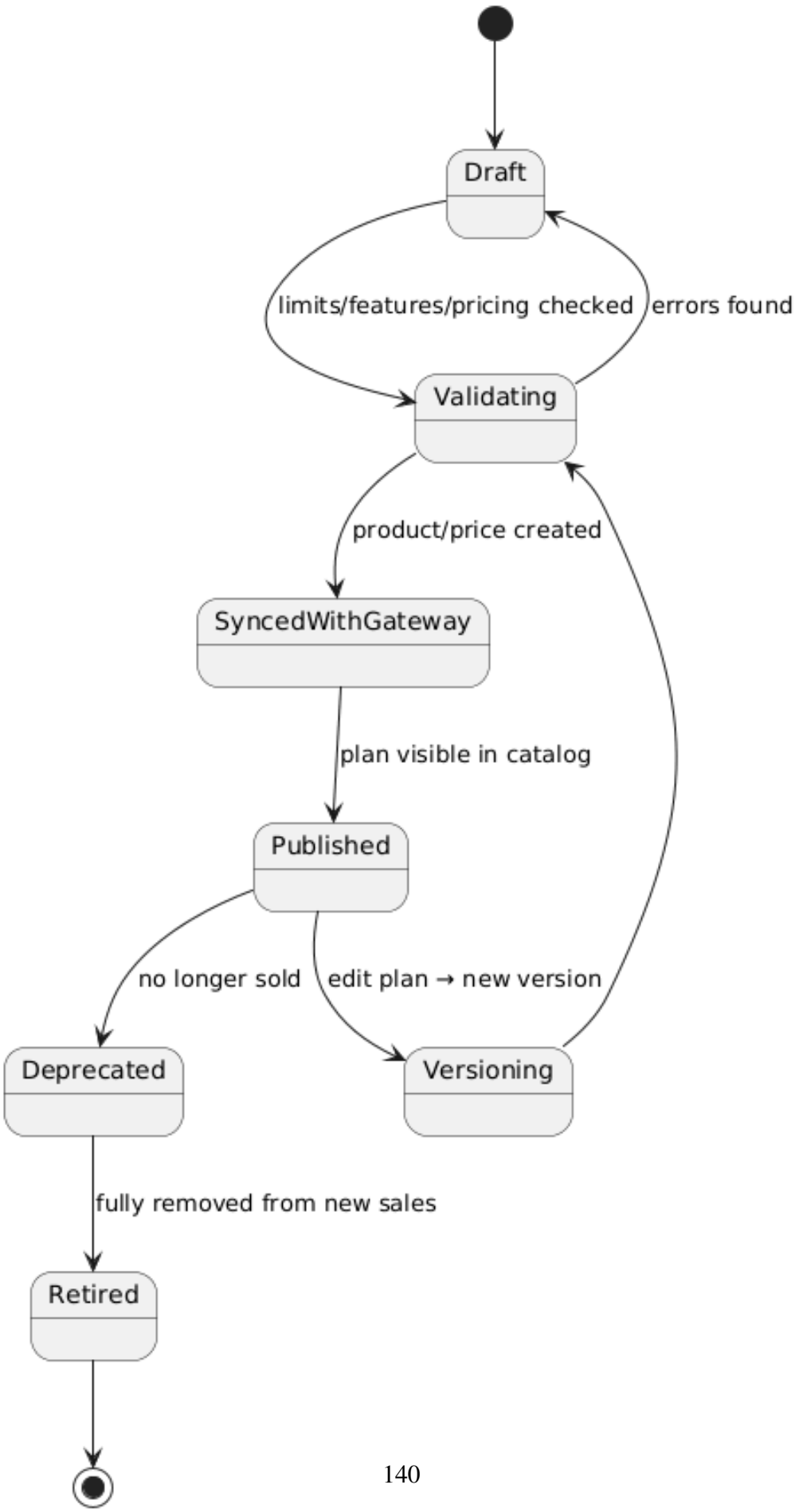


Figure 3.93: Manage Plans and Billing

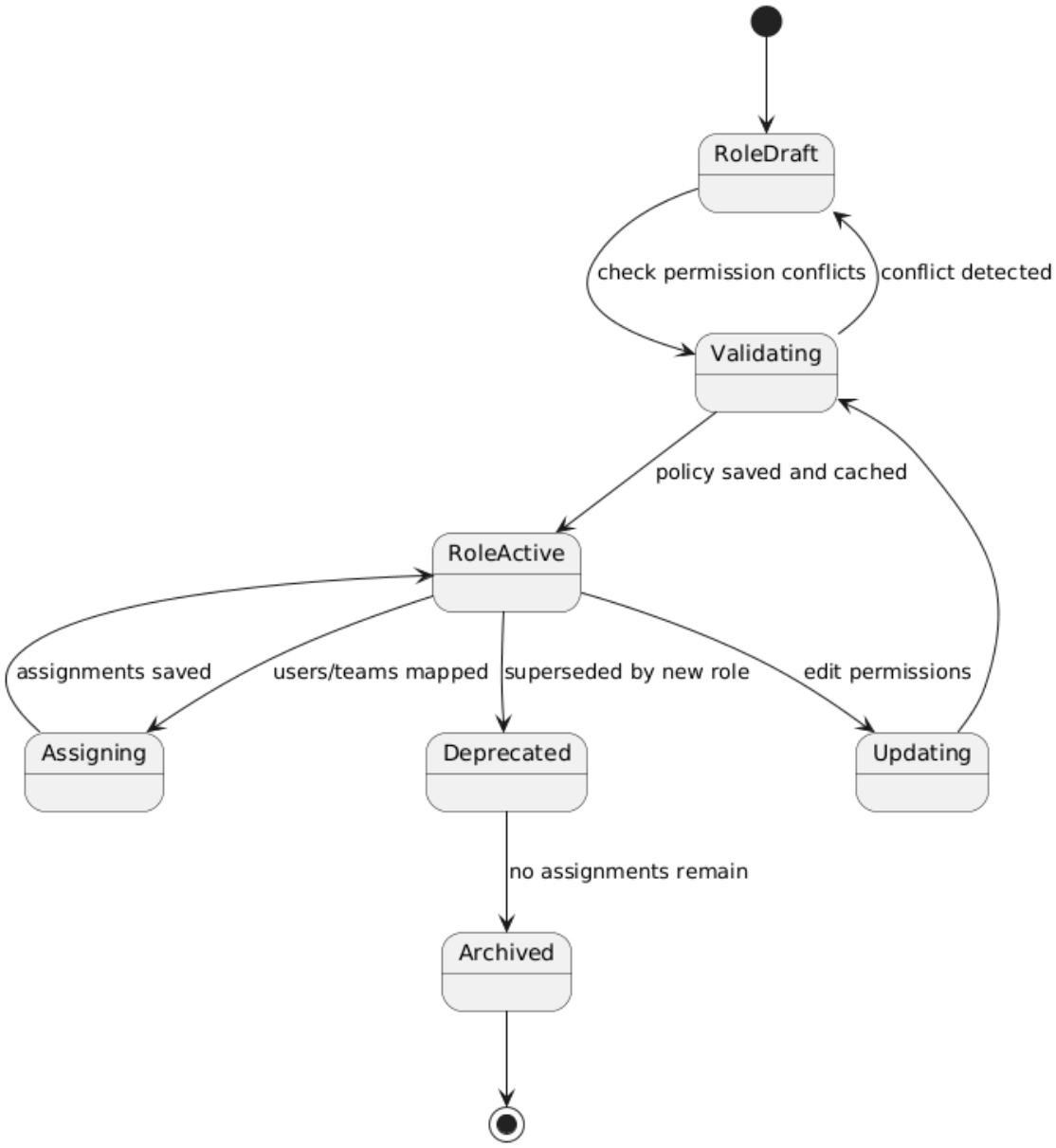


Figure 3.94: Configure Roles and Access

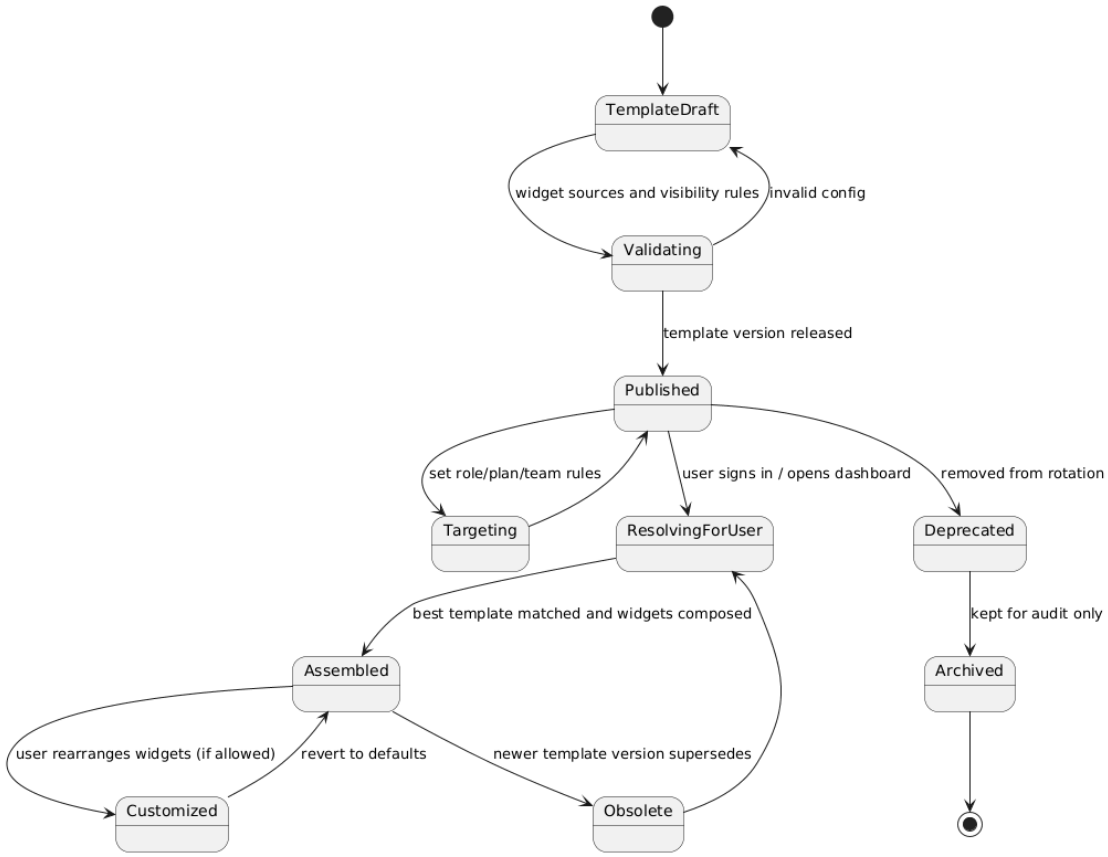


Figure 3.95: Generate Personalized Dashboards

3.3 Data Design

The data design of the Splash Platform tells how information within the system is stored, and accessed across different modules. Since the system integrates features such as influencer on-boarding, campaign management, AI-based generation, analytics, payments, and alerts, the database schema is structured to maintain consistency, referential integrity, and modular separation while also optimizing for scalability and query efficiency. The project uses Drizzle ORM together with SQLite for lightweight relational modeling in local and early production environments, with a clear migration path to PostgreSQL when scaling.

3.3.1 Schema and Entity Design

The schema follows a modular domain-driven structure, where each core entity corresponds to a business domain:

- **User and Account Tables:** Manage authentication, authorization, and user profiles. Support role-based access and verification.
- **Profiles Table:** Stores influencer or brand data, social links, engagement metrics, and trust scores.
- **Campaign Table:** Represents collaborations between brands and influencers. Includes milestones, deliverables, and statuses.
- **Messages and Conversations Tables:** Store real-time chat data and attachments between campaign participants.
- **CreativeDraft and Versions Tables:** Capture AI-generated creatives, edits, and compliance metadata.
- **Recommendations and Shortlists Tables:** Contain ranked influencer lists generated by the recommendation engine.
- **Payments, Escrows, and Subscriptions Tables:** Manage financial transactions, subscription plans, and payment lifecycle tracking.
- **Analytics Tables:** Record performance metrics, engagement data, and AI-generated insights.
- **Notifications and Preferences Tables:** Handle alert delivery and user-defined notification rules.
- **Admin Tables:** Include moderation cases, plans, and RBAC configurations for roles and permissions.

3.3.2 Data Storage and Indexing Strategy

The data storage layer is optimized for high-frequency reads and moderate writes, with strategies including:

- Clustered Indexing on unique identifiers for quick lookups.
- Composite Indexes for frequent joins, such as (campaign_id, influencer_id) and (user_id, role).
- Partial Indexes for filtered queries, e.g., active campaigns or unread notifications.
- Caching Layer (in-memory) for KPI aggregates and leaderboard data to avoid recomputation.
- Full-Text Search Indexing on influencer bios, brand briefs, and chat messages for faster discovery.

3.3.3 Backup and Recovery Plan

Given the modular schema, backups are automated through versioned dumps of the SQLite database. Future deployment of PostgreSQL will perform:

- Nightly incremental backups with version tagging.
- Weekly full snapshots stored on secure cloud storage.
- Transactional consistency checks before every backup job.
- Automated recovery verification on a staging instance.

3.3.4 Access Control and Security Model

Access to data is enforced through role-based access control and row-level policies, mirroring Supabase-like logic:

- Brands can view their own campaigns, payments, and analytics.
- Influencers can only access their verified profiles, assigned campaigns, and chat data.
- Admins have global read/write privileges with auditing.
- AI and Background Services use dedicated service roles with scoped API keys.

All sensitive data, including API tokens and payment credentials, are AES-256 encrypted at rest and TLS 1.3 encrypted in transit. Audit logs are immutable and retained for at least one year.

3.3.5 Scalability and Migration Plan

As the dataset grows, the following enhancements are planned:

- Transition from SQLite to PostgreSQL or Neon Serverless.
- Introduce read replicas for analytics queries.
- Use object storage (S3 compatible) for heavy assets such as creatives, reports, and chat attachments.
- Introduce Redis caching for leaderboard and AI-insight endpoints.
- Partition large tables (e.g., analytics_metrics, messages) by campaign ID.

3.4 Deployment Diagram

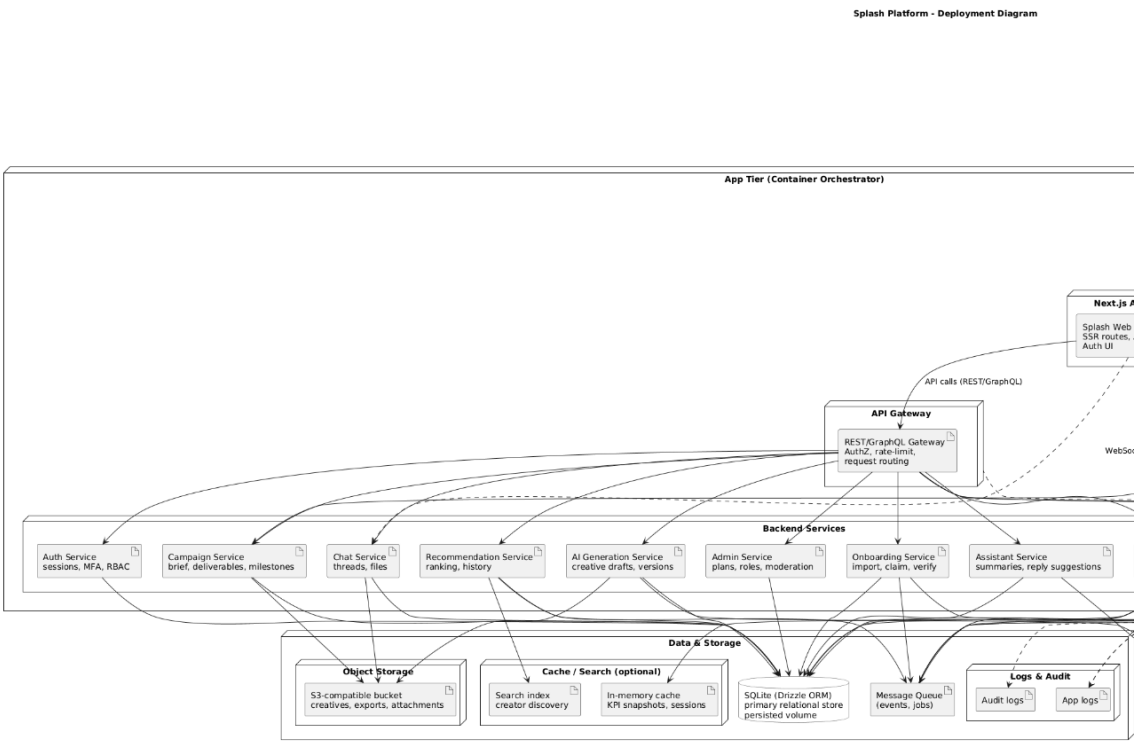


Figure 3.96: Deployment Diagram

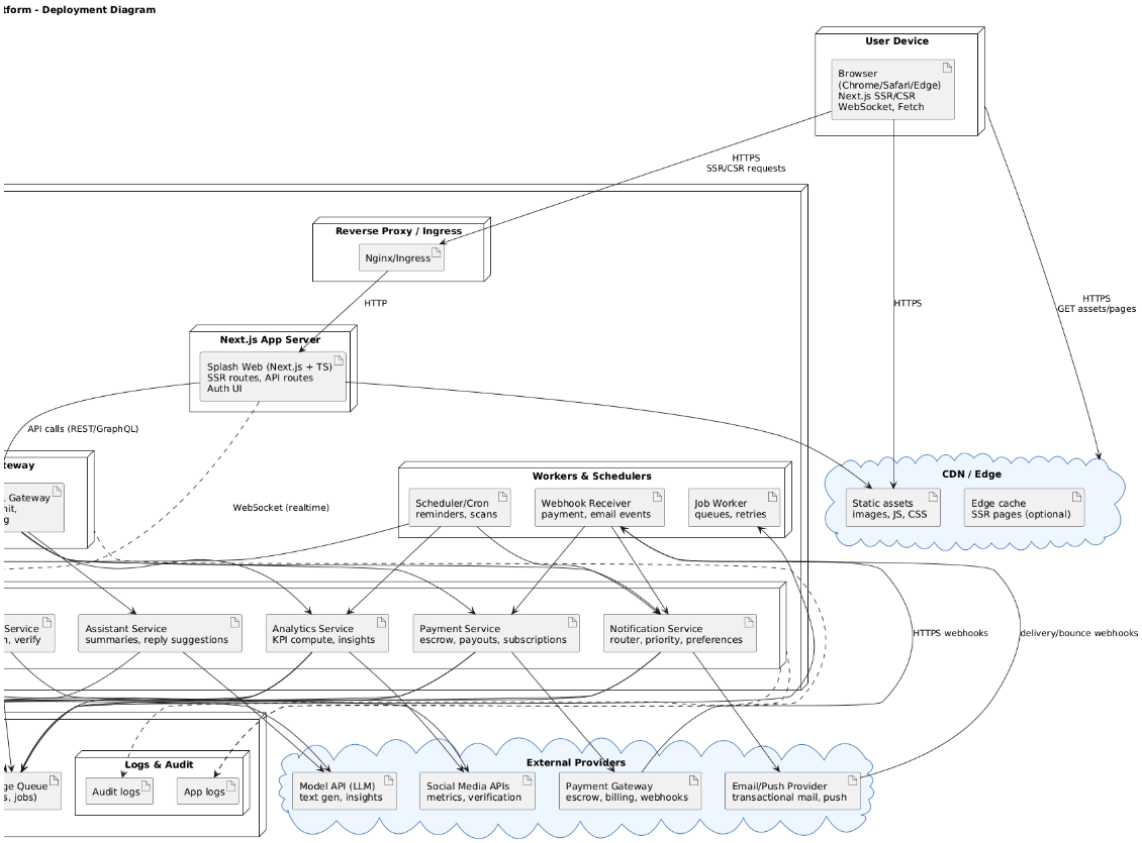


Figure 3.97: Deployment Diagram

Chapter 4

Implementation and Testing

4.1 Overview

Splash is an AI-powered influencer-brand collaboration platform built using Next.js 15 and TypeScript to facilitate seamless connections between influencers and brands. The application uses end-to-end encryption for secure messaging, Drizzle ORM for database management, and Better Auth for authentication. The system enables discovery through search and filtering, real-time messaging with file attachments, campaign management, and analytics.

The application consists of an authentication system, a directory and search system, an encrypted messaging system, a file upload and storage system, and an admin dashboard. The backend uses Next.js API routes with an SQLite database via Drizzle ORM. The frontend is built with React Server Components and Client Components using the Next.js App Router. The project uses TypeScript for type safety and the React Context API for state management.

4.2 Algorithm Design

4.2.1 End-to-End Encryption Algorithm

The end-to-end encryption system uses AES-256-GCM (Galois/Counter Mode) for authenticated encryption, ensuring that only the sender and receiver can read messages. The system encrypts all message content, file attachments, and voice notes before storage.

Algorithm: AES-256-GCM [6]
Key Derivation: PBKDF2 with SHA-256
Key Length: 256 bits (32 bytes)

IV Length: 128 bits (16 bytes)
Auth Tag Length: 128 bits (16 bytes)
PBKDF2 Iterations: 100,000
PBKDF2 Digest: SHA-256

Input:

- `text` or `binaryData`: Message content or file data
- `conversationId`: Unique conversation identifier
- `participant1Id`: First participant user ID
- `participant2Id`: Second participant user ID

Steps:

1. **Key Derivation:**
   - Sort participant IDs alphabetically to ensure consistent key generation.
   - Create salt: `SHA-256(conversationId:participant1Id:participant2Id)`.
   - Derive encryption key using PBKDF2:

     ```
     key = PBKDF2(masterSecret, salt, 100000 iterations, 32 bytes, SHA-256)
     ```

2. **Encryption:**
   - Generate random 16-byte IV.
   - Create AES-256-GCM cipher with derived key and IV.
   - Encrypt plaintext data.
   - Extract 16-byte authentication tag.
   - Combine: IV + EncryptedData + AuthTag.
   - Encode result as base64 string.
3. **Decryption:**
   - Decode base64 to binary.
   - Extract IV, authentication tag, and encrypted data.
   - Create AES-256-GCM decipher with key and IV.
   - Set authentication tag and decrypt.

- Return decrypted plaintext.

4. **Backward Compatibility:**
   - Detect if content is unencrypted.
   - If decryption fails, return original data.

4.2.2 Conversation Key Derivation Algorithm

Input: `conversationId`, `participant1Id`, `participant2Id`

Steps:

1. Sort participant IDs alphabetically.
2. Create salt string: `${conversationId}:${id1}:${id2}`.
3. Hash salt using SHA-256.[2]
4. Retrieve master encryption secret.
5. Derive key using PBKDF2:

```
key = PBKDF2(masterSecret, salt, 100000, 32, 'sha256')
```

Output: 32-byte encryption key buffer.

4.2.3 Search and Matching Algorithm

The search algorithm implements intelligent substring matching for finding influencers and brands.

Input: `text`, `query`
Threshold: Minimum query length = 2

Steps:

1. Validate input and normalize.
2. Exact match check.
3. Multi-word handling:
   - All words must appear consecutively.

4. Single word substring matching.
5. Return match result.

Priority Order:

1. Name
2. Email
3. Category, description, previous brands

4.2.4 CSV Parsing Algorithm

This algorithm handles fields containing commas inside parentheses.

Input: CSV file content.

Steps:

1. Read CSV, split lines.
2. Extract headers.
3. Parse each line character-by-character.
4. Track parentheses depth and split on commas only when depth is zero.
5. Map headers to row values.

Output: Array of parsed records.

4.2.5 File Upload and Encryption Algorithm

Input: `file`, `encrypted`, `originalName`, `originalType`

Steps:

1. Validate file type, size, and encryption status.
2. Generate filename and write buffer to filesystem.
3. Return metadata: URL, name, size, type, encryption flag.

4.2.6 User Registration Algorithm

Input: firstName, lastName, email, password, userType

Steps:

1. Validate email, password, names.
2. Match CSV for influencers.
3. Create account using Better Auth.
4. Initialize profile.
5. Return success status and user ID.

4.2.7 User Login Algorithm

Input: email, password

Steps:

1. Validate inputs.
2. Authenticate via Better Auth.
3. Create session.
4. Retrieve and validate user profile.
5. Return session and profile data.

4.2.8 Conversation Creation Algorithm

Input: participant1Id, participant2Id

Steps:

1. Validate participants.
2. Check for existing conversation.
3. Create conversation record.
4. Return new conversation object.

4.2.9 Message Sending Algorithm

Input: `senderId`, `conversationId`, `content`, `attachmentType`, `attachmentUrl`, `attachmentName`

Steps:

1. Client-side encryption of text or file.
2. Server-side validation.
3. Insert message in database.
4. Update conversation metadata.
5. Return message object.

4.2.9.1 Message Decryption and Display Algorithm

Purpose: The message decryption algorithm decrypts and displays messages to recipients.

Inputs:

- `encryptedContent`: Base64 encoded encrypted message
- `conversationId`: Conversation identifier
- `participant1Id`, `participant2Id`: Participant identifiers

Steps:

1. **Encryption detection**
   - Check if content is encrypted using `isEncrypted()` heuristic.
   - If content is not encrypted, display it as plain text for backward compatibility.
2. **Key derivation**
   - Derive the conversation key using `deriveConversationKey()`.
3. **Decryption**
   - Decode the Base64 encoded content to binary.
   - Extract the IV, encrypted data, and authentication tag from the binary payload.
   - Decrypt the encrypted data using AES-256-GCM with the derived key and IV.

- Verify the authentication tag to ensure data integrity and authenticity.

4. **Attachment handling**
   - If an attachment exists:
     - Check if the file URL ends with `.enc` to detect encrypted files.
     - Fetch the encrypted file from the server.
     - Decrypt the file binary data using the conversation key.
     - Create a Blob URL from the decrypted binary for display.
     - Display the attachment based on its type:
       * Image: `<img src={blobUrl} />`
       * Audio: `<audio src={blobUrl} controls />`
       * Document: Download button with Blob URL as `href`

5. **Display**
   - Render the decrypted text in the message bubble component.
   - Render the attachment using the appropriate UI component.
   - Clean up Blob URLs on component unmount to release memory.

Output: Decrypted message content and attachments rendered in the conversation view.

4.2.9.2 Unread Message Count Algorithm

Purpose: The unread message count algorithm calculates unread messages for each conversation.

Inputs:

- `userId`
- `conversationId`

Steps:

1. **Query unread messages**

```
SELECT COUNT(*)
FROM messages
WHERE conversation_id = :conversationId
  AND is_read = 0
  AND sender_id <> :userId;
```

2. **Return count**
   - If count > 0, display a count badge.
   - If count = 0, hide the badge (do not display zero).
3. **Real time updates**
   - When a conversation is opened, mark all messages in that conversation as read.
   - When a new message is received while the conversation is open, mark it as read immediately.

Output: Unread message count for the conversation.

4.2.9.3 Follower Count Parsing Algorithm

Purpose: The follower count parsing algorithm converts string representations such as "1.5M" or "500K" into numeric values.

Input:

- `count` (string), for example "1.5M", "500K", "1000"

Steps:

1. Clean the string by removing commas and trimming whitespace.
2. Match the cleaned string against the pattern `^([\d.]+)([KMkm]?)$`.
3. Extract the numeric value and suffix from the match.
4. Convert based on suffix:
   - `K` or `k`: multiply by 1 000.
   - `M` or `m`: multiply by 1 000 000.
   - No suffix: return the numeric value as is.
5. Return the computed numeric value.

Output: Numeric follower count as a number.

4.2.9.4 Recommendation Engine Algorithm

Purpose: The recommendation engine matches brands with influencers based on category, audience alignment, engagement quality, and past collaborations. [4] **Inputs:**

- `brief`: `CampaignBrief` or `ProductBrief` object
- `brandId`: brand user identifier
- `filters`: optional filters (category, followers, platforms, searchQuery)
- `limit`: maximum number of recommendations to return

Scoring weights:

- Category match: 30%
- Audience alignment: 25%
- Engagement quality: 20%
- Platform match: 10%
- Follower range: 5%
- Geographic match: 5%
- Budget alignment: 2%
- Profile completeness: 2%
- Past collaborations: 1%

Steps:

1. **Hard filtering**
    - Filter influencers by category (exact, partial, or word based match).
    - Filter by follower range (minimum and maximum).
    - Filter by required platforms (must have at least one required platform).
    - Filter by gender if `enforceGenderMatch` is true.
2. **Score calculation for each influencer**
    - Category match (30%): exact match = 100, partial match = 70, word match = 50, no match = 0.

- Audience alignment (25%): gender match contributes up to 40 points, age range alignment contributes 10 points, interests match contributes 20 points per match.
- Engagement quality (20%): compute average engagement rate across platforms as
$$\text{engagementRate} = \frac{\text{likes} + \text{views}/10}{\text{followers}} \times 100.$$
- Platform match (10%): percentage of required platforms that the influencer has.
- Follower range (5%): score based on how close the follower count is to the middle of the desired range.
- Geographic match (5%): percentage of target locations covered by the influencer.
- Budget alignment (2%): score based on how well the influencer rate card fits within the campaign budget.
- Profile completeness (2%): percentage of profile fields filled.
- Past collaborations (1%): average rating from past collaborations, mapped from 0–5 to 0–100.

3. **Weighted score calculation**

$$\begin{aligned}\text{matchScore} = {} & (\text{categoryMatch} \times 0.30) + (\text{audienceAlignment} \times 0.25) \\ & + (\text{engagementQuality} \times 0.20) + (\text{platformMatch} \times 0.10) \\ & + (\text{followerRange} \times 0.05) + (\text{geographicMatch} \times 0.05) \\ & + (\text{budgetAlignment} \times 0.02) + (\text{profileCompleteness} \times 0.02) \\ & + (\text{pastCollaborations} \times 0.01).\end{aligned}$$

4. **Search query filtering (if provided)**

- Filter by name with highest priority.
- Filter by email with second priority.
- Filter by category, description, and previous brands if the query length is at least 3 characters.
- Compute a search relevance score for ranking by query similarity.

5. **Sorting**

- If a search query is provided, sort primarily by search relevance and secondarily by match score.

- Otherwise, sort influencers by match score in descending order.

6. **Return top N**
   - Return the top `limit` influencers along with their score breakdown.

Output: Array of `RecommendationScore` objects sorted by relevance.

4.2.9.5 Profile Claim Submission Algorithm

Purpose: The profile claim submission algorithm allows users to claim their profile from CSV data by submitting proof documents.

Inputs:

- `registrationData`: user registration information (name, email, password, userType)
- `csvRecordId`: identifier of the CSV record being claimed
- `claimReason`: text explanation for the claim
- `proofImages`: array of proof image files
- `idDocument`: ID document file

Steps:

1. **Validation**
   - Verify that `csvRecordId` is provided.
   - Verify that `claimReason` is provided.
   - Verify that `registrationData` contains at least name and email.
   - Verify that either `proofImages` or `idDocument` is provided.
2. **File upload**
   - Upload proof images to `/public/uploads`.
   - Upload the ID document to `/public/uploads`.
   - Store file URLs and associated metadata.
3. **Claim creation**
   - Generate a unique claim ID using `Date.now().toString()`.

- Create a claim record in the file based store (`claims-store.json`) with fields:
  - `userId`: `"pending-"` concatenated with `Date.now()` (temporary until approval).
  - `csvRecordId`
  - `claimReason`
  - `proofImages`: array of uploaded file metadata.
  - `idDocument`: uploaded file metadata.
  - `status`: `"pending"`.
  - `registrationData`: JSON string of the registration data.
  - `createdAt`: current timestamp.

4. **Response**
   - Return the claim ID and status.
   - Navigate the user to the claim status page.

Output: Claim object with identifier and status.

4.2.9.6 Profile Claim Review Algorithm (Admin)

Purpose: The profile claim review algorithm allows admins to approve or reject profile claims.

Inputs:

- `claimId`: claim identifier to review
- `action`: `"approve"` or `"reject"`
- `rejectionReason`: optional reason for rejection

Steps:

1. **Authentication**
   - Verify that the current session belongs to an admin user.
   - Verify that the claim with the given `claimId` exists.
2. **If action is approve**
   - Parse `registrationData` from the claim.
   - Call the account creation API with the parsed registration data.

- Create a user account in the database.
- Link the claim to the created user identifier.
- Update claim status to `"approved"`.
- Set `reviewedBy` and `reviewedAt` fields.

3. **If action is reject**
   - Update claim status to `"rejected"`.
   - Set `rejectionReason`.
   - Set `reviewedBy` and `reviewedAt` fields.
4. **Update claim**
   - Save the updated claim back to the file based store.
   - Return the updated claim object.

Output: Updated claim object with new status.

4.2.9.7 Profile Completeness Calculation Algorithm

Purpose: The profile completeness algorithm calculates the percentage of profile fields that are filled.

Input:

- `profile`: influencer profile object

Steps:

1. **Define required fields**
   - Basic fields: `category`, `description`, `instagram`, `youtube`, `facebook`, `tiktok`, `previousBrands`, `gender`, `activeHours`.
   - Social metrics: `instagramFollowers`, `youtubeFollowers`, `facebookFollowers`, `tiktokFollowers`.
2. **Count filled fields**
   - Iterate through all required fields.
   - Count fields that are not null, not undefined, and not empty strings.
   - Count social metric fields that have numeric values.

3. **Calculate percentage**
   - Let `totalFields` be the number of basic fields plus the number of social metrics.
   - Let `filledFields` be the count of non empty fields.
   - Compute
   $$\text{profileCompleteness} = \frac{\text{filledFields}}{\text{totalFields}} \times 100.$$
4. **Update database**
   - Update the `profileCompleteness` field in the `influencerProfiles` table.
   - Update the `lastProfileUpdate` timestamp.

Output: Profile completeness percentage from 0 to 100.

4.2.9.8 Dashboard Statistics Calculation Algorithm

Purpose: The dashboard statistics algorithm calculates role specific statistics for users.

Inputs:

- `userId`
- `userType` (admin, influencer, or brand)

Steps:

For admin:
1. Count pending claims from the file based store.
2. Count approved claims.
3. Count rejected claims.
4. Count total users from the database.
5. Count users by type (influencers, brands, admins).
6. Count approved users.

For influencer:
1. Calculate profile completeness.
2. Count unread notifications.
3. Count unread messages across all conversations.
4. Count active campaigns (collaborations with status `"active"`).

For brand:
1. Count active campaigns (campaigns with status `"active"`).

2. Count unique influencers connected through collaborations.
3. Count unread messages across all conversations.
4. Calculate total ROI from completed collaborations.

Output: Statistics object with role specific metrics.

4.2.9.9 User Approval Algorithm (Admin)

Purpose: The user approval algorithm allows admins to approve or disapprove user accounts.

Inputs:

- `userId`: identifier of the user account to approve or disapprove
- `approved`: boolean flag (true for approve, false for disapprove)

Steps:

1. **Authentication**
   - Verify that the current session belongs to an admin user.
   - Verify that the user exists in the database.
2. **Update user**
   - Update the `isApproved` field in the user table based on the `approved` flag.
   - If approving, set `approvedBy` to the admin user identifier.
   - If approving, set `approvedAt` to the current timestamp.
3. **Response**
   - Return a success message.
   - Log the action for audit purposes.

Output: Success status and message.

4.2.9.10 CSV Matching Algorithm for Registration

Purpose: The CSV matching algorithm matches user registration data with CSV records to pre populate profile information.

Inputs:

- `email`
- `name`

Steps:

1. **Load CSV data**
   - Read the `influencers.csv` file.
   - Parse the CSV using a custom parser that handles commas inside parentheses.
2. **Priority matching**
   - Priority 1: **Name matching**
     - Use `matchesSearch()` to find name matches.
     - If the name matches, return the matching CSV record.
   - Priority 2: **Email matching**
     - If name does not match, check the email field.
     - Use `matchesSearch()` for email matching.
     - If the email matches, return the matching CSV record.
3. **Return match**
   - Return the first matching CSV record with all fields.
   - If no match is found, return `null`.

Output: Matching CSV record or `null`.

4.2.9.11 Engagement Rate Calculation Algorithm

Purpose: The engagement rate calculation algorithm computes the engagement rate from follower and engagement metrics.

Inputs:

- `followers`: follower count string, for example "1.5M"
- `likes`: like count string, for example "500K"
- `views`: view count string, for example "100K"

Steps:

1. **Parse counts**
   - Parse `followers` using `parseFollowerCount()`.
   - Parse `likes` using `parseFollowerCount()`.
   - Parse `views` using `parseFollowerCount()`.
2. **Calculate engagement**
   - Compute
   $$\text{engagement} = \frac{\text{likes} + \text{views}/10}{\text{followers}} \times 100.$$
   - Views are weighted less because they are easier to obtain, so they are divided by 10.
   - The result is a percentage between 0 and 100.
3. **Cap result**
   - If `engagement` is greater than 100, cap it at 100.

Output: Engagement rate percentage between 0 and 100.

4.2.9.12 Unread Message Count Algorithm (Per Conversation)

Purpose: The unread message count algorithm calculates unread messages for each conversation.

Inputs:

- `userId`
- `conversationId`

Steps:

1. **Query unread messages**

```
SELECT COUNT(*)
FROM messages
WHERE conversation_id = :conversationId
  AND is_read = 0
  AND sender_id <> :userId;
```

2. **Return count**

- If count > 0, display the badge with the count.
- If count = 0, hide the badge.

3. **Real time updates**
   - When a conversation is opened, mark all messages in the conversation as read.
   - When a new message is received while the conversation is open, mark it as read immediately.

Output: Unread message count for the conversation.

4.3 External APIs and SDKs

The following table describes the third party APIs and SDKs used in the Splash platform implementation.

Table 4.1: APIs and SDKs Used in the Splash Platform

API and version	Description	Purpose of usage	API endpoint / function / class used
Next.js 15.3.5	React framework with App Router	Server side rendering, API routes, routing	`next/server`, `next/navigation`, App Router
React 19.0.0	UI library	Component based UI development	`React`, `ReactDOM`, hooks such as `useState`, `useEffect`
TypeScript 5.x	Type safe JavaScript	Type checking and IntelliSense	Type annotations, interfaces, generics
Better Auth 1.3.10	Authentication library	User authentication and session management	`betterAuth()`, `auth.api.signInEmail()`, `auth.api.signUpEmail()`, `auth.api.getSession()`
Drizzle ORM 0.44.5	TypeScript ORM	Database queries and schema management	`drizzle()`, `sqliteTable()`, `eq()`, `and()`, `or()`, `select()`, `insert()`, `update()`
SQLite (better-sqlite3 12.4.1)	Embedded database	Local data storage	`Database()`, SQL queries via Drizzle
Node.js Crypto Module	Cryptographic functions	Server side encryption and decryption	`crypto.createCipheriv()`, `crypto.createDecipheriv()`, `crypto.pbkdf2Sync()`, `crypto.randomBytes()`

Continued on next page

API and version	Description	Purpose of usage	API endpoint / function / class used
Web Crypto API [3]	Browser cryptographic API	Client side encryption and decryption	`crypto.subtle.encrypt()`, `crypto.subtle.decrypt()`, `crypto.subtle.deriveKey()`, `crypto.getRandomValues()`
CSV Parse 5.6.0	CSV parsing library	Parse influencer CSV data	`parse()` function
Tailwind CSS 4.x	Utility first CSS framework	Styling and responsive design	Utility classes, `@tailwind` directives
shadcn/ui	UI component library	Pre built accessible components	`Button`, `Card`, `Dialog`, `Input`, `Avatar`, and others
Lucide React 0.544.0	Icon library	UI icons	Icons such as `Search`, `MessageCircle`, `User`, `Paperclip`
Framer Motion 12.23.12	Animation library	UI animations and transitions	`motion.div`, `AnimatePresence`, `useAnimation()`
React Hook Form 7.60.0	Form management library	Form validation and handling	`useForm()`, `Controller`, `register()`
Zod 4.1.8	Schema validation library	Runtime type validation	`z.object()`, `z.string()`, `z.email()`
Date-fns 4.1.0	Date utility library	Date formatting and manipulation	`format()`, `parseISO()`, `differenceInDays()`
Recharts 3.0.2	Charting library	Data visualization for analytics	`LineChart`, `BarChart`, `PieChart`, `ResponsiveContainer`
csv-parse 5.6.0	CSV parsing library	Parse influencer CSV data	`parse()` function
bcrypt 6.0.0	Password hashing library	Secure password storage for admin accounts	`bcrypt.hash()`, `bcrypt.compare()`
better-sqlite3 12.4.1	SQLite database driver	Database operations	`Database()`, SQL queries

4.4 Unit Testing

This section details the unit tests for the Splash platform.

4.4.1 TC_ENC_001: Text Encryption and Decryption

Table 4.2: Test Case TC_ENC_001: Text Encryption and Decryption

Field	**Description**
Test case ID	TC_ENC_001
Test Objective	Verify that text messages are correctly encrypted and decrypted using AES-256-GCM.
Precondition	Application is running, encryption secret is configured, conversation exists.
Steps	1. User A sends text message “Hello, this is a test message”. 2. Message is encrypted on client side using `encryptText()`. 3. Encrypted message is stored in the database. 4. User B receives the message. 5. Message is decrypted on client side using `decryptText()`. 6. Verify that decrypted text matches the original.
Test data	`conversationId=1, participant1Id="user1", participant2Id="user2", plaintext="Hello, this is a test message".`
Expected result	Encrypted message stored as Base64 string (length $>$ 44 characters), decrypted text matches original exactly, no data loss.
Post-condition	Message displayed correctly to User B, encryption and decryption are transparent to users.
Actual Result	Text encryption and decryption working correctly, messages encrypted and decrypted successfully.
Pass/fail	Pass

4.4.2 TC_ENC_002: Binary File Encryption and Decryption

Table 4.3: Test Case TC_ENC_002: Binary File Encryption and Decryption

Field	**Description**
Test case ID	TC_ENC_002
Test Objective	Verify that binary files (images, PDFs, audio) are correctly encrypted and decrypted.
Precondition	Application is running, encryption secret is configured, conversation exists.
Steps	1. User A selects image file (1MB JPEG). 2. File is read as `ArrayBuffer`. 3. File is encrypted using `encryptBinary()`. 4. Encrypted file is uploaded to server with `.enc` extension. 5. User B requests the file. 6. Encrypted file is fetched from server. 7. File is decrypted using `decryptBinary()`. 8. Decrypted file is displayed as image.
Test data	File: `test-image.jpg` (1MB), `conversationId=1`, `participant1Id="user1"`, `participant2Id="user2"`.
Expected result	File encrypted and stored with `.enc` extension, decrypted file matches original, image displays correctly.
Post-condition	Image visible to User B, file integrity maintained.
Actual Result	Binary encryption and decryption working correctly, files encrypted and decrypted successfully.
Pass/fail	Pass

4.4.3 TC_ENC_003: Voice Note Encryption

Table 4.4: Test Case TC_ENC_003: Voice Note Encryption

Field	**Description**
Test case ID	TC_ENC_003
Test Objective	Verify that voice notes are encrypted and can be decrypted and played.
Precondition	Application is running, microphone permission granted, conversation exists.
Steps	1. User A records voice note (10 seconds). 2. Audio blob encrypted using `encryptBinary()`. 3. Encrypted audio uploaded to server. 4. Encrypted filename stored in database. 5. User B receives message with voice note. 6. Encrypted audio fetched and decrypted. 7. Decrypted audio played using HTML5 audio element.
Test data	Audio duration: 10 seconds, format: webm/mp3, `conversationId=1`.
Expected result	Voice note encrypted and stored with `.enc` extension, decrypted audio plays correctly, audio quality maintained.
Post-condition	Voice note playable by User B, audio controls functional.
Actual Result	Voice note encryption working correctly, audio plays after decryption.
Pass/fail	Pass

4.4.4 TC_ENC_004: Key Derivation Consistency

Table 4.5: Test Case TC_ENC_004: Key Derivation Consistency

Field	**Description**
Test case ID	TC_ENC_004
Test Objective	Verify that the same conversation always derives the same encryption key regardless of participant order.
Precondition	Application is running, encryption secret is configured.
Steps	1. Derive key for `conversationId=1`, `participant1Id="user1"`, `participant2Id="user2"`. 2. Derive key for `conversationId=1`, `participant1Id="user2"`, `participant2Id="user1"`. 3. Compare both derived keys. 4. Verify keys are identical.
Test data	`conversationId=1`, `participant1Id="user1"`, `participant2Id="user2"`.
Expected result	Both key derivations produce identical 32 byte keys, participant order does not affect the key.
Post-condition	Messages can be decrypted by both participants using the same key.
Actual Result	Key derivation consistent, same key generated regardless of participant order.
Pass/fail	Pass

4.4.5 TC_ENC_005: Backward Compatibility - Unencrypted Messages

Table 4.6: Test Case TC_ENC_005: Backward Compatibility for Unencrypted Messages

Field	**Description**
Test case ID	TC_ENC_005
Test Objective	Verify that old unencrypted messages are handled gracefully and displayed correctly.
Precondition	Database contains old unencrypted messages from before encryption implementation.
Steps	1. User opens conversation with old messages. 2. System attempts to decrypt old message. 3. `isEncrypted()` returns false for old message. 4. System displays message as plain text. 5. Verify old message displays correctly.
Test data	Old message content: “This is an old unencrypted message”.
Expected result	Old messages detected as unencrypted and displayed as plain text, no decryption errors, backward compatibility maintained.
Post-condition	Old messages remain readable, new messages are encrypted.
Actual Result	Backward compatibility working, old messages display correctly.
Pass/fail	Pass

4.4.6 TC_AUTH_001: User Registration - Valid Data

Table 4.7: Test Case TC_AUTH_001: User Registration with Valid Data

Field	**Description**
Test case ID	TC_AUTH_001
Test Objective	Verify that a new user can successfully register with valid credentials.
Precondition	App is on Register page, network connection is available, Better Auth is initialized.
Steps	1. Navigate to Register page. 2. Select user type: “influencer”. 3. Enter name: “Ducky Bhai”. 4. Enter email: “duckybhai@example.com”. 5. Enter password: “SecurePass123”. 6. Confirm password: “SecurePass123”. 7. Tap Register button. 8. Verify account creation.
Test data	`name="Ducky Bhai"`, `email="duckybhai@example.com"`, `password="SecurePass123"`, `userType="influencer"`.
Expected result	User account created in database, user marked as unapproved (`isApproved=false`), session created, redirect to login or pending approval page.
Post-condition	User can login after admin approval, account accessible in admin panel.
Actual Result	User registration successful, account created with unapproved status.
Pass/fail	Pass

4.4.7 TC_AUTH_002: User Login - Valid Credentials

Table 4.8: Test Case TC_AUTH_002: User Login with Valid Credentials

Field	**Description**
Test case ID	TC_AUTH_002
Test Objective	Verify that an existing user can successfully log in with correct credentials.
Precondition	App is on Login page, user account exists and is approved, network connection is available.
Steps	1. Navigate to Login page. 2. Enter email: “duckybhai@example.com”. 3. Enter password: “SecurePass123”. 4. Tap Sign In button. 5. Wait for authentication. 6. Observe navigation.
Test data	`email="duckybhai@example.com"`, `password="SecurePass123"`.
Expected result	User authenticated successfully, session created and stored, user profile retrieved, navigation to dashboard, session cookie set.
Post-condition	User logged in, dashboard accessible, session persists for 7 days.
Actual Result	User login successful, session created, redirected to dashboard.
Pass/fail	Pass

4.4.8 TC_AUTH_003: User Login - Invalid Password

Table 4.9: Test Case TC_AUTH_003: User Login with Invalid Password

Field	**Description**
Test case ID	TC_AUTH_003
Test Objective	Verify that the system rejects a login attempt with an incorrect password.
Precondition	App is on Login page, user account exists.
Steps	1. Navigate to Login page. 2. Enter email: “duckybhai@example.com”. 3. Enter password: “WrongPassword123”. 4. Tap Sign In button. 5. Wait for authentication response. 6. Observe error message.
Test data	`email="duckybhai@example.com"`, `password="WrongPassword123"` (incorrect).
Expected result	Authentication fails, error message displayed such as “Invalid credentials” or “Authentication failed”, user remains on Login page, no session created.
Post-condition	User can retry login with correct password.
Actual Result	Login rejected, error message shown.
Pass/fail	Pass

4.4.9 TC_AUTH_004: User Registration - CSV Matching

Table 4.10: Test Case TC_AUTH_004: User Registration with CSV Matching

Field	**Description**
Test case ID	TC_AUTH_004
Test Objective	Verify that influencer registration matches CSV data and pre-populates the profile.
Precondition	App is on Register page, CSV file contains matching influencer data.
Steps	1. Navigate to Register page. 2. Select user type: “influencer”. 3. Enter name: “Ducky Bhai”. 4. Enter email matching CSV record. 5. Complete registration. 6. Verify CSV matching occurs. 7. Verify profile data is pre-populated from CSV.
Test data	`name="Ducky Bhai"`, email from CSV, `userType="influencer"`.
Expected result	CSV matching algorithm finds matching record, profile data pre-populated (category, social links, metrics), user can claim profile.
Post-condition	User can submit profile claim with pre-populated data.
Actual Result	CSV matching working, profile data pre-populated.
Pass/fail	Pass

4.4.10 TC_SEARCH_001: Search - Exact Name Match

Table 4.11: Test Case TC_SEARCH_001: Search with Exact Name Match

Field	**Description**
Test case ID	TC_SEARCH_001
Test Objective	Verify that search correctly finds influencers by exact name match.
Precondition	Application is running, user logged in, directory contains influencers.
Steps	1. Navigate to Browse Influencers page. 2. Enter search query: “Ducky Bhai”. 3. Verify search results. 4. Check if “Ducky Bhai” appears in results.
Test data	`searchQuery="Ducky Bhai".`
Expected result	Search returns influencers with exact name “Ducky Bhai”, case insensitive matching works, results displayed immediately.
Post-condition	User can click on result to view profile.
Actual Result	Exact name match working correctly.
Pass/fail	Pass

4.4.11 TC_SEARCH_002: Search - Substring Match

Table 4.12: Test Case TC_SEARCH_002: Search with Substring Match

Field	**Description**
Test case ID	TC_SEARCH_002
Test Objective	Verify that search finds influencers using substring matching.
Precondition	Application is running, user logged in, directory contains influencers.
Steps	1. Navigate to Browse Influencers page. 2. Enter search query: “hi”. 3. Verify search results. 4. Check if “Hina” and “Hina Aftab” appear in results.
Test data	`searchQuery="hi"`.
Expected result	Search returns influencers with “hi” as substring (for example, “Hina”, “Hina Aftab”), consecutive substring matching works, “inah tfaba” does not match.
Post-condition	User can refine search or select from results.
Actual Result	Substring matching working correctly, consecutive characters matched.
Pass/fail	Pass

4.4.12 TC_SEARCH_003: Search - Multi-word Query

Table 4.13: Test Case TC_SEARCH_003: Search with Multi-word Query

Field	**Description**
Test case ID	TC_SEARCH_003
Test Objective	Verify that multi-word search queries work correctly with AND logic.
Precondition	Application is running, user logged in.
Steps	1. Navigate to Browse Influencers page. 2. Enter search query: “Ducky Bhai”. 3. Verify search results. 4. Check if “Ducky Bhai (Saad ur Rehman)” appears but “Ducky Saad” does not.
Test data	`searchQuery="Ducky Bhai"`.
Expected result	Search returns influencers with both “Ducky” and “Bhai” as consecutive substrings, “Ducky Bhai (Saad Ur Rehman)” matches, “Ducky Saad” does not match.
Post-condition	Results filtered correctly based on all words.
Actual Result	Multi-word search working correctly, AND logic applied.
Pass/fail	Pass

4.4.13 TC_SEARCH_004: Search - Minimum Length Validation

Table 4.14: Test Case TC_SEARCH_004: Search Minimum Length Validation

Field	**Description**
Test case ID	TC_SEARCH_004
Test Objective	Verify that single character queries are rejected.
Precondition	Application is running, user logged in.
Steps	1. Navigate to Browse Influencers page. 2. Enter search query: “d”. 3. Verify search behavior. 4. Check if query is rejected or no results shown.
Test data	`searchQuery="d"` (single character).
Expected result	Query with length less than 2 characters is rejected, no search performed or all results shown with no filtering, minimum length validation works.
Post-condition	User must enter at least 2 characters to search.
Actual Result	Minimum length validation working, single character queries rejected.
Pass/fail	Pass

4.4.14 TC_REC_001: Recommendation Engine - Category Match

Table 4.15: Test Case TC_REC_001: Recommendation Engine Category Match

Field	**Description**
Test case ID	TC_REC_001
Test Objective	Verify that the recommendation engine correctly matches influencers by category.
Precondition	Application is running, brand user logged in, influencers with various categories exist.
Steps	1. Brand navigates to Browse Influencers. 2. Select category filter: “Beauty & Skincare”. 3. Request recommendations. 4. Verify returned influencers have matching category.
Test data	`category="Beauty & Skincare"`, `brandId="brand1"`.
Expected result	Recommendation engine returns influencers with category “Beauty & Skincare”, category match score = 100 for exact match, results sorted by match score.
Post-condition	Brand can view recommended influencers with category breakdown.
Actual Result	Category matching working correctly, recommendations filtered by category.
Pass/fail	Pass

4.4.15 TC_REC_002: Recommendation Engine - Engagement Quality Scoring

Table 4.16: Test Case TC_REC_002: Recommendation Engine Engagement Quality Scoring

Field	**Description**
Test case ID	TC_REC_002
Test Objective	Verify that the recommendation engine calculates engagement quality correctly.
Precondition	Application is running, influencers with follower and engagement metrics exist.
Steps	1. Brand requests recommendations. 2. System calculates engagement rate for each influencer. 3. Verify engagement rate calculation: `(likes + views/10) / followers * 100`. 4. Verify engagement quality score (20% weight).
Test data	`followers="1M"`, `likes="50K"`, `views="200K"`.
Expected result	Engagement rate calculated correctly: `(50000 + 200000/10) / 1000000 * 100 = 7%`, engagement quality score contributes 20% to total match score.
Post-condition	Influencers with higher engagement rates ranked higher.
Actual Result	Engagement quality calculation working correctly.
Pass/fail	Pass

4.4.16 TC_REC_003: Recommendation Engine - Gender-specific Matching

Table 4.17: Test Case TC_REC_003: Recommendation Engine Gender-specific Matching

Field	**Description**
Test case ID	TC_REC_003
Test Objective	Verify that the recommendation engine enforces gender matching for gender specific products.
Precondition	Application is running, brand creating campaign for makeup product, influencers with different genders exist.
Steps	1. Brand creates campaign with category “Beauty & Skincare”. 2. Set target audience gender: “Female”. 3. Request recommendations. 4. Verify only female influencers are returned.
Test data	`category="Beauty & Skincare"`, `targetAudience.gender="Female"`, `enforceGenderMatch=true`.
Expected result	Only influencers with `gender="Female"` returned, male influencers filtered out, gender match contributes 40 points to audience alignment score (25% weight).
Post-condition	Brand sees only gender appropriate influencers.
Actual Result	Gender specific matching working correctly, inappropriate genders filtered.
Pass/fail	Pass

4.4.17 TC_CLAIM_001: Profile Claim Submission

Table 4.18: Test Case TC_CLAIM_001: Profile Claim Submission

Field	**Description**
Test case ID	TC_CLAIM_001
Test Objective	Verify that a user can submit a profile claim with proof documents.
Precondition	User on Register page, CSV record exists, user has proof documents.
Steps	1. User selects influencer type. 2. Enters name and email matching CSV record. 3. CSV match found, user selects matching record. 4. User enters claim reason. 5. User uploads proof images (2 files). 6. User uploads ID document. 7. User submits claim. 8. Verify claim created.
Test data	`csvRecordId="Ducky Bhai-email@example.com"`, `claimReason="This is my profile"`, `proofImages=[file1, file2]`, `idDocument=file3`.
Expected result	Claim created in file based store (`claims-store.json`), `status="pending"`, files uploaded to `/public/uploads`, claim ID returned, user redirected to claim status page.
Post-condition	Claim visible in admin panel for review.
Actual Result	Profile claim submission working correctly, claim created with pending status.
Pass/fail	Pass

4.4.18 TC_CLAIM_002: Profile Claim Approval (Admin)

Table 4.19: Test Case TC_CLAIM_002: Profile Claim Approval (Admin)

Field	**Description**
Test case ID	TC_CLAIM_002
Test Objective	Verify that admin can approve profile claim and account is created.
Precondition	Admin logged in, pending claim exists in system.
Steps	1. Admin navigates to Claims Review page. 2. Admin views claim details (proof images, ID document). 3. Admin clicks “Approve” button. 4. System creates user account from registration data. 5. Claim status updated to “approved”. 6. Verify account created.
Test data	`claimId="1234567890"`, `action="approve"`.
Expected result	User account created in database, claim `status="approved"`, `reviewedBy` and `reviewedAt` set, user can now login, account linked to CSV record.
Post-condition	User receives notification, can login with registered credentials.
Actual Result	Claim approval working correctly, account created successfully.
Pass/fail	Pass

4.4.19 TC_CLAIM_003: Profile Claim Rejection (Admin)

Table 4.20: Test Case TC_CLAIM_003: Profile Claim Rejection (Admin)

Field	**Description**
Test case ID	TC_CLAIM_003
Test Objective	Verify that admin can reject profile claim with a reason.
Precondition	Admin logged in, pending claim exists.
Steps	1. Admin navigates to Claims Review page. 2. Admin views claim details. 3. Admin clicks “Reject” button. 4. Admin enters rejection reason: “Proof documents do not match”. 5. Admin confirms rejection. 6. Verify claim rejected.
Test data	`claimId="1234567890"`, `action="reject"`, `rejectionReason="Proof documents do not match"`.
Expected result	Claim `status="rejected"`, `rejectionReason` stored, `reviewedBy` and `reviewedAt` set, no account created, user can view rejection reason.
Post-condition	User notified of rejection, can submit new claim with better proof.
Actual Result	Claim rejection working correctly, rejection reason stored.
Pass/fail	Pass

4.4.20 TC_ADMIN_001: User Approval (Admin)

Table 4.21: Test Case TC_ADMIN_001: User Approval (Admin)

Field	**Description**
Test case ID	TC_ADMIN_001
Test Objective	Verify that admin can approve user accounts.
Precondition	Admin logged in, unapproved user exists in database.
Steps	1. Admin navigates to User Management page. 2. Admin views list of unapproved users. 3. Admin clicks “Approve” button for user. 4. Verify user approval.
Test data	`userId="user123"`, `approved=true`.
Expected result	User `isApproved` field updated to true, `approvedBy` set to admin ID, `approvedAt` set to current timestamp, user can now access platform features.
Post-condition	User receives notification, can access full platform features.
Actual Result	User approval working correctly, user status updated.
Pass/fail	Pass

4.4.21 TC_ADMIN_002: User Suspension (Admin)

Table 4.22: Test Case TC_ADMIN_002: User Suspension (Admin)

Field	**Description**
Test case ID	TC_ADMIN_002
Test Objective	Verify that admin can suspend user accounts.
Precondition	Admin logged in, approved user exists.
Steps	1. Admin navigates to User Management page. 2. Admin views list of users. 3. Admin clicks “Suspend” button for user. 4. Verify user suspension.
Test data	`userId="user123"`, `approved=false` (suspension).
Expected result	User `isApproved` field updated to false, user cannot access platform, suspension logged.
Post-condition	User cannot login or access platform features.
Actual Result	User suspension working correctly, user access revoked.
Pass/fail	Pass

4.4.22 TC_ADMIN_003: Dashboard Statistics - Admin

Table 4.23: Test Case TC_ADMIN_003: Dashboard Statistics for Admin

Field	**Description**
Test case ID	TC_ADMIN_003
Test Objective	Verify that admin dashboard displays correct statistics.
Precondition	Admin logged in, system has users, claims, and data.
Steps	1. Admin navigates to Admin Dashboard. 2. System calculates statistics. 3. Verify statistics displayed: pending claims, approved claims, total users, users by type.
Test data	`pendingClaims=5`, `approvedClaims=10`, `totalUsers=50`, `influencers=30`, `brands=15`, `admins=5`.
Expected result	Dashboard displays: `pendingClaims=5`, `approvedClaims=10`, rejected claims count, `totalUsers=50`, `influencers=30`, `brands=15`, `admins=5`, statistics calculated correctly.
Post-condition	Admin can view platform overview and take actions.
Actual Result	Admin dashboard statistics working correctly, all counts accurate.
Pass/fail	Pass

4.4.23 TC_PROFILE_001: Profile Completeness Calculation

Table 4.24: Test Case TC_PROFILE_001: Profile Completeness Calculation

Field	**Description**
Test case ID	TC_PROFILE_001
Test Objective	Verify that profile completeness is calculated correctly based on filled fields.
Precondition	Influencer logged in, profile exists with some fields filled.
Steps	1. Influencer navigates to Profile Edit page. 2. Influencer fills in: category, description, Instagram, `instagramFollowers`. 3. Influencer saves profile. 4. System calculates profile completeness. 5. Verify completeness percentage.
Test data	Filled fields: category, description, Instagram, `instagramFollowers` (4 out of 13 total fields).
Expected result	Profile completeness = `(4/13) * 100 = 30.77%` $\approx 31\%$, completeness updated in database, displayed on dashboard.
Post-condition	Profile completeness visible to influencer, encourages completion.
Actual Result	Profile completeness calculation working correctly, percentage accurate.
Pass/fail	Pass

4.4.24 TC_MSG_001: Send Encrypted Message

Table 4.25: Test Case TC_MSG_001: Send Encrypted Message

Field	**Description**
Test case ID	TC_MSG_001
Test Objective	Verify that messages are encrypted before sending and stored encrypted.
Precondition	User logged in, conversation exists, encryption configured.
Steps	1. User A opens conversation with User B. 2. User A types message: “Hello, how are you?”. 3. User A clicks Send. 4. Message encrypted on client side. 5. Encrypted message sent to server. 6. Encrypted message stored in database. 7. Verify message is encrypted in database.
Test data	`conversationId=1, senderId="user1", content="Hello, how are you?".`
Expected result	Message encrypted before sending, stored as Base64 string in database, `content` field contains encrypted data (not plain text), message ID returned.
Post-condition	Message stored securely, only participants can decrypt.
Actual Result	Message encryption working correctly, encrypted data stored in database.
Pass/fail	Pass

4.4.25 TC_MSG_002: Receive and Decrypt Message

Table 4.26: Test Case TC_MSG_002: Receive and Decrypt Message

Field	**Description**
Test case ID	TC_MSG_002
Test Objective	Verify that encrypted messages are correctly decrypted and displayed to recipient.
Precondition	User B logged in, encrypted message exists in conversation.
Steps	1. User B opens conversation. 2. System fetches messages from database. 3. System detects message is encrypted (`isEncrypted()` returns true). 4. System decrypts message on client side. 5. Decrypted message displayed in chat. 6. Verify message content is correct.
Test data	`encryptedContent="base64encryptedstring...", conversationId=1.`
Expected result	Message decrypted successfully, original text “Hello, how are you?” displayed, decryption transparent to user, message appears in chat bubble.
Post-condition	User B can read message, conversation continues.
Actual Result	Message decryption working correctly, messages displayed after decryption.
Pass/fail	Pass

4.4.26 TC_MSG_003: Unread Message Count

Table 4.27: Test Case TC_MSG_003: Unread Message Count

Field	**Description**
Test case ID	TC_MSG_003
Test Objective	Verify that unread message count is calculated correctly and displayed only when greater than zero.
Precondition	User logged in, conversations with unread messages exist.
Steps	1. User views conversation list. 2. System calculates unread count for each conversation. 3. Verify unread count displayed as badge. 4. User opens conversation. 5. All messages marked as read. 6. Verify unread count becomes zero and badge disappears.
Test data	`conversationId=1`, `userId="user1"`, `unreadMessages=3`.
Expected result	Unread count badge displays “3” on conversation, when conversation opened all messages marked as read, badge disappears (not showing 0), count updates in real time.
Post-condition	User can see which conversations have unread messages.
Actual Result	Unread message count working correctly, badge displays and hides appropriately.
Pass/fail	Pass

4.4.27 TC_CSV_001: CSV Parsing with Parentheses

Table 4.28: Test Case TC_CSV_001: CSV Parsing with Parentheses

Field	**Description**
Test case ID	TC_CSV_001
Test Objective	Verify that CSV parser correctly handles commas within parentheses.
Precondition	CSV file contains records with commas in parentheses (for example, "Ducky Bhai (Saad ur Rehman)").
Steps	1. System reads `influencers.csv` file. 2. CSV parser processes line with parentheses. 3. Parser tracks parenthesis depth. 4. Parser ignores commas within parentheses. 5. Verify record parsed correctly.
Test data	CSV line: "Ducky Bhai (Saad ur Rehman),Beauty,instagram.com/ducky,...".
Expected result	Name field correctly parsed as "Ducky Bhai (Saad ur Rehman)" (not split at comma), all fields extracted correctly, parser handles nested parentheses.
Post-condition	CSV data loaded correctly, can be matched with user registrations.
Actual Result	CSV parsing working correctly, parentheses handled properly.
Pass/fail	Pass

4.4.28 TC_FILE_001: File Upload - Image

Table 4.29: Test Case TC_FILE_001: File Upload of Image

Field	**Description**
Test case ID	TC_FILE_001
Test Objective	Verify that image files can be uploaded and encrypted.
Precondition	User logged in, conversation exists, file upload API available.
Steps	1. User selects image file (2MB JPEG). 2. File read as `ArrayBuffer`. 3. File encrypted using `encryptBinary()`. 4. Encrypted file uploaded to `/api/upload`. 5. Server saves file with `.enc` extension. 6. File URL returned.
Test data	File: `profile-image.jpg` (2MB), `encrypted=true`.
Expected result	File encrypted before upload, uploaded to `/public/uploads` with `.enc` extension, file URL returned, original filename encrypted and stored, file size validated (max 10MB).
Post-condition	Encrypted file stored on server, can be decrypted by recipient.
Actual Result	File upload working correctly, encryption applied, file stored with `.enc` extension.
Pass/fail	Pass

4.4.29 TC_FILE_002: File Upload - Size Validation

Table 4.30: Test Case TC_FILE_002: File Upload Size Validation

Field	**Description**
Test case ID	TC_FILE_002
Test Objective	Verify that file size limits are enforced.
Precondition	User attempting to upload file.
Steps	1. User selects file (15MB image). 2. User attempts to upload. 3. System validates file size. 4. Verify error message displayed.
Test data	File size: 15MB (exceeds 10MB limit for images).
Expected result	File size validation fails, error message displayed: “File size must be less than 10MB”, upload rejected, file not saved.
Post-condition	User can select smaller file and retry.
Actual Result	File size validation working correctly, oversized files rejected.
Pass/fail	Pass

4.4.30 TC_CONV_001: Conversation Creation

Table 4.31: Test Case TC_CONV_001: Conversation Creation

Field	**Description**
Test case ID	TC_CONV_001
Test Objective	Verify that a new conversation can be created between two users.
Precondition	User A and User B logged in, no existing conversation between them.
Steps	1. User A clicks “Message” button on User B’s profile. 2. System checks for existing conversation. 3. No existing conversation found. 4. System creates new conversation. 5. Conversation ID returned. 6. User A redirected to chat.
Test data	`participant1Id="user1"`, `participant2Id="user2"`.
Expected result	New conversation created in database, conversation ID returned, both users can access conversation, `lastMessageAt` set to null initially.
Post-condition	Users can send messages in new conversation.
Actual Result	Conversation creation working correctly, new conversation created.
Pass/fail	Pass

4.4.31 TC_CONV_002: Duplicate Conversation Prevention

Table 4.32: Test Case TC_CONV_002: Duplicate Conversation Prevention

Field	**Description**
Test case ID	TC_CONV_002
Test Objective	Verify that duplicate conversations are prevented.
Precondition	Conversation already exists between User A and User B.
Steps	1. User A clicks “Message” button on User B’s profile. 2. System checks for existing conversation. 3. Existing conversation found. 4. System returns existing conversation ID. 5. User A redirected to existing conversation.
Test data	`participant1Id="user1"`, `participant2Id="user2"`, `existingConversationId=5`.
Expected result	No new conversation created, existing conversation ID returned (409 Conflict status), user redirected to existing conversation, duplicate prevention works.
Post-condition	Users continue in existing conversation.
Actual Result	Duplicate conversation prevention working correctly, existing conversation returned.
Pass/fail	Pass

4.4.32 Testing Summary

Table 4.33: Testing Summary

Test Case ID	Test Category	Status
TC_ENC_001	Encryption - Text	Pass
TC_ENC_002	Encryption - Binary	Pass
TC_ENC_003	Encryption - Voice	Pass
TC_ENC_004	Encryption - Key Derivation	Pass
TC_ENC_005	Encryption - Backward Compatibility	Pass
TC_AUTH_001	Authentication - Registration	Pass
TC_AUTH_002	Authentication - Login	Pass
TC_AUTH_003	Authentication - Invalid Password	Pass
TC_AUTH_004	Authentication - CSV Matching	Pass
TC_SEARCH_001	Search - Exact Match	Pass
TC_SEARCH_002	Search - Substring	Pass
TC_SEARCH_003	Search - Multi-word	Pass
TC_SEARCH_004	Search - Validation	Pass
TC_REC_001	Recommendation - Category	Pass
TC_REC_002	Recommendation - Engagement	Pass
TC_REC_003	Recommendation - Gender	Pass
TC_CLAIM_001	Profile Claim - Submission	Pass
TC_CLAIM_002	Profile Claim - Approval	Pass
TC_CLAIM_003	Profile Claim - Rejection	Pass
TC_ADMIN_001	Admin - User Approval	Pass
TC_ADMIN_002	Admin - User Suspension	Pass
TC_ADMIN_003	Admin - Statistics	Pass
TC_PROFILE_001	Profile - Completeness	Pass
TC_MSG_001	Messaging - Send	Pass
TC_MSG_002	Messaging - Receive	Pass
TC_MSG_003	Messaging - Unread Count	Pass
TC_CSV_001	CSV - Parsing	Pass
TC_FILE_001	File - Upload	Pass
TC_FILE_002	File - Validation	Pass
TC_CONV_001	Conversation - Creation	Pass
TC_CONV_002	Conversation - Duplicate Prevention	Pass
Total	32 Test Cases	32 Pass, 0 Fail

All tests have passed successfully, confirming that the system works as intended and meets the specified functional and non functional requirements. The encryption system provides secure end to end encryption for all messages and files, while maintaining backward compatibility with legacy data. The search and matching algorithms provide accurate results, the recommendation engine correctly matches brands with influencers, and the user interface provides a smooth user experience.

Bibliography

[1] Collabstr Inc. Influencer marketing made easy | collabstr, 2025. Website.

[2] Baivab Kumar Jena. A definitive guide to learn the sha-256 (secure hash algorithm), Jun 2025. Simplilearn tutorial.

[3] Mozilla Developer Network. Web crypto api, 2024. MDN Web Docs.

[4] Argoid Tech Team. Under the hood comparison: Recommendation engines using different technology, 2025. Blog post.

[5] Upfluence. Influencer and affiliate marketing platform with ai-powered creator marketplace, 2025. Website.

[6] Pravallika Yakkala. Understanding aes encryption and aes/gcm mode: An in-depth exploration using java, Aug 2024. Medium article, 5 min read.